▼リスト1　オプション設定（tsconfig.json）

```
{
  "compilerOptions": {
    "rootDirs": ["src"],
    "outDir": "dist",
    "lib": ["es2020"],
    "target": "es2020",
    "module": "esnext",
    "moduleResolution": "node",
    "esModuleInterop": true,
  }
}
```

▼リスト2　package.jsonを書き換える

```
"scripts": {
  "transpile": "tsc",
  "start": "npm run transpile && node ./dist/index.js"
}
```

▼リスト4　スキーマの読み込み（src/index.ts）

```
const schema = loadSchemaSync('schema.graphql',
  { loaders: [new GraphQLFileLoader()] }
);
```

▼リスト3　スキーマを定義する（schema.graphql）

```
type PostTag {
  name: String!
}

type Post {
  id: ID!
  title: String!
  body: String!
  tags: [PostTag!]!
}

interface User {
  name: String!
}

type Author implements User {
  id: ID!
  name: String!
  posts: [Post!]!
}

type Query {
  authors: [Author!]!
}
```

スキーマの定義

本稿では3-2で定義したGraphQLスキーマの一部を利用します。Queryにauthors: [Author!]!を定義することで、クライアントはauthorsというクエリの実行が可能になり、このクエリが実行されたときにGraphQLサーバは[Author!]!型の値を返します。

まずスキーマを定義します（**リスト3**）。定義したgraphql-toolsからimportしたloadSchemaSyncとGraphQLFileLoaderを利用してスキーマを読み込みます（**リスト4**）。

次にサンプルデータを用意します（**リスト5**）。これは、クライアントがGraphQLサーバにクエリを実行したときに取得可能なサンプルデータです。例としてサンプルデータをハードコーディングしていますが、実際のGraphQLサーバでは、データベース（DB）やほかのサーバなどにデータソースがあることが想定されます。

リゾルバーの実装

スキーマ（**リスト3**）とサンプルデータ（**リスト5**）を用意しました。しかし、スキーマはデー

▼リスト5　サンプルデータの用意（src/database.ts）

```
export const db = {
  blogs: {
    1: {
      title: "Signed Query は GraphQL の Trusted Document ➤
 の新しい実装パターンです",
      author: "YutaUra",
      body: `こんにちは。スタディサプリの小中新規開発チームで➤
 Web エンジニアをしている @YutaUra で(..略..)`,
      tags: ["GraphQL"],
    },
    (..略..)
  },
  users: {
    YutaUra: {
      name: "YutaUra",
    },
    (..略..)
  },
};
```

タの形を定義しますが、データの取得方法は定義しません。それぞれの型やフィールドでデータを取得するのはリゾルバーの役割です。

リゾルバーは、**GraphQLの各オペレーション（Query/Mutation/Subscription）と、それぞれのtypeのフィールドやスカラーで、どのような処理を行いどのような値を返すのか**を定義します。また、それぞれのリゾルバーは関数であり次の4つの引数を受け取れます。

- **parent：**
  - 親のリゾルバーの戻り値
  - ルートリゾルバー[注4]直下のリゾルバーの場合は空になる
- **args：**
  - スキーマで定義したフィールドの呼び出し引数
- **context：**
  - GraphQLオペレーション全体で共有されるオブジェクト
  - DBへの接続情報やユーザーの認証情報など任意のデータを持たせられる
- **info：**
  - その他の実行されるオペレーションに関する詳細情報
  - デバッグやエラー通知に含めるメッセージの生成などに使用される

さっそくリゾルバーを定義します（**リスト6**）。各フィールドで返却する値は、スキーマで定義した型と一致させます。たとえば、Queryのauthorsフィールドの戻り値の型はAuthor型の配列か空配列である必要があります。**リスト6**によって、スキーマのそれぞれの型とフィールドに対応するデータの取得方法を定義できます。

▼リスト6　リゾルバーの定義（src/index.ts）

```
const resolvers = {
  Query: {
    authors: () =>
      Object.entries(db.users).map(([id, user]) => ({ id, ...user })),
  },
  Post: {
    tags: ({ tags }) => tags.map((name) => ({ name })),
  },
  Author: {
    posts: ({ id }) =>
      Object.entries(db.blogs)
        .filter(([_, post]) => post.author === id)
        .map(([id, post]) => ({ id, ...post })),
  },
};
```

default resolver

リゾルバーはスキーマで定義したすべての型やそのフィールドに対して定義できます。つまり、QueryやMutationに定義したフィールドのみならず、PostやAuthor.nameに対してもリゾルバーを定義できます。この性質により、GraphQLは非常に高い柔軟性を持ちます。

リスト6で定義したリゾルバーを見ると、Post.titleやAuthor.nameに対してリゾルバーを定義していないことがわかります。**リスト7**のようにそれぞれのフィールドでリゾルバーが必要になるのではないかと思うかもしれません。

しかし、すべてのtypeやフィールドに対してリゾルバーを書く必要はありません。書かずともクエリは正常に動きます。

GraphQLでは、リゾルバーが定義されていないときに**default resolver**というしくみが動作します。親のオブジェクト（parent）がそれぞれの型のフィールドと同名のプロパティを持っているとき、default resolverによってそのオブジェクトが展開されます。

Apollo Serverの起動と実行

Apollo Serverの起動

スキーマ、リゾルバーを定義したことで、Apollo Serverを起動するために必要なものは

注4）Query/Mutation/Subscription直下のフィールドのリゾルバーのこと。

▼リスト7　未定義のフィールドのリゾルバーを定義する

```
const resolvers = {
  Query: { (..略..) },
  Post: { title: (parent) => { return parent.title } },
  Author: { name: (parent) => { return parent.name } },
};
```

▼リスト8　サーバの起動（src/index.tsへ追記）

```
const schemaWithResolvers = addResolversToSchema({ schema, resolvers });
const apolloServer = new ApolloServer({ schema: schemaWithResolvers });
const { url } = await startStandaloneServer(apolloServer);

console.log(`💻 Apollo Server started: ${url}`);
```

▼リスト9　GraphQLサーバに対して送信するクエリ

```
query GetAuthors {
  authors {
    name
    posts {
      title
      body
      tags {
        name
      }
    }
  }
}
```

そろいました。また、Apollo Serverのインスタンスを startStandaloneServer()関数に渡すことで、アプリケーションの起動時にmiddlewareにApolloServerを差し込むことができます。

サーバを起動するためのコードをsrc/index.tsの末尾に追記します（**リスト8**）。

npm startコマンドでサーバを起動します。サーバが正常に起動すると**図2**のように出力されます。

▼図2　サーバの起動結果

```
$ npm start
💻 Apollo Server started: http://localhost:4000/
```

クエリの実行

クライアントからサーバにクエリを送信します。http://localhost:4000にアクセスすると、Apollo Sandbox[注5]というプレイグラウンド環境が起動しています。起動した画面の左には、実行可能なクエリやレスポンスに含まれるパラメータがフィールドとして定義されています。

GraphQLサーバに対して**リスト9**のクエリを送信します。画面の［GetAuthors］をクリックし、サーバにリクエストします。画面右のResponseにSTATUS 200やレスポンスパラメータが表示されたら成功です。

◆ ◆ ◆

以上がGraphQLとApollo Serverを使用したサーバサイドの実装例です。実際に本番環境に導入するときは、DBやほかのサーバから取得したデータを返すようなリゾルバーを定義することになるはずです。

resolver chainsのしくみと特徴

先ほど定義したリゾルバーはどのように実行されるのでしょうか。

それぞれのリゾルバーには親のリゾルバーが存在します。親のリゾルバーから子のリゾルバーへ鎖のようにつなぎ、最終的にスキーマの最下層のリゾルバーまで処理が連鎖するしくみや、それぞれのリゾルバーが実行される順序のことをApolloでは**resolver chains**と呼びます。

resolver chainsはスキーマの定義に従い、パーサーによって生成されるAbstract syntax tree（AST）[注6]をもとに自動的に解決します。このしくみによって、GraphQLサーバはクライアントが指定したクエリを正しい順序で取得して返すことを保証します。

実際にパーサーによって生成されるASTはhttps://astexplorer.net/でクエリを入力することで確認できます。また、GraphQLサーバはASTとスキーマを比較しクエリの有効性を検

注5）URL https://www.apollographql.com/docs/graphos/platform/sandbox

注6）URL https://en.wikipedia.org/wiki/Abstract_syntax_tree

▼リスト10　実行するクエリ例①

```
query GetPosts {
  authors {
    posts {
      tags {
        name
      }
    }
  }
}
```

▼リスト11　実行するクエリ②

```
query GetPosts {
  authors {
    name
    posts {
      title
      body
      tags {
        name
      }
    }
  }
}
```

証します。無効な場合、リゾルバーを実行することなくクライアントにエラーを返します。

詳しく見ていきましょう。例として**リスト10**のクエリを実行したとします。このとき、まずルートリゾルバー直下のフィールドであるauthorsフィールドのリゾルバーが実行されます。そのあと、Author型のpostsフィールドのリゾルバーが実行され、Post型のtagsフィールドのリゾルバーが実行され……、と続き、最終的にPostTag型のnameフィールドのリゾルバーが実行されます。

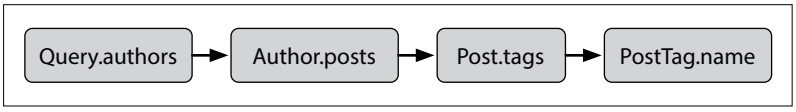
図3を見るとわかりやすいかもしれません。左のリゾルバーが親で右のリゾルバーが子です。postsフィールドを解決するリゾルバーには、引数parentにAuthor型のオブジェクトがauthorsフィールドを解決したリゾルバーから渡ってきます。

次に、Author.name、Post.title、Post.body、PostTag.nameも追加で取得したくなったとします。**リスト11**のクエリを実行するとresolver chainsはどうなるでしょうか。この場合、各フィールドの深さごとにresolver chainsが分岐します（**図4**）。さらに、分岐したそれぞれのサブチェーンのリゾルバーは並列で実行されます。

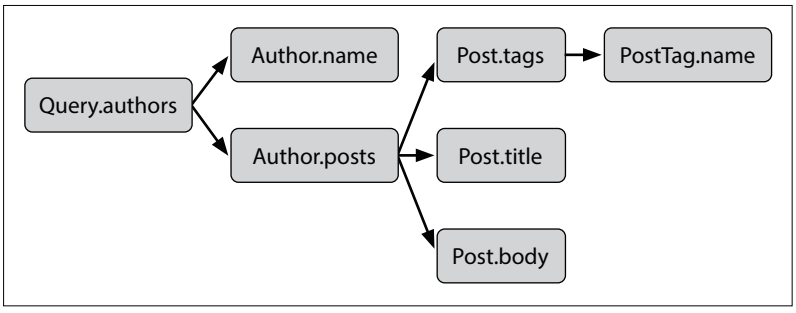
それぞれのリゾルバーはresolver chainsによって順序立って実行され、最終的に各リゾルバーの実行結果を収集したJSONオブジェクトをクライアントに返します。

このresolver chainsによって、GraphQLサーバの実装は非常に楽でシンプルなものになります。一度、型やリゾルバーを実装するとそれらは再利用可能になります。新たなフィールドで定義済みの型やリゾルバーを再利用するとき、少ないソースコードで機能を実現できます。つまり、実装時間が短縮され、ソースコードをクリーンに保ちやすくなります。

▼図3　resolver chainsのイメージ



▼図4　resolver chainsが分岐する



GraphQLサーバで発生しやすいN+1という問題

GraphQLではN＋1という問題が発生しやすいとされています。GraphQLはクエリ全体をどのように解決するかという問いに対して、それぞれのフィールドのリゾルバーが解決しそれを収集するというシンプルな解を出しています。その自然さがゆえに、リゾルバーが複数回呼ばれる可能性があるということをしばしば忘れさせてしまいます。

例として、先ほど

▼リスト12　N＋1問題が発生する例

```
const resolvers = {
  Query: {
    // すべてのAuthorを取得するSQLを発行する関数
    authors: () => { return getAllAuthors() },
  },

  Author: {
    // 指定されたidのAuthorのすべてのPostを取得するSQLを発行する関数
    posts: (parent) => { return getPostsByAuthorId(parent.id) },
  },
  Post: {
    // 指定されたidのPostのすべてのPostTagを取得するSQLを発行する関数
    tags: (parent) => { return getTagsByPostId(parent.id) },
  },
  PostTag: {
    // 指定されたidのPostTagのnameを取得するSQLを発行する関数
    name: (parent) => { return getNameByPostTagId(parent.id) },
  }
};
```

のリスト10のクエリが実行された場合を見てみます。GraphQLサーバでは次のようにそれぞれのリゾルバーが実行されます。

1. Query.authors フィールドのリゾルバーが1回実行される
2. Author.posts フィールドのリゾルバーがN（authors.count）回実行される
3. Post.tags フィールドのリゾルバーがN×M（posts.count）回実行される
4. PostTag.name フィールドのリゾルバーがN×M×L（tags.count）回実行される

ここまで、ハードコードしたデータオブジェクトのプロパティを参照していましたが、仮にそれぞれのリゾルバーがリスト12のようにデータソースへのアクセスを必要とするものだった場合にはどうでしょう。

GraphQLはリゾルバーを個別にかつ再帰的に実行するため、リゾルバーを愚直に実装するとN＋1問題が発生します。さらに、それぞれのフィールドの数だけN×M×L回のデータアクセスを実行することになるため、N＋1回どころではなく非常にクリティカルな問題であることがわかります。

このGraphQLのN＋1問題を解決する方法として、遅延読み込みによるバッチ処理というアプローチが有効です。GraphQLではDataLoaderというライブラリが遅延読み込みを行うための機能を提供しており、これを利用することが一般的です。

DataLoaderのしくみと特徴

DataLoader[注7]はアプリケーションのデータフェッチレイヤの一部を担い、効率的なデータの取得を目的とした汎用的なライブラリであり、N＋1問題の解決パターンの名前です。このライブラリはBatchとCacheという2つの機構をシンプルなAPIで公開しています。

DataLoaderは次の機能を提供しています。

- Batchによる非同期かつ一括で行うデータアクセス
- Cacheによる（Key-Value Storeへの）データのメモ化

Batchはデータを取得するためのデータソースへのアクセスを一定期間待機し、その間に行われたリクエストをまとめて処理するための機構です。Batch Functionと呼ばれる関数を定義

注7）URL https://github.com/graphql/dataloader

▼リスト13　Postを取得するためのBatch Function

```
class PostsDataSource {
  // keysで指定されたauthor_idを持つpostの配列を返す
  private loader = new DataLoader(async (keys: number[]) => {
    // 実際のGraphQLサーバで発行されることが想定されるクエリをログに出力する
    console.log(`SELECT * FROM post WHERE author_id IN (${keys.join(",")})`);
    const results = posts.filter(post => keys.includes(post.author_id));

    return keys.map((key: number) => results.filter(post => post.author_id === key));
  });
}
```

▼リスト14　Tagを取得するためのBatch Function

```
class TagsDataSource {
  // keysで指定されたpost_idを持つtagの配列を返す
  private loader = new DataLoader(async (keys: number[]) => {
    // 実際のGraphQLサーバで発行されることが想定されるクエリをログに出力する
    console.log(`SELECT * FROM tag WHERE post_id IN (${keys.join(",")})`);
    const results = tags.filter(tag => keys.includes(tag.post_id));

    return keys.map((key: number) => results.filter(tag => tag.post_id === key));
  });
}
```

▼リスト15　PostのBatch Functionにkeyを登録する

```
class PostsDataSource {
  async getPostsBy(author_id: number) {
    return this.loader.load(author_id);
  }
}
```

▼リスト16　TagのBatch Functionにkeyを登録する

```
class TagsDataSource {
  async getTagsBy(post_id: number) {
    return this.loader.load(post_id);
  }
}
```

し、DataLoaderのコンストラクタに渡します。この関数はデータ取得の際のkeyの配列を引数に受け取り、それに対応するvalueの配列(TypeScriptではvalueの配列のPromise)またはErrorを返します。

また、Batch Functionの実装には次のルールがあります。

・keyの配列とvalueの配列の長さが等しいこと
・keyの配列の順序とvalueの配列の順序が等しいこと

リスト12で示したN＋1問題を解決するためのBatch Functionを用意します。また、実際にはDBなどへのリクエストが発生することが想定されるため、発行されるであろうSQL文をログに出力します(**リスト13**、**リスト14**)。

遅延読み込みを行うBatch Functionへのkeyの登録はloader.load(key)で行います(**リスト15**、**リスト16**)。load(key)の戻り値はkeyに対応するvalueのPromiseになります。また、loadMany(keys)を用いることで複数件の遅延読み込みも可能です。

リスト15、**リスト16**で用意した関数をそれぞれのリゾルバーから呼び出すことでバッチ処理を行います。

Cacheはload()によって詰めたkeyとそれに対応するvalueの組をKey-Value storeにメモ化します。これにより、同一のDataLoaderのインスタンスから実行されたBatch内で、同一のkeyを用いて複数回valueを取得するとき、メモ化されたKey-Value storeから値を返すことができます。これを用いることで、パフォーマンスの向上やDBなどのデータソースへの負荷軽減が期待できます。

▼リスト17　contextのdataSourcesをリクエストごとに初期化する

```
interface ContextValue {
  dataSources: {
    posts: PostsDataSource;
    tags: TagsDataSource;
  };
}

const resolvers = {
  Query: {
    authors: () => getAuthors(),
  },
  Author: {
    posts: (parent, _, { dataSources }) => dataSources.posts.getPostsBy(parent.id),
  },
  Post: {
    tags: (parent, _, { dataSources }) => dataSources.tags.getTagsBy(parent.id),
  },
};

const schemaWithResolvers = addResolversToSchema({ schema, resolvers });
const apolloServer = new ApolloServer<ContextValue>({ schema: schemaWithResolvers });
const { url } = await startStandaloneServer(apolloServer, {
  context: async () => {
    return {
      dataSources: {
        posts: new PostsDataSource(),
        tags: new TagsDataSource(),
      },
    };
  },
});
```

ここで注意するべきことは、DataLoaderのインスタンスはリクエストごとに生成するべきであるということです。リクエストごとに生成することで、Cacheがメモ化するKey-Valueの組が、ユーザーやコンテキストの異なるリクエストから参照されることや、メモリに書き込まれ続け大量のメモリを消費することを防ぎます。

Apollo Serverではcontextのdata Sourcesをリクエストごとに初期化します。これにより、DataLoaderのインスタンスのライフタイムを単一のリクエストに限定します（**リスト17**）。

ここで先ほどの**リスト11**のクエリを実行すると、**図5**のログが出力されます。**図5**を見ると、N＋1問題が発生せずにデータソースへのリクエストがバッチ処理されていることがわかります。

▼図5　ログの出力

```
SELECT * FROM author
SELECT * FROM post WHERE author_id IN (1,2,3)
SELECT * FROM tag WHERE post_id IN (1,2,3,4)
```

まとめ

本稿では、3-2で定義したスキーマの読み込みに始まり、リゾルバーの実装、Apollo Serverの起動と実行、resolver chainsのしくみとN＋1問題、そしてDataLoaderパターンによるN＋1問題の解消方法について見てきました。本稿で使ったコードの全体像はサンプルリポジトリで確認できます。

ここで紹介したトピックの多くは、TypeScript以外の言語やApollo Server以外のライブラリを選定した場合にも有用性のあるものになっています。本稿がGraphQLサーバの実装に役立てば幸いです。SD

3-4 クライアントアプリケーションの実装

GraphQLならではの設計と実装を押さえる

Author 内山 高広(うちやま たかひろ)
株式会社リクルート スタディサプリ開発チーム
GitHub highwide X(Twitter) @highwide

本稿ではWebフロントエンドの実装を通してクライアント側の開発を解説します。ライブラリを使わなくてもGraphQLは利用できますが、urqlを利用したうえで、実践的な設計手法を取り入れた実装例を見ていきましょう。

はじめに

GraphQLを利用したアプリケーションにおける重要な考え方は「クライアントが必要なデータは、クライアントが決定する」というものです。これはREST APIにおける「まずサーバサイドがリソースのCRUD[注1]を実現して、クライアントはそのAPIを利用することで機能を実現する」という設計手法との違いと言えるでしょう。このようにクライアントありきでAPIを利用できる技術だからこそ、そのメリットを最大限に引き出せるようなクライアント実装を行わないと、GraphQLを採用する意味が薄れてしまいます[注2]。

本稿では、Webフロントエンドの実装を通してGraphQLを利用したクライアントアプリケーション実装のエッセンスをつかんでいただこうと思います。

なお、今回はWebフロントエンドを例にとりますが、GraphQLの技術はWebフロントエンドに限らず利用でき[注3]、そのエコシステムはiOSやAndroidといったNativeアプリケーションにも広がっています。GraphQLを活用したクライアントアプリケーション設計の勘所は、WebアプリケーションでもNativeアプリケーションでも共通すると言えます。

▼リスト1 GraphQLクエリの定義

```
const query = `
  query posts($tags: [String!]) {
    posts(tags: $tags) {
      title
      body
    }
  }
`;
```

▼リスト2 変数の定義

```
const variables = {
  tags: ["GraphQL"],
};
```

fetch()を利用したGraphQLクエリの実行

最初に、最もシンプルなクライアントの実装例として、JavaScriptを使ってfetch()[注4]を利用したGraphQLクエリの発行をしてみましょう。なお、本稿では3-2で定義したGraphQLスキーマを利用します。

まずはGraphQLクエリの定義です。titleとbodyのfieldを指定してpostを取得するpostsクエリを文字列で定義し、query変数に代入します(**リスト1**)。次に、先ほどのGraphQLクエリで必要とする変数をObjectの形でvariables変数に代入します(**リスト2**)。

注1) Create、Read、Update、Deleteの頭文字をとった略語。

注2) 利用しているライブラリは雑誌掲載時のものです。適宜最新のものをご確認ください。

注3) DroidKaigi 2024では9月13日にスタディサプリのAndroidアプリにおけるGraphQL利用事例の発表があります。Nativeアプリにおける GraphQLクライアントアプリケーション実装については、ぜひこちらも参考にしてください。
URL https://2024.droidkaigi.jp/timetable/694203/

注4) URL https://developer.mozilla.org/ja/docs/Web/API/Window/fetch

queryとvariablesをキーに持つJSONをHTTPリクエストのbodyに詰めて、fetch()を実行します（**リスト3**）。もしミューテーションを実行したい場合であっても、GraphQLミューテーションを定義した文字列をquery要素に渡せば実行できます。この結果得られるレスポンスのJSONは**リスト4**のようなものになります。レスポンスのJSONのdata要素の中にクエリで問い合わせたデータが返ってきていることがわかります。

このように、シンプルなfetch()のみでGraphQLを利用できることがわかりました。GraphQLはあくまでも「Query Language」の仕様であって、特殊な通信プロトコルを必要とするものではないことも体感できたかと思います。

▼リスト3　fetch()の実行

```
fetch("YOUR_SERVER_URL", {
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ query, variables }),
  method: "POST",
}).then((res) => res.json());
```

▼リスト4　レスポンスのJSON

```
{
  "data": {
    "posts": [
      {
        "title": "Signed Query は(..略..)"
        "body": "こんにちは。スタディサプリの(..略..)"
      },
(..略..)
```

GraphQLクライアントライブラリの利用

ここまで見たように、GraphQLサーバへのリクエストは単なるfetch()によるHTTPリクエストで実現できました。一方で、GraphQLを利用するクライアントアプリケーションのためのライブラリは多く存在しています。GraphQLクエリを発行するだけならfetch()でも事足りるのに、なぜライブラリを必要とするのでしょうか。それらのメリットを整理していきます。

UIライブラリとのインテグレーション

モダンなWebフロントエンドアプリケーションであれば、ReactやVue.jsといったUIライブラリを利用することが多いと思います。その中で、GraphQLのクエリをどのように発行し、その結果をどのようにハンドリングするかという点においても、UIライブラリの作法が求められます。たとえば、Reactとのインテグレーションができるライブラリであれば、GraphQLのリクエストを行うHooksがライブラリから提供されているかと思います。

HTTPリクエストのクライアントライブラリとして

GraphQLを利用していないアプリケーションで、TanStack QueryやSWRといったライブラリを利用したことがある方も多いかもしれません。fetch()を直接使うのではなく、これらのライブラリを利用するモチベーションは、GraphQLのクライアントライブラリを使うモチベーションにつながると言えます。

これらのライブラリが提供するリクエストがローディング中なのかエラーになったのか、といったHTTPリクエストのステータス管理機能や、通信結果をキャッシュしアプリケーションの「状態」として管理してくれる機能は、GraphQLの利用有無を問わず「HTTPリクエストのクライアントライブラリ」としてニーズがあると言えます。

静的型付け言語の型とのインテグレーション

GraphQLスキーマにおいてtitleというフィールドがString!のように定義されていたならば、「titleはnullにならず、Stringが返ってくること」が期待されます。それならば、クライアントアプリケーションにおいては、クエリ結果の型定義をTypeScriptのstringとして自動生成できれば、実装者がGraphQLスキーマの型情報をいちいち確認せずに済みますし、うっかりスキー

マ定義と誤った型を書いてしまうミスを防いだり、スキーマの変更に追従したりすることも容易になります。

「スキーマ駆動開発」と呼ばれるGraphQLスキーマを先に決めて開発を行う手法においては、自動生成した型定義を用いることで、サーバサイドの実装完了を待たずにクライアントの実装を始めることもできます。

クライアントライブラリ「urql」

ここまでに挙げたような利点をふまえて、本稿で利用するクライアントライブラリ「urql[注5]」を紹介します。

urqlは、同じくGraphQLのクライアントライブラリであるApollo ClientやRelayに比べると後続のライブラリで、その特徴はシンプルさにあります。

Apollo ClientやRelayでは、クエリ結果をidと__typenameをキーに正規化してキャッシュする「正規化キャッシュ」を採用しています。これによって、ミューテーションによるデータ更新の際も、キャッシュ上のどのデータを更新すべきかライブラリが知ることができ、ネットワークアクセスを介さずにクライアント上の状態を更新できます。

一方で、urqlにおいては「ドキュメントキャッシュ」と呼ばれるキャッシュ手法をデフォルトで採用しています。これは、ミューテーションによるデータ更新後、同一__typenameを持つクエリのキャッシュを破棄して再取得するというものです。当然、ネットワークアクセスの頻度はドキュメントキャッシュのほうが相対的に高くなります。ただし少なくとも、更新したデータをそのまま画面上に表示する必要がないアプリケーションであれば、正規化キャッシュほどの高度なキャッシュ機能は不要と言えます。

今回urqlを採用したのは、「ライブラリの挙動を追う際の難度やバンドルサイズ等の観点から、必要十分にシンプルなライブラリを選定したい」と考えたからです。なお、urqlにおいても正規化キャッシュの利用は可能で、公式ドキュメントでもそのためのガイドが用意されています[注6]。

▼図1　プロジェクトの作成

```
$ npm create vite@5.4.0 graphql-client-sample -- --template react-swc-ts
$ cd graphql-client-sample
$ npm install --save urql@4.1.0
```

urqlを使ったサンプルアプリケーション

3-2のGraphQLスキーマを利用して、ブログポストとその著者を表示する簡単なクライアントアプリケーションを、urqlを利用して実装してみましょう。誌面の都合で掲載するコードは省略されていますが、すべてのコードが置かれたリポジトリ[注7]を用意していますのでそちらもご参照ください。

まずは、Node.jsを手元にインストールしたうえで、Vite[注8]＋SWC[注9]によるReactのテンプレートを作成し、urqlをインストールします(**図1**)。CSSやassetsなど、今回のサンプルアプリケーションで利用しないものはコード内のimportやファイルを削除してください。

それでは、main.tsxにおいてこのアプリケーションがGraphQLサーバを利用できるようにします。**リスト5**では、まずGraphQLクエリの発行やその結果のキャッシュを管理するClientを作成しています。urlにはGraphQLサーバのURLを指定します。なお、exchangeとはurqlにおけるミドルウェアやプラグインにあたる機構です。そして、Clientを配下のコンポーネン

注5) URL https://nearform.com/open-source/urql/

注6) URL https://nearform.com/open-source/urql/docs/graphcache/normalized-caching/

注7) URL https://github.com/quipper/software-design-2024-09-graphql

注8) URL https://ja.vite.dev/

注9) URL https://swc.rs

トがReactのContext API[注10]を介して利用できるようProviderに渡しています。

これで、App.tsx以下でurqlを使ったGraphQLクエリを実行する準備ができました。

実際にアプリケーションで必要なデータとなる、ブログポストとその著者を取得するGraphQLクエリを書いてみます（**リスト6**）。

そして、**リスト6**のクエリをurqlからimportしたuseQuery()で発行します（**リスト7**）。記事を取得する際の変数となるtagsはuseState()を利用して状態として管理しておきましょう。

useQuery()の戻り値として、dataというクエリの結果に加えて、fetchingやerrorといったHTTPリクエストのステータスも取得できるのがわかります（**リスト7**）。

サンプルリポジトリからアプリケーションを動かしてみると、チェックボックスで選択したタグに合わせて記事の表示を行うアプリケーションが確認できるかと思います。また、ブラウザのDeveloper ToolsのNetworkタブなどで確認してみると、一度取得したタグに対しては再度リクエストされずにキャッシュからレンダリングされていることが確認できるかと思います。

graphql-codegenによる型生成

次に、GraphQLの型とTypeScriptの型とのインテグレーションを行って、このアプリケーションをブラッシュアップしていきましょう。

まず、先ほどの**リスト7**を再度見てみましょう。実はGraphQLクエリの結果としてで得られたdataのTypeScriptとしての型はanyです。

▼リスト5　GraphQLサーバを利用できるようにする（src/main.tsx）

```
const client = new Client({
  url: "YOUR_SERVER_URL",
  exchanges: [cacheExchange, fetchExchange],
});

ReactDOM.createRoot(document.getElementById("root")!).render(
  <React.StrictMode>
    <Provider value={client}>
      <App />
    </Provider>
  </React.StrictMode>
);
```

▼リスト6　必要なデータを取得するクエリ（src/App.tsx）

```
const PostsQuery = gql`
  query posts($tags: [String!]) {
    posts(tags: $tags) {
      title
      body
      author {
        name
      }
    }
  }
`;
```

▼リスト7　リスト6のクエリを発行する（src/App.tsx）

```
function App() {
  const [tags, setTags] = useState(["GraphQL"]);

  const [result] = useQuery({
    query: PostsQuery,
    variables: { tags },
  });

  const { data, fetching, error } = result;
```

▼リスト8　TypeScriptによる型定義

```
type Post = {
  title: string;
  body: string;
};
```

せっかくGraphQLスキーマによってHTTPレスポンスのデータ型が保証されているのに、anyとなってしまうのは残念です。

これに対応するためレスポンスの型を**リスト8**のように自分で定義することもできますが、GraphQLスキーマで**リスト9**のように定義されているものと本質的に同じなので、二度手間になってしまいます。このような型定義の二度

注10）URL https://ja.react.dev/learn/passing-data-deeply-with-context

手間を避けるだけでなく、スキーマと不一致の型を書いてしまうミスを防ぐためにも、自動で型生成を行いたいところです。

Relayにはrelay-compilerという機構があり、TypeScriptやFlowといったプログラミング言語の型を生成できますが、Apollo Clientやurqlによる開発ではgraphql-codegen[注11]というライブラリを使うのが一般的です。urqlの公式ドキュメントにおいても「Basics > TypeScript Integration」のチャプターにおいて紹介されています[注12]。このドキュメントに沿って、graphql-codegenや関連ライブラリをインストールし、プロジェクトルートにcodegen.tsを置くと型生成の準備が整います。codegenを行ってみると、src/配下にgql/というディレクトリと型定義が書かれたいくつかのファイルが作成されたことが確認できるでしょう。

実際にsrc/App.tsxに書かれたGraphQLクエリから型を生成していきましょう。**リスト6**で書いたGraphQLクエリを、先ほど生成したgql/からimportしたgraphql()を通すように変更します(**リスト10**)。すると、これまでanyとなってしまっていた型が、GraphQLスキーマを利用した型に変更されていることが確認できるかと思います。



次に「Fragment Colocation」という手法を利用して、よりアプリケーションを改善していきます。Colocationは「一緒に(co)+配置する(location)」というニュアンスでとらえてもらうとよいでしょう。何を「一緒に配置する」のかと言えば「GraphQLクエリ」と「UIコンポーネント」です。

GraphQLとは「UIが自身の必要なデータを定義すること」を可能とする技術です。そして、UI上の各コンポーネントが「自身の必要なデータ」をGraphQLのFragmentを使って定義するのがFragment Colocationの本質です。そうしてUIコンポーネントとGraphQLクエリのツリーを相似形にさせれば、「そのコンポーネントが、どのようなデータを必要とするのか」をうまく対応付けられます(**図2**)。

これによって、あるコンポーネントに変更があって必要なデータが増えたときや減ったときに、修正するファイルはそのコンポーネントに閉じるため、メンテナンス性が向上したと言えるでしょう。フィールドの増減に気づきやすい構造であるため、オーバーフェッチングのリスクも低くなっています。

先ほど**リスト6**では、Postと、その子要素であるAuthorを取得するGraphQLクエリを書きましたが、Fragmentに分割すると**リスト11**と**リスト12**のようになります。これをそれぞれPostコンポーネント、Authorコンポーネントに配置することで、「UIコンポーネント」と「自身の必要なデータ」の関係性がわかりやすくなるというのが確認できるかと思います。

App.tsxに書かれているGraphQLクエリの最上位階層はFragmentの埋め込みのみになりました(**リスト13**)。ここで再度graphql-codegenを実行することで、Fragmentに分割された型定義を利用できます。

▼リスト9　GraphQLスキーマの定義

```
type Post {
  title: String!
  body: String!
}
```

▼リスト10　リスト6のクエリを変更

```
const PostsQuery = graphql(/* GraphQL */ `
  query posts($tags: [String!]) {
```

▼図2　UIとGraphQLクエリの相似形

```
App                          Query
 └ Post component             └ Post fragment
    └ Author component           └ Author fragment
```

注11) URL https://the-guild.dev/graphql/codegen
注12) URL https://nearform.com/open-source/urql/docs/basics/typescript-integration/

▼リスト11　Postコンポーネントに対応するFragment

```
const PostFragment = graphql(/* GraphQL */ `
  fragment PostFragment on Post {
    title
    body
    author {
      ...AuthorFragment
    }
  }
`);
```

そして、各コンポーネントがこのFragmentに対応するデータが渡されることを期待する設計にします（**リスト14**）。useFragment()をgqlからimportして利用することによって、Fragmentで取得したデータの中身を触ることができるようになります。また、useFragment()で取得したObjectにはgraphql-codegenを通して生成したGraphQLのFragmentに対応する型が付けられています。仮にPostのpublishedAtを新たに表示したいとなった場合に、{post.publishedAt}と書いてもそのようなフィールドがないというエラーが出るでしょう。これによって、GraphQLのFragmentを修正してサーバから情報を新たに取得する必要があると実装者が気がつくことができます。

また、Authorコンポーネントには Authorの Fragment部分をそのまま渡しています。Postコンポーネントにとっては「Authorがどんなフィールドを必要としているか」という内部情報を気にする必要がないことが読み取れるかと思います。仮に Post Component に post.author.name

▼リスト12　Authorコンポーネントに対応するFragment

```
const AuthorFragment = gql`
  fragment AuthorFragment on Author {
    name
  }
`;
```

▼リスト13　App.tsxのGraphQLクエリ定義

```
const PostsQuery = graphql(/* GraphQL */ `
  query posts($tags: [String!]) {
    posts(tags: $tags) {
      ...PostFragment
    }
  }
`);
```

のように、Author Componentに閉じたフィールドへのアクセスを書いた場合、TypeScriptのエラーとなるはずです。これはgraphql-codegeのFragment Maskingと呼ばれるしくみです。

このようにFragment Colocationは多くのメリットを開発にもたらします。発展的なトピックのように思えるかもしれませんが、GraphQLという技術を使うからにはぜひ実践したい手法です。



本稿ではfetch()を利用してGraphQLクエリを発行するところから始まり、クライアントライブラリの導入、TypeScriptの型生成、Fragment Colocationの実践と、クライアントアプリケーションを実践的に改善させていきました。コードの全体像は、サンプルリポジトリで確認していただけます。たとえば、誌面の都合で載せきれなかったミューテーションの実行についてもサンプルアプリケーションではコードを用意しています。

GraphQLの利用有無を問わず、本稿で学んだ「GraphQLの考え方」がみなさんのクライアントアプリケーション実装に役立てばうれしいです。SD

▼リスト14　各コンポーネントがFragmentに対応するデータが渡されるようにする（src/Post.tsx）

```
export const Post: FC<{
  postFragment: FragmentType<typeof PostFragment>;
}> = ({ postFragment }) => {
  const post = useFragment(PostFragment, postFragment);

  return (
    <>
      <h2>{post.title}</h2>
      <div>{post.body}</div>
      <Author author={post.author} />
    </>
  );
};
```

3-5

Author 上原 拓也(うえはら たくや)
株式会社リクルート スタディサプリ開発チーム
GitHub indigolain X(Twitter) @indigolain

GraphQL アプリケーションの実運用

パフォーマンス改善、セキュリティ対策、APIの進化戦略のプラクティスを押さえる



3-5では、本番環境で運用するうえで重要なモニタリング、エラートラッキング、パフォーマンス改善、セキュリティ対策、そしてAPIの拡張・廃止まで、現場の実践例をもとに解説します。実践におけるプラクティスを知ることで、GraphQLの利点を最大限享受できるようになるでしょう。

はじめに

本稿では、実際に本番環境でGraphQLクライアント／サーバを運用するうえで重要な観点について、筆者らが運用しているSSKD[注1]向けサービスにおいて実践しているプラクティスも交えて紹介します。

モニタリングとパフォーマンス改善

GraphQL、RESTにかかわらず、APIのパフォーマンスは重要です。ユーザーが期待するパフォーマンスを提供できなくなってしまうと、ユーザーが離脱してしまうかもしれません。ユーザーが離脱する前に、継続的なモニタリングと改善を行いましょう。

GraphQLはRESTと異なり、クライアント側がほしいデータをクエリという形で定義するアプローチを取ります。GraphQLを運用する場合は、このように自由にクエリが作られる前提のもと、モニタリングやパフォーマンスの改善が必要です。

ここでは、GraphQLで運用しているサービス群について、筆者らがどのようにそのパフォーマンスをモニタリング、改善しているかを紹介します。

注1) Small Set of Known Developersの略。社内など特定の信頼できるユーザーを指します。

モニタリング

GraphQL APIの健康状態

APIの健康状態を表現するうえでよくある指標として、リクエストに対して返却されるステータスコードによるSuccess Rateや、Response Timeが存在します。REST APIでこれらを計測する手法はある程度確立されていて、あまり迷うことがないかと思います。

しかし、GraphQL APIについては同じ手法を適用できません。エンドポイントごとにパフォーマンスモニタリングが行えるREST APIとは異なり、GraphQLは通常1つのエンドポイントですべてのリクエストを処理するため、エンドポイントのパフォーマンスだけを見ても何が起きているのかよくわかりません。細かいパフォーマンスを把握するためには、リゾルバーで行われている処理や、クライアントで定義されているクエリごとのリクエストのモニタリングが必要になってきます。

そのほかにもステータスコードに関して、GraphQLサーバライブラリのいくつかはデフォルトで成功／エラーの双方でステータスコード200を返すようになっていることがあります。この場合、ステータスコードはモニタリング指標としては不適切になります。

また、GraphQLはpartial responseという概

▼リスト1　dd-trace graphqlプラグイン設定

```
import tracer from 'dd-trace'
(..略..)
tracer.init({
  logInjection: true,
  service: serviceName,
})
(..略..)
tracer.use('graphql', {
  service: `${serviceName}/graphql`,
  // クエリの名前をresource nameとして出すための設定
  signature: false,
})
```

念があり、「正常に処理が実行されたフィールドはそのまま返しつつ、エラーが起きていたフィールドはnullにする」といった部分的な成功が存在し得ます。そのため、これをモニタリングの指標としてどう扱うのかなどを判断する必要があります。

Datadogを活用したモニタリング

筆者らのサービスではGraphQLサーバのパフォーマンスモニタリングにDatadog Application Performance Monitoring（APM）[注2]を活用しています[注3]。具体的にはDatadogのNode.js用APM Tracer Libraryであるdd-trace[注4]のgraphqlプラグインを用いて**リスト1**のように設定し、DatadogのAPM上でクエリの実行ごとにパフォーマンスを見られるようにしています（**図1**）。

Datadog APMを設定することによって、次のようなことを確認できるようになっています。

- どのクエリの実行が性能劣化しているのか
- その実行にどのような処理が含まれているのか
- どこがボトルネックになっているのか

そのほかにも、GraphQLサービスのService Level Objective（SLO）[注5]も設定してあり、これもDatadogのダッシュボードで可視化しています（**図2**）。ここで見ているのは、Http Success RateとResponse Timeです。これらがあらかじめ決めていた基準を下回った場合、開発者に通知が飛び、調査や改善などが行われるようになっています。

また、先述したGraphQLのpartial responseに関しては、筆者らのサービスの構成として、部分的にでも失敗するとユーザーはアプリ上の操作ができなくなってしまうため、部分失敗は全体失敗とし、「status code 500」を返す選択をしています。

注2） URL https://docs.datadoghq.com/ja/tracing/

注3） 過去にはNew Relicのapollo server pluginを利用して同様のモニタリングをしていましたが、社内で利用しているモニタリング用SaaSをDatadogに一本化するために乗り換えています。

注4） URL https://github.com/DataDog/dd-trace-js

注5） サービスの品質に関する内部的な目標値。下回ると意図した品質でサービスを届けられていないことになります。

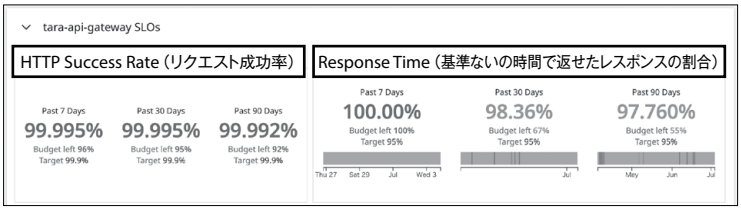
▼図1　Datadog APM Traces

↑ DATE	SERVICE	RESOURCE	DURATION	N	S	SPANS	LATENCY BREAKDOWN
Jun 25 13:57:10.	tara-api-gateway…	mutation LowerElementary_Webview_UseSen…	67.9 ms			122	
Jun 25 13:57:12.	tara-api-gateway…	mutation Junior_Webview_UseSendQuestion…	125 ms			101	
Jun 25 13:57:15.	tara-api-gateway…	query Webview_UseScreenKind_Query	84.1 ms			62	
Jun 25 13:57:27.	tara-api-gateway…	query Home	158 ms			239	
Jun 25 13:57:27.	tara-api-gateway…	query Pc_MissionDetail_Query	311 ms			223	
Jun 25 13:57:28.	tara-api-gateway…	query Pc_MissionDetail_LessonProgresses	41.2 ms			92	
Jun 25 13:57:28.	tara-api-gateway…	query Pc_MissionDetail_Query	171 ms			224	
Jun 25 13:57:30.	tara-api-gateway…	mutation Junior_Webview_LessonPage_Calc…	341 ms			217	
Jun 25 13:57:31.	tara-api-gateway…	query Course	281 ms			265	
Jun 25 13:57:33.	tara-api-gateway…	mutation Junior_Webview_UseWatchVideo_W…	148 ms			79	
Jun 25 13:57:43.	tara-api-gateway…	query RootScreen	40.0 ms			107	
Jun 25 13:57:52.	tara-api-gateway…	query LowerElementarySchoolTopicsCourses	29.9 ms			109	
Jun 25 13:57:54.	tara-api-gateway…	mutation LowerElementary_Webview_Lesson…	411 ms			298	
Jun 25 13:57:57.	tara-api-gateway…	query CoursesWithCategorie	138 ms				

クエリ名

処理時間や処理の単位

▼図2　SLOダッシュボード



▼図3　fingerprintの修正方針



Sentryによるエラートラッキング

パフォーマンスモニタリングと同様に、エラー検知のしくみもサービスをモニタリングするうえで大切です。筆者らのサービスではエラー検知にSentry[注6]というエラー通知サービスを利用しています。Sentryでは通常異なるエラーをissueという単位で管理してくれるようになっています。

しかし、運用をしていく中で、「異なる原因を持つエラーであっても1つのissueにまとめられてしまう」という問題に遭遇しました。筆者らはGraphQLサーバを実現するためのライブラリとして、Apollo Serverを利用していますが、GraphQLの各種処理の中で発生するエラーをSentryへ漏れなく通知するために、このApollo ServerのdidEncounterErrorsというイベントハンドラを利用していました。

また、Sentryはfingerprintというエラーをまとめるための設定があり、デフォルトの状態では通知されるエラーイベントの利用可能な情報に基づいてエラーをまとめる設定がされていました。しかし、この設定では、didEncounterErrorsを通る特定の種類のエラーは、原因が異なっていても1つのissueにまとめられてしまっていたのです。

そこで筆者らは、エラーメッセージをfingerprintに追加することで細かくグルーピングをさせることを選択しました（**図3**）。

実際に**リスト2**のようにエラー発生時にfingerprintを設定し、エラーメッセージに基づいてグルーピングをさせることで原因別にエラーをまとめられるようにしています。

パフォーマンス改善

APMを活用してN＋1を解消する

パフォーマンスの劣化が検知できたら、次は改善です。GraphQLではN＋1問題が発生しやすいと3-3で紹介しました。実際に筆者らが検知したパフォーマンス劣化で、とくにN＋1問題が発生していたものについて、どのように解決したかを紹介します。

ことの発端はSentryのエラー通知でした。ORMが定期的にRDSに接続できていない旨を

注6）URL https://github.com/getsentry/sentry

表すエラーがいくつか発生しており（**図4**）、アプリケーションとしてのパフォーマンス劣化につながってしまっていたので改善する必要がありました。

Datadog上のRDS接続数を示すチャート（**図5**）を見ると、ある日を境にRDSの接続数が一気に増えているタイミングがあることがわかりました。これによって、この日付周辺で入った変更が関連してきそうなことがわかりました。

調べていくと、直近変更が入って、かつレスポンスに時間がかかっているCourse Detail_Courseクエリが浮かび上がってきました（**図6**）。Datadog APMでより詳しく見ていくと、**図7**のようにQuestion.findManyというRDSに対する問い合わせがN＋1の形で発生していました。

実際に実行されていたクエリを観察したところ、3-3で紹介したDataLoaderを利用することでN＋1を回避し、パフォーマンスが改善できることがわかりました。DataLoaderを利用することでRDSの接続数は回復し、Sentryのエラーも発生しなくなり、問題が解決したことが確認できました。

▼リスト2　fignerprintの設定

```
export const sentryPlugin = () => {
  const plugin: ApolloServerPlugin<ExpressContext> = {
    async requestDidStart() {
      return {
        async didEncounterErrors(ctx) {
          (..略..)
          for (const gqlError of ctx.errors) {
            const { originalError } = gqlError
            const err = originalError ?? gqlError
            Sentry.withScope((scope) => {
              (..略..)
              // ここでエラーメッセージをfingerprintに付与
              scope.setFingerprint(['{{ default }}', err.message])
              (..略..)
              Sentry.captureException(err, { level: 'error' })
            })
          }
        },
      }
    },
  }
  return plugin
}
```

▼図4　RDSの接続エラー

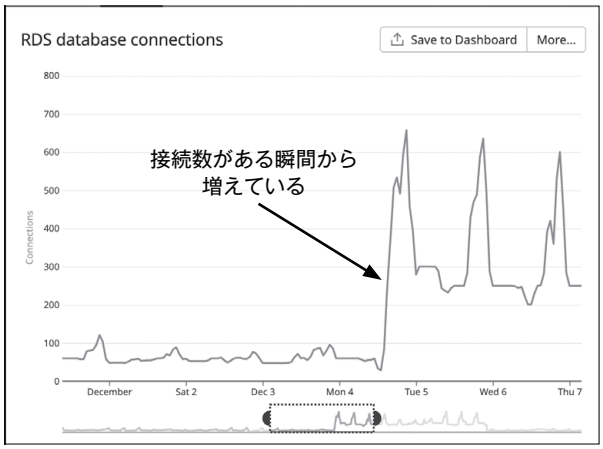


フロントエンドのパフォーマンス改善

フロントエンドのパフォーマンスはさまざまな指標があり、Core Web Vitals[注7]で挙げられているLCP（Largest Contentful Paint）、CLS（Cumulative Layout Shift）、INP（Interaction to Next Paint）などが代表的な指標です。ユーザーにすばやくページの内容を返すためにでき

注7）URL https://web.dev/articles/vitals

▼図5　RDS接続数



る改善はさまざまあり、3-4ではGraphQLリクエストで得られたデータをクライアント側でキャッシュしつつ、適切に利用することでフロントエンドの通信回数を減らし、パフォーマンスを改善するしくみを紹介しました。

フロントエンドでできるパフォーマンス改善はこれにとどまらず、GraphQLの@deferや@streamディレクティブを利用した、レスポンスを非同期的に徐々に受け取れるようにする手法もあります。サーバ側の対応も必要になりますが、これらを活用することでデータを部分的に取得できるようになるため、取得できたデータから順次表示を行えるようになります。

たとえば、@deferや@streamディレクティブを利用した**リスト3**のクエリを実行したとします。

まず、最初に返ってくるレスポンスは**リスト4**のようになります。その後非同期的に**リスト5**のような@deferを付与したfragmentのレスポンスが返ってくるようになります。最後に@streamを付与したフィールドのレスポンス（**リスト6**）が返ってくるようになります。

このように、@deferや@streamディレクティブを利用することで、GraphQLサーバから徐々にレスポンスを返すことができます。フロントエンドでは受け取ったレスポンスを利用してページの初期表示をすばやく行いつつ、ページを徐々に組み立てることが可能になり、ユーザー体験の改善につなげられます。

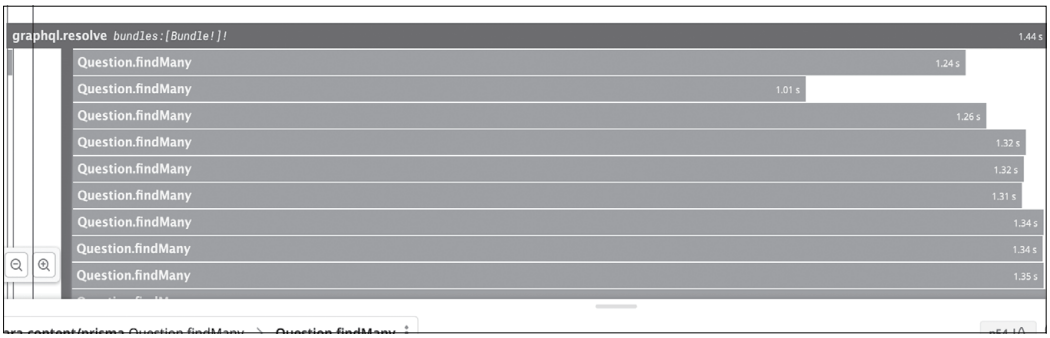


GraphQLはREST APIと比較してクライア

▼図6　遅いクエリのリスト

	RESOURCE NAME	REQUESTS	TOTAL TIME	↓ P95 LATENCY	ERRORS
☆	query CourseDetail_Course	179	6 min 37 s	5.68 s	0
☆	query MissionDetail_Query	66	56.3 s	2.87 s	0
☆	query MissionAdjust_Course	1	4.27 s	1.84 s	0

▼図7　遅いクエリのFlame Graph



ントが柔軟にクエリを組み立てられる点で利便性が高いことが知られていますが、それを乱用されてしまうとサービス運用者は困ってしまいます。ここでは乱用を防ぐための手法を紹介していきます。

外部公開されているGraphQL APIの場合

外部のクライアントから参照される可能性のあるGraphQLサーバは、利用者のさまざまなユースケースに対応しつつ、サービスを持続させるために悪意のあるリクエストや、意図されていない使い方を検知し防ぐ必要があります。

Depth limiting

Depth limitingは再帰的なネストが可能なオブジェクトタイプのネストの深さを制限する手法です。

たとえば、**リスト7**のスキーマが存在するとします。スキーマとしてAuthor（著者）とPost（投稿）は互いに参照できるため、**リスト8**のように互いにネストして参照し続けるようなクエリを実行することが可能となってしまいます。

クエリされている各フィールドを計算する必要があるため、このネストが深くなればなるほど処理する回数やデータ量が増え、レスポンスを返す時間が延びてしまいます。サービス運用者はこのように意図しないネストの深さを持つクエリを防ぐために、graphql-armor[注8]のmax-depth pluginなどのライブラリを利用して、ネストの深さの違反を検知してエラーにすることで、GraphQLサーバに対す

▼リスト3　@defer、@streamを用いたクエリ

```
query {
  author {
    # @deferディレクティブが付与されているProfessionFragment
    # fragmentの内容が解決されしだい非同期的にレスポンスが返ってくる
    ...ProfessionFragment @defer(label: "professionDefer")
    name
    # @streamディレクティブが付与されているarticlesフィールド
    # initialCountは同期的にレスポンスが返ってくる件数
    # それ以降は非同期的にレスポンスが返ってくる
    articles @stream(initialCount: 2, label: "articlesStream") {
      title
    }
  }
}
fragment ProfessionFragment on Author {
  profession {
    name
  }
}
```

▼リスト4　最初に返ってくるレスポンス

```
{
  "data": {
    "author": {
      "name": "Luke Skywalker",
      "articles": [
        { "title": "GraphQL is awesome!" },
        { "title": "5 reasons why you should be using GraphQL" }
      ]
    }
  },
  // hasNext: trueという、まだデータが来る可能性がある旨を示す項目
  "hasNext": true
}
```

▼リスト5　deferを付与したfragmentのレスポンス

```
{
  "label": "professionDefer",
  // 元のクエリのうちどこに書かれていたのかを示す項目
  "path": ["author"],
  "data": {
    "profession": {
      "name": "Web Developer"
    }
  },
  // @streamが付与されているデータがまだ存在しているのでhasNext: trueのまま
  "hasNext": true
}
```

注8）URL https://github.com/Escape-Technologies/graphql-armor



▼リスト6　streamを付与したフィールドのレスポンス

```
{
  "label": "articlesStream",
  // 元のクエリのうちどこに書かれていたのかを示す項目
  // author.articlesの最初の2件よりも先のデータであることが示されている
  "path": ["author", "articles", 2],
  "data": {
    "title": "Fragment Colocation is the way to go"
  },
  // すべてのデータが返されたのでhasNext: falseになる
  "hasNext": false
}
```

▼リスト7　Author（著者）とPost（投稿）のスキーマ

```
type Author {
  authorId: ID!
  posts: [Post!]!
}

type Post {
  postId: ID!
  title: String!
  author: Author!
}

type Query {
  author(id: ID!): Author!
}
```

▼リスト8　ネストの深いクエリ

```
query NestingQuery {
  author(id: "123") {
    authorId
    posts {
      postId
      author {
        authorId
        posts {
          postId
          author {
            authorId
            posts {
              postId
            }
          }
        }
      }
    }
  }
}
```

る過度な負荷を防ぐ機構を持たせることが可能となります。



Query Cost Analysis

ほかにも、あらかじめ決められた計算式を用いてクエリのコストを計算し、それが一定のコストを超える場合リクエストを失敗させることで不正な利用を防ぐ手法もあります。

例としてShopifyが提供しているGraphQL APIでは、あらかじめAPIの利用者に一定時間ごとに割り当てられるpointが振られており、このpointを消費してGraphQLリクエストを行うようになっています。このpointが尽きてしまう場合、リクエストが失敗するようになっています。

このときのGraphQLリクエストを行う際に消費するpointは、次に示すクエリの要素を用いて、Shopifyが内部で持っている計算式に当てはめることで計算されるようになっています[注9]。

- Mutationか否か
- クエリしているフィールドがどのような種類の値を返すのか
- interfaceやunionをクエリに含めているか
- connectionを用いていくつのデータをクエリするか

たとえば、**リスト9**のクエリをリクエストすると、**リスト10**のレスポンスが返ってくるようになっており、extensionsの項目でクエリが消費したpointや、あとどのくらいpointを利用できるかなどを確認できるようになっています。

これらと、そのほか一度にクエリできるNodeの数の制限を設けるなど細かい制約を組み合わせることで不正な利用を防ぐようになっています。

また、細かいルールは異なりますが、GraphQLクエリの要素を用いてクエリのコストを計算しつつ、それを一定時間ごとに割り振られている予算内に抑えさせるこのアプローチは、GitHubでも採用されています[注10]。

注9）URL https://shopify.dev/docs/api/usage/rate-limits#cost-calculation

注10）URL https://docs.github.com/ja/graphql/overview/rate-limits-and-node-limits-for-the-graphql-api

▼リスト9　Shopifyに対するクエリ

```
query ShopAndProducts {
  shop {
    name
  }
  products (first: 3){
    nodes {
      title
    }
  }
}
```

内部利用のみのGraphQL APIの場合

GraphQLサーバと、それを利用するクライアントアプリ両方を運用している内部利用のみを想定している場合、外部公開のAPIと比べてユースケースが限定されるので、より細かくクエリの制限をすることが可能になります。内部利用のGraphQL APIのセキュリティを高めるために、Trusted Documentという、「信頼されたクエリ」のみを許可して、ほかのリクエストを制限する手法を紹介します。

Persisted Query

内部利用を想定する場合、事前にどのようなGraphQLクエリがリクエストされ得るのかをコードから把握することが可能です。これらのクエリを事前にPersisted Queryという形で登録しておき、登録されているクエリのリクエストのみを許可するようにすることで、リクエストの制限を行えます。このように作られたPersisted Queryは信頼性のあるクエリであることから、Trusted Documentの一例として紹介されることが多いです[注11]。

具体的なフローとしては次のような形になります（**図8**）。

1. GraphQL Codegenなどのツールを利用してクライアントアプリで利用しているクエリをまとめてハッシュ化し（**リスト11**）、元となったクエリとともにサーバへ保存する

▼リスト10　Shopifyから返却されるレスポンス

```
{
  "data": {
    "shop": {
      "name": "hogeshop"
    },
    "products": {
      "nodes": [
        {
          "title": "winter feather"
        },
        {
          "title": "wispy leaf"
        },
        {
          "title": "dark mountain"
        }
      ]
    }
  },
  "extensions": {
    "cost": {
      // クエリの構成から予測された消費point
      "requestedQueryCost": 5,
      // 実際に消費したpoint
      "actualQueryCost": 5,
      "throttleStatus": {
        // クライアントに割り当てられるpointの上限
        "maximumAvailable": 2000,
        // 使える残りpoint
        "currentlyAvailable": 1995,
        // 1秒ごとに回復するpoint
        "restoreRate": 100
      }
    }
  }
}
```

2. クライアントからは、クエリをハッシュ化したものを用いてGraphQLリクエスト（**リスト12**）を行い、サーバでハッシュに対応するクエリを見つけて実行する
3. 2でハッシュに対応するクエリが存在しない場合、想定していないクエリだと判断し、エラーを返す

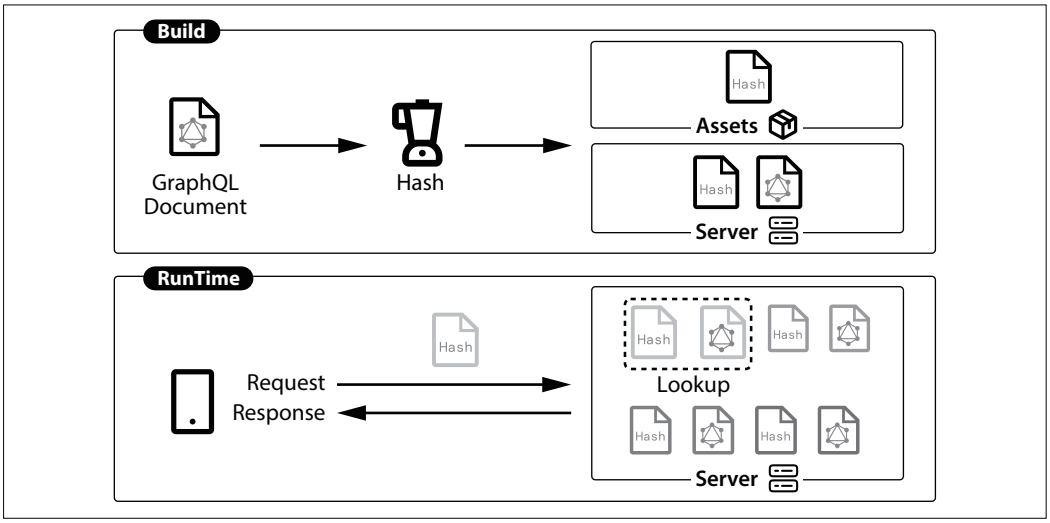
Signed Query

Signed Query[注12]もTrusted Documentによるリクエスト制限を実現するための実装パターンです。Persisted Queryと異なる点としては、サーバにホワイトリスト的にクエリを事前に登録するのではなく、共通鍵を用いたメッセージ

注11）URL https://benjie.dev/graphql/trusted-documents

注12）Signed Queryは「スタディサプリ」発の実装パターンです。

▼図8　Persisted Queryのしくみ



▼リスト11　Persisted Queryの例：ハッシュ値とクエリの対応

```
{
  "ff86b91924(..略..)": "query GetPosts { author { posts { title } } }",
  "9b84e5d43e(..略..)": "query GetAuthorId { author { authorId } }",
  (..略..)
}
```

▼リスト12　クライアントのリクエストボディ

```
{
  "operationName": "GetPosts",
  "variables": {},
  "extensions": {
    "persistedQuery": {
      "version": 1,
      // GraphQLリクエストにはハッシュ値を使う
      // Persisted QueryのGetPostsクエリに対応しているため、サーバでGetPostsクエリが実行される
      "sha256Hash": "ff86b91924(..略..)"
    }
  }
}
```

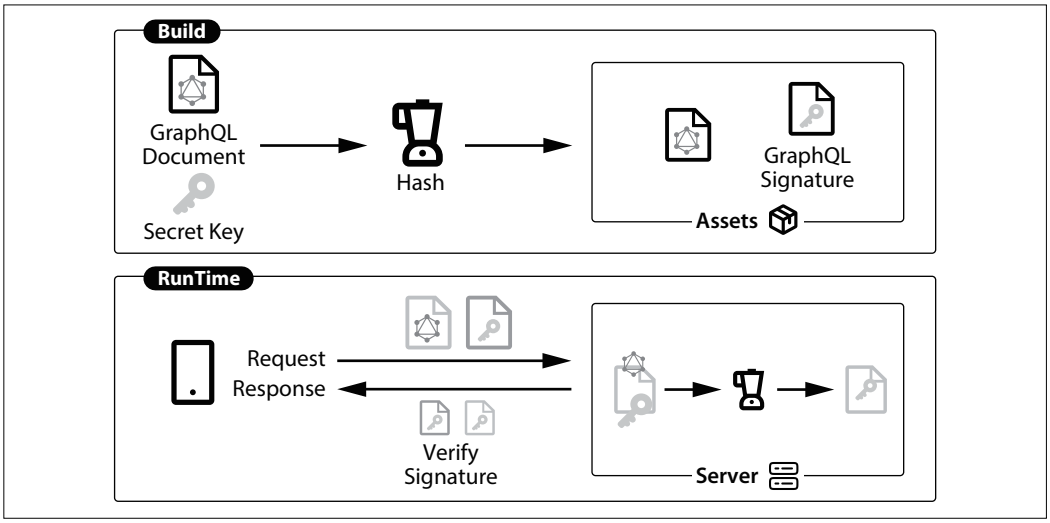
認証でクエリの信頼性を検証する点にあります。

Signed Queryは次のフローでリクエストを制限します（**図9**）。

1. クライアントをビルドする際に、GraphQL Codegenなどのツールを利用し、クライアントアプリで利用しているクエリをまとめ、それぞれのHash Based Message Authentication Code（HMAC）を計算し、それをアプリに埋め込む
2. ランタイムでは、クライアントはクエリとそれに対応したHMACをサーバに送信し、サーバではあらかじめ共有しておいた共通鍵を用いてHMACを検証することで信頼されたクエリかを判断する
3. 信頼されていないクエリと判断された場合エラーを返す

この方式であれば、一度共通鍵を共有できる状態を構築できれば、Persisted Queryと比較

▼図9　Signed Queryのしくみ



して事前にクエリをGraphQLサーバに登録する工程が不要となります。つまり、クエリの信頼性を担保しつつ、クライアントアプリをいつでもデプロイできるようになります。

Signed Queryのより詳しい説明はスタディサプリのブログ[注13]をご参照いただければと思います。

GraphQLによるAPIの進化

REST APIと同様に、GraphQLでもサービス運用を続けていくうえでAPIの更新は避けられません。その場合でも、クライアントからのリクエストは安定して処理する必要があります。また更新によっては、ときにAPIに対して破壊的な変更を加えていく必要が出てきてしまう可能性があります。

GraphQLは進化をさせやすい特性を持つAPIであると3-1で紹介しましたが、どのようにこの特性を利用して実際にAPIを進化させているのか紹介します。

APIの拡張

APIでクエリできるものを増やすことは、アプリの機能を新規実装していくうえで必要になってきます。REST APIの場合、すでに存在しているAPIを拡張しようとすると、いくつか悩まされるポイントが出てきます。

たとえば、複数の画面で同一のAPIがリクエストされていた場合、用途によってはオーバーフェッチングとなり、パフォーマンスの劣化につながる可能性があります。もちろん、オーバーフェッチングを回避するためにAPIを分離することも可能ですが、クライアントのユースケースが画面等、用途ごとに異なってきてしまう場合はそのぶんだけAPIを用意する必要が発生し、管理コストが増えてしまいます（**図10**）。

一方で、GraphQLはクライアントがユース

▼図10　REST APIの拡張イメージ

/api/posts	➡ postsのtitle/description/author/likesの情報
/api/simple_posts	➡ postsのtitleの情報のみ
/api/posts_with_authors	➡ postsのtitle/authorの情報
/api/posts_with_likes	➡ postsのtitle/likesの情報のみ

REST APIでは用途によってエンドポイントを分けるかどうかの判断が必要。
あるいはオーバーフェッチングを許容して多くの情報を取得するAPIを参照する形になることがある。

注13）「Signed QueryはGraphQLのTrusted Documentの新しい実装パターンです」
URL https://blog.studysapuri.jp/entry/2024/04/15/100000

▼リスト13　GraphQLの拡張イメージ

```
# クライアントのクエリ、クライアントで
# クエリを好きに組み立てることで
# 個別のユースケースに対応できる
query postsWithWhatYouWant {
  posts {
    title
    author {
      name
    }
    likes {
      userId
    }
  }
}

# サーバのGraphQLスキーマ定義、クエリできる
# 内容を定義しておくだけで
# あとはクライアントが好きに組み合わせて
# リクエストできるようになる
type Query {
  posts: [Post!]!
}

type Post{
  title: String!
  author: Author!
  likes: [Like!]!
}

type Author {
  name: String!
}

type Like {
  userId: ID!
}
```

ケースをクエリで表現するようになっているため、これらの問題を回避してAPIを拡張していくことが可能になります（**リスト13**）。

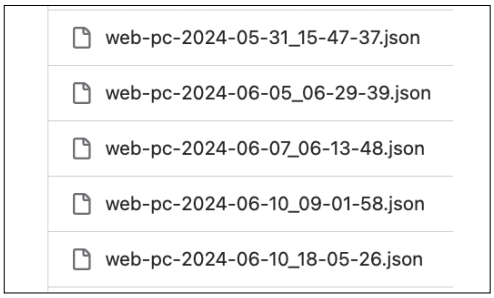
内部利用APIの廃止

APIの利用状況を把握し、マイグレーションを周知する

参照されなくなったAPIを廃止していくのも、APIを健全な状態に保っていくためには必要です。しかし、本当に参照されなくなったかどうかを判断するのは難しいです。REST APIを採用しているサービスの場合、APIの特定のフィールドが参照されているかどうかは、詳細な実装まで見る必要があります。

ほかにもエンドポイントに対するリクエストの実績をモニタリングすることで、廃止するかどうかを判断することも可能です。しかし、実績値での判断になるため、その期間中たまたま対象のAPIがリクエストされなかった可能性もあります。

▼図11　ファイル名がクライアント種別とビルドされた日時で構成されている



一方でGraphQLでは、セキュリティ対策の内部利用で紹介した、Trusted Documentというしくみの中で、あらかじめクライアントがクエリする内容を収集できます。そのため、どのようなクエリが実際にリクエストされるのかを静的に残すことができ、APIの現在や、過去の利用状況を把握することが比較的容易です。

実際に筆者らは、**図11**のようにファイル名にクライアント種別とクエリが収集されたタイムスタンプで構成されたTrusted Documentのファイルを保存しており、これを用いて安全にフィールドなどを削除できるかを判断しています。

また、削除するフィールドなどを明確にしてマイグレーションを促進するために、3-2で紹介した@deprecatedディレクティブを利用してGraphQLスキーマを定義し、**リスト14**のようにクライアントに対象のフィールドやenumの値が廃止される予定であることを予告しつつ新たなマイグレーション先の案内をしています。

ネイティブアプリケーションに対してAPIを提供している場合

iOSやAndroidアプリをクライアントとして提供している場合、APIの破壊的な変更などを

▼リスト14　@deprecatedディレクティブを利用したマイグレーション促進

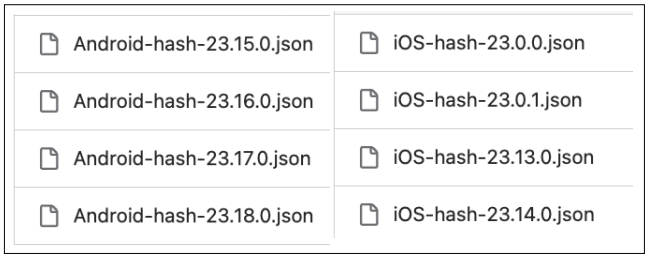
```
type query {
  (..略..)
  getLessonProgresses(lessonCodes: [String!]!): [LessonProgress!] @deprecated(reason: "Use ➥
 `lessonProgresses` instead.")

  lessonProgresses(lessonCodes: [String!]!): [LessonProgress!]
  (..略..)
}
```

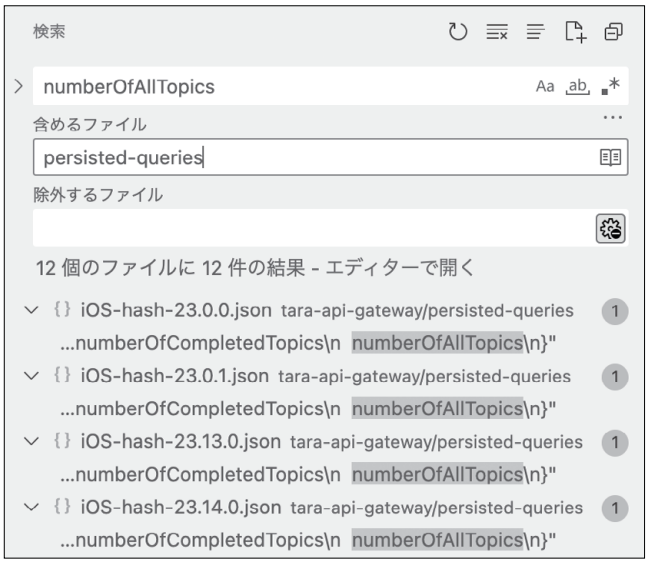
適用する際には、一定以上のバージョンを強制するしくみなどが必要になってくるでしょう。REST APIに対してリクエストを行っているアプリの場合、そのアプリのバージョンがAPIのどのフィールドを参照していたのかを過去の実装を遡って確認する必要があり、バージョンの数、実装の複雑さ、クライアントコードを確認する人間の言語に対する習熟度などによってはコストが増えてしまいます。

GraphQLの場合、この問題は先ほど紹介したTrusted Documentを利用することで解決できます。実際に筆者らのサービスでは、アプリケーションのビルド時にクエリを収集し、Trusted Documentを生成、その名前に**図12**のようにアプリのプラットフォームとバージョンを用いることで、どのアプリのバージョンでどのクエリが使用されているのかを簡単に把握できるようにしています。

▼図12　Android（左）やiOS（右）のPersisted Queryファイル名



▼図13　Persisted Queryを用いたAPI使用状況の把握



このようにバージョンごとにまとめることで、たとえばnumberOfAllTopicsフィールドがdeprecatedであるとしたときに、**図13**のようにコード検索をすることで、どのバージョンのアプリがそれを利用しているのかをすぐに確認でき、そのフィールドの実装を削除してもよいのかどうか判断できるようになります。

先述した@deprecatedディレクティブを利用してマイグレーションを促し、新しいアプリのバージョンでマイグレーションが完了したことをTrusted Documentから確認したあとに、マイグレーションが完了したバージョン以降に制限するように強制バージョンアップします。このようなしくみを採用することによって、筆者らは安全にネイティブアプリケーションをリリースしています。SD

Masato Matsuwaki, for y@lambda.publishing.com

OS・Web・プログラミング・技術者のスキルアップのための総合誌

Software Design [ソフトウェアデザイン] 総集編 2018→2023

好評発売中!!

6年分のバックナンバーがこの1冊に!

Software Design2018年1月号～2023年12月号までの特集、連載、一般記事、特別企画の記事PDFを収録。
総ページ数12,000ページ超。

- ■1号→1PDFファイルで、タブレットからも読みやすい
- ■PCからは全号一括検索も可能
- ■Webから同データをダウンロードできるシリアルコードも同梱
- ■書き下ろし特集「生成AI時代を生き抜くITエンジニアの生きる技術」



B5判　88ページ
定価2,640円（本体2,400円+税）
ISBN=978-4-297-14471-5

技術評論社　全国の書店、またはオンライン書店でお買い求めください。
〒162-0846　東京都新宿区市谷左内町21-13　販売促進部　TEL:03-3513-6150　FAX:03-3513-6151

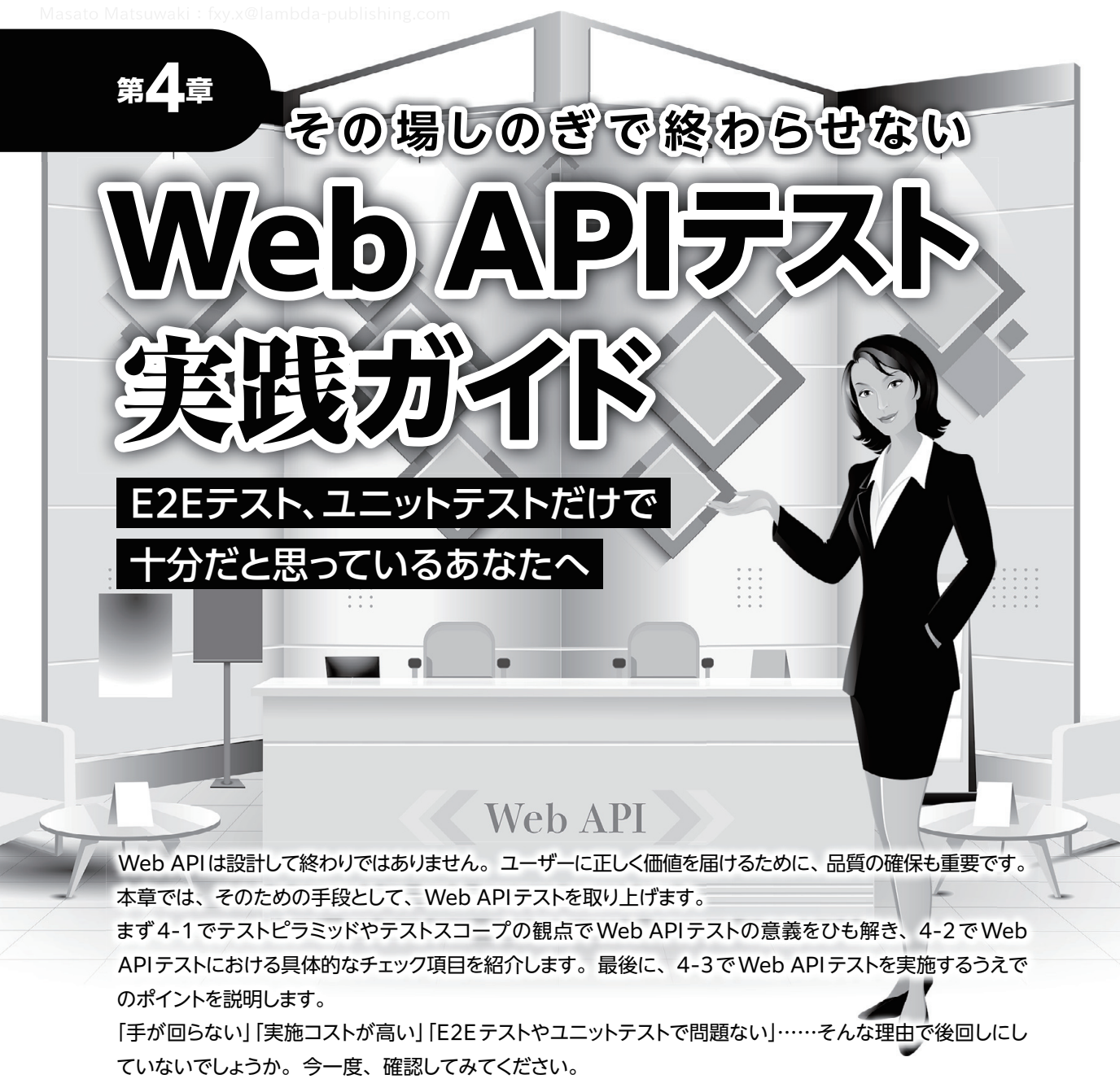
Masato Matsuwaki : fxy.x@lambda-publishing.com

第4章

その場しのぎで終わらせない

Web APIテスト実践ガイド

E2Eテスト、ユニットテストだけで十分だと思っているあなたへ



Web APIは設計して終わりではありません。ユーザーに正しく価値を届けるために、品質の確保も重要です。本章では、そのための手段として、Web APIテストを取り上げます。

まず4-1でテストピラミッドやテストスコープの観点でWeb APIテストの意義をひも解き、4-2でWeb APIテストにおける具体的なチェック項目を紹介します。最後に、4-3でWeb APIテストを実施するうえでのポイントを説明します。

「手が回らない」「実施コストが高い」「E2Eテストやユニットテストで問題ない」……そんな理由で後回しにしていないでしょうか。今一度、確認してみてください。

4-1 Web APIテストの意義
テストスコープ中心で考えるアプローチ
Author sumiren ——— P.130

4-2 Web APIテスト時のチェック項目
ユーザー体験とセキュリティを両立させるポイント
Author 川崎 庸市 ——— P.138

4-3 実践的なWeb APIテストの考え方
APIの品質を担保するヒント
Author こたうち さんさん ——— P.147



4-1 Web APIテストの意義

テストスコープ中心で考えるアプローチ

ひとくちにWeb APIテストといっても、その目的や方式によって得られる効果は大きく異なります。本稿では、Web APIテストがカバーしうるスコープや技術的特性を体系的に解説します。プロダクトや組織に応じて適切なWeb APIテストの意義を設定し、それを実現するための方式を選択できるようになるでしょう。

Author sumiren X(Twitter) @sumiren_t

異なるテストスコープを組み合わせる意義



現代のソフトウェアエンジニアリングでは、E2Eテストやユニットテストなど、さまざまな自動テストの方式やサイズを組み合わせて品質と生産性を高めます。本稿では、そうした自動テストの一種としてのWeb APIテストについて解説します。本稿ではまず、テストサイズの組み合わせを考えます。その後Web APIテストの立ち位置やメリット、テストアーキテクチャを考察し、Web APIテストの意義をひも解きます。

E2Eテストだけだとパターンが爆発的に増加する

まずはE2Eテストしか存在しない場合を考えてみます。ここでいうE2Eテストとは、ユーザージャーニーに沿って実際にフロントエンドを操作して、フロントエンドからデータベース含むバックエンドまでを一気通貫でテストするものを指します。その特性から、結果的にこれらをホスティングするインフラもテストの範囲に含まれます。

E2Eテストはその特性上、最も多くのコンポーネントを結合したテストが可能です。また、エンドユーザーの最終的な体験を検証できます。仮にフロントエンド単体やバックエンド単体の外部品質や、その中の個別の関数などの内部品質を保ったとしても、最終的にすべてを結合したときに想定どおりにシステムが動くとは限りません。そうした意味では、E2Eテストは最も安心しやすいテストになります。

それでは、なぜすべての自動テストをE2Eテストでまかなわないのでしょうか。一般に言われることとして、E2Eテストは遅いこと、そしてメンテナンスコストがかかるということが挙げられます。たとえば、「The Practical Test Pyramid」という記事でも次のように書かれています[注1]。

Furthermore, end-to-end tests require a lot of maintenance and run pretty slowly.

この記事が書かれたのは2018年のことですが、本稿執筆時点の2024年においても、状況は大きく変わっていないと言えるでしょう。

しかし、すべての自動テストをE2Eテストでまかなわない最も大きい理由は別にあると筆者は考えています。それは、E2Eテストでは個別の振る舞いまで網羅的にテストしようとすると、ケース数は振る舞いのかけ算となり爆発

注1) https://martinfowler.com/articles/practical-test-pyramid.html#End-to-endTests



的に増加することです。たとえばECサイトでカートに商品が1つあるとします。ユーザーはゴールド会員で10%オフで商品が買えます。商品はセール中で20%オフだとします。これらの割引が両立する場合、ユーザージャーニーに沿ったE2Eテストで、結果が28%オフになっていることを確認するだけで安心できるでしょうか。実は両方ともバグがあるのにバグが重なった結果として正常な値を示しているかもしれません。セール商品がないときには会員側の割引が適用されていないかもしれません。

考え過ぎのように思えるかもしれませんが、これが価格を決めるというビジネス上クリティカルなロジックだとしたら、不安を感じることは妥当でしょう。不当に金額が高く計算されれば顧客エンゲージメントに大きなリスクがありますし、誤って低い金額が算出されれば事業継続性のリスクがあります。しかも、ポイントやクーポン、カート内の複数商品の組み合わせやサブスク商品との組み合わせなど、実際には組み合わせの要素はもっと多いはずです。ユーザージャーニーに沿ったE2Eテストですべてを網羅するには、膨大なテストケース数となるでしょう。

ユニットテストだけだとクラス間やサービス間の結合がテストできない

ユニットテストの強みは、そうした振る舞いを個別撃破することで、パターン数の爆発を抑えられることです。たとえば、会員タイプごとの金額の変動、商品のセール時と未セール時、ポイントを使う場合と使わない場合など、組み合わせであれば3×2×2×……となるところを、3+2+2+……といったリニアなケース数の増加に抑えることができます。これが可能となるのは、ユニットテストが、クラスや関数、UIコンポーネントなどに対して書くことができる内部的なテストだからです。E2Eテストでセール時の価格を単体でテストしたいと思っても、どうしても会員という概念は入ってきてしまいます。デフォルトの会員種別として一般会員を選んだとしても、セール時の価格と一般会員の両方にバグがあった結果、値が正常に見えているだけという可能性を否定できません。

また、振る舞いを分解してテストし内部品質を高めることは、長期的な品質を高めることにもつながります。E2Eテストはシステム全体がアウトプットする結果に対するブラックボックステストであり、過程を検証しません。先述のとおり、複数の振る舞いが混じり合うことで、バグがうまく隠れてしまい、あるとき機能変更で別の機能の既存バグが顕在化するといったこともよくあります。メンテナンスコストについても、E2Eテストはテスト結果に関わる振る舞いが1つでも機能変更されれば期待される結果が変わる懸念が高く、メンテナンスの手間がかかります。ユニットテストでは、全体の結果を生み出す過程である個別の振る舞いをテストできるため、より細かい粒度で丁寧に品質にアプローチでき、書いたテストのライフサイクルはより長期的なものになります（**図1**）。

技術的にも、ユニットテストは作成コストが低く、実行速度が速いです。ユニットテストがテスト対象である小さな関数やモジュールを含めたプログラムであるからです。

一方で、すべてのテストをユニットテストで統一すると、E2Eテストで担保可能だった品質の一部をドロップすることになります。たとえばそもそもバックエンド内のクラス間を組み合わせたり、マイクロサービス間を組み合わせたりしたときに想定どおり動かなかったらどうでしょうか。また、たとえばコンテナやインフラの設定が誤っていて、そもそも本番環境でサーバが立ち上がっていなかったらどうでしょうか。こうした品質の担保には、より本番環境に近い状態でのテストが必要になります。内部品質にアプローチすることで得られているユニットテストの強みは、そうした広いスコープでの品質の検証とのトレードオフになります。

テストピラミッドを意識して組み合わせる

このように、ユニットテストだけでもE2E

▼図1　E2Eテストで振る舞いを網羅する場合の問題と、ユニットテストでどう改善されるか

E2Eテストの場合

テストケース	振る舞いA 3パターン	振る舞いB 2パターン	振る舞いC 2パターン	期待される結果
E2Eケース1	パターン1	パターン1	パターン1	0%オフ
E2Eケース2	パターン1	パターン1	パターン2	10%オフ
E2Eケース3	パターン1	パターン2	パターン1	20%オフ
E2Eケース4	パターン1	パターン2	パターン2	28%オフ
…				
E2Eケース12	パターン3	パターン2	パターン2	35%オフ

ただでさえ重たいE2Eのテストケース数が増大

1つの振る舞いが変わるだけで多くのテストケースの期待される結果が影響を受ける

ユニットテストの場合

テストケース	振る舞いA 3パターン	振る舞いB 2パターン	振る舞いC 2パターン	期待される結果
ユニットケース1	パターン1	—	—	0%オフ
ユニットケース2	パターン2	—	—	5%オフ
ユニットケース3	パターン3	—	—	10%オフ
ユニットケース4	—	パターン1	—	0%オフ
…				
ユニットケース7	—	—	パターン2	10%オフ

軽量なうえ、テストケース数もリニアな増加に抑えられる

限られたテスト対象の仕様変更にしか影響を受けない

テストだけでも、得られる品質特性に偏りが生まれてしまいます。そこで、各テスト方式を組み合わせることで、効率的かつバランスよく品質と生産性を維持できます。

こうした考え方自体は目新しいものではないと筆者は考えています。伝統的なウォーターフォール開発で利用されるテストスコープの分類である「ユニットテスト」「結合テスト」「統合テスト」等も、すべてを結合してテストするとケース数が増大したり十分にカバレッジが得られなかったりすることから、分解して丁寧に品質を得ることで、よりスコープの大きなテストのケース数増加を抑制する意味合いが含まれていると考えます。

組み合わせるにあたり、ユニットテストのさまざまな利点をふまえると、なるべく多くの品質をE2Eテストよりもユニットテストでまかないたいということになります。どちらか一方しか書かないのであれば、E2Eテストを10ケース書くよりユニットテストを100ケース書くほうが、より効果的に品質のカバレッジを高めることができ、より内部品質が高まって長期的な品質と生産性に寄与し、より作成コストやメンテナンスコストが低く、実行速度も速いです。

コストのかかるE2Eテストでは、クリティカルユーザージャーニーといった最も重要なユーザーシナリオ（商品が購入できることなど）や、機能横断的にインフラやアプリケーションが起動しているかの検証などに注力することが理想的です。

一般に言われるテストピラミッドも、このような考え方と整合するものです。本質的には、より多くの品質を内部品質でカバーするという方針だと筆者は考えます（**図2**）。

Web APIテストの意義



Web APIテストとは、Webサーバの公開エンドポイントの振る舞いに対するテストです。たとえば、GET /usersに実際にリクエストを送ってみて、返ってきた結果を検証するといったテストは、Web APIテストです。本項では、

ここまでに議論してきたテストピラミッドのトピックもふまえながら、Web APIテストの意義を議論します。

E2Eテストとユニットテストの中間サイズとしてのWeb APIテスト

ユニットテストは最も内部品質に近いテストで、1モジュールの振る舞いをテストするものです。E2Eテストはフロントエンドや複数のマイクロサービス）を統合してテストするものです。この2つだけでも丁寧に品質にアプローチできると思われますが、それぞれ極端な性質を持つとも考えられます。中間的なサイズのテスト、たとえば1サービス単体[注2]に対するテストがあれば、よりきめ細やかに品質を得ることができるのではないでしょうか。

筆者はWeb APIテストの利点がここにあると考えています。たとえば、あるサービスの注文APIについて、在庫がなければエラーを返す仕様だとします。ユニットテストだけでは、モジュールが組み合わさったときにサービスの外部品質としてこの仕様が満たされるか不安を覚えるかもしれません。逆にE2Eテストでこれをテストすると、ここまで議論してきたとおり、ほかのサービスの振る舞いとの組み合わせで、ケース数の肥大化やバグの摘出漏れが発生するかもしれません。こうした中間的なサイズのテストを行うのに、Web APIテストはちょうどよいと筆者は考えます。システム全体レベルで見れば内部品質、サービス単体レベルで見れば外部品質にアプローチするWeb APIテストは、前者のより長期的・効率的に品質を高めるという特性と、後者の信頼性と安心感という特性の、両方を備えています（**図3**）。

UIテストの代替としてのAPIテスト

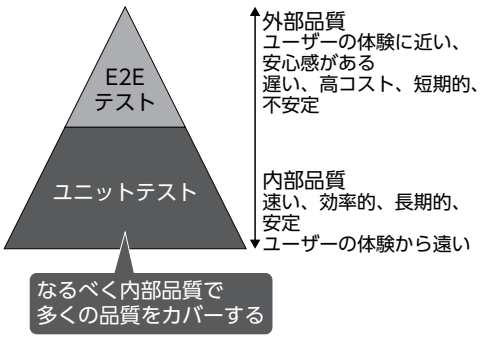
また、Web APIテストには、技術的なメリットとして、UIベースではテストしづらい部分をテストできるメリットもあります。

たとえば、消費者からショップ側のデータが見えないという認可の仕様を検証するにはどうしたらよいでしょうか。バリデーションロジック自体はシンプルなので、どちらかというとリスクは単純なバリデーション忘れであり、ドメインオブジェクトに対する狭義のユニットテストでは安心感がないでしょう。技術スタック次第ではこの仕様はフレームワークに対するフックで実装されるかもしれず、いわゆるプレゼンテーションレイヤとの結合が避けられないかもしれません。

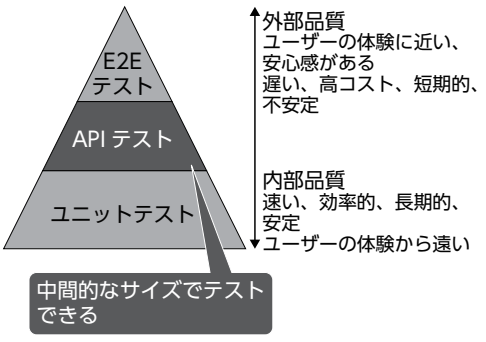
一方で、UIを用いたE2Eテストでこのテストを実施することは難しいです。たとえば消費者のアカウントでログインしてショップの管理画面にアクセスできないことを確認しても、それはフロントエンドで認証トークンのメタデータ等を見て出し分けているだけかもしれません。したがって悪意のあるユーザーが直接Web APIを呼び出したときにデータを守れるとい

注2）本稿ではフロントエンドやモノリスのバックエンド、マイクロサービスすべてを抽象化して、サービスと呼称します。

▼図2　E2Eテストとユニットテストのテストピラミッド



▼図3　Web APIテストの立ち位置



う保証がありません。

このように、Web APIテストには、E2Eテストとユニットテストの中間的なサイズのテストという強みと、技術的にUIでは難しい部分を検証できるという強みがあります。

テストピラミッドとWeb APIテスト



テストスコープとテスト方式

ところで、テストのサイズ（以降、本稿では、このテストのサイズのことをテストスコープと呼称します）はテスト方式やテスト技術によって決まるのでしょうか。たとえば、すべてないしは複数のシステムを統合してWeb APIテストを行った場合、それは本当に中間的なスコープということになるのでしょうか。フロントエンド単体でUIテストを行ったら、それは本当にE2Eなのでしょうか。

筆者は異なると考えています。テストスコープは、テスト方式とは分けて考えられます。Web APIテストというテスト方式でも複数システムを統合すればそれはE2Eのスコープであり、UIテストというテスト方式でもバックエンド以降をすべてスタブ[注3]にしたとしたら、それはサービス単体のスコープです。

そして、重要なこととして、筆者はテストピラミッドの層分けの軸は、テスト方式ではなく、このテストスコープであると考えています。上記の例に関して言えば、前者はスコープが大きいためWeb APIでありながら相対的により短期的で安心感があるという特性を持ち、後者はスコープが小さいためUIテストでありながらより長期的でコスト効率が高いという特性を持つでしょう。

ただし、テスト方式とテストスコープの間にまったく依存関係がないというわけではありません。テスト方式とテストスコープには相性があると考えています。たとえば、UIテストをサービス単体スコープ（フロントエンド単体）で行うことは、あまり一般的ではありません。技術的に、UIテストというテスト形式が作成コスト・実施コストが高く、またUI自体が本質的でない理由で変わることも多いことから、細かくテストを書いて長期的・効率的に品質を得るという内部品質寄りのテストスコープと相性が良くないのだと考えます。

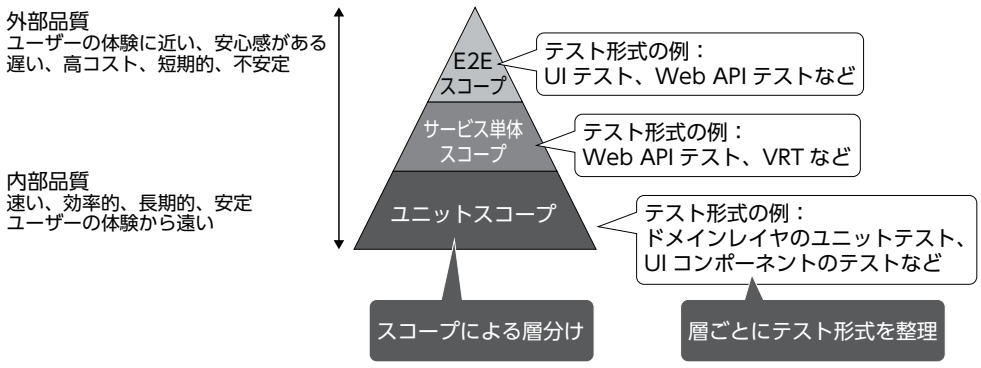
テストスコープの設計

以上のような議論から、テストピラミッドの考え方を採用する場合、まずは明示的にテストスコープによる層分けを提唱したいと筆者は考えています。そのうえで、層ごとにテスト方式やテスト技術を整理します。

層分けについては、たとえば「UIテスト」「Web APIテスト」といった層名は、テスト方

注3）https://martinfowler.com/bliki/TestDouble.html

▼図4　スコープによる層分けと、スコープごとのテスト形式の整理



式を指しているように思われます。たとえば、「E2Eテスト（E2Eスコープ）」「サービステスト（サービススコープ）」「ユニットテスト（ユニットスコープ）」といった3層であれば、スコープを指していることが明確です（**図4**）。

層の数と名前については、組織の構造やシステムの特性をふまえて意思決定すべきです。たとえばテンプレートエンジンを用いた伝統的なMPA（マルチページアプリケーション）であれば、E2Eとユニットの2層で十分かもしれません。また、E2Eテストの定義もプロダクトの特性によるでしょう。すべてのサブシステムを統合したうえでのテストは明らかにE2Eと呼べますが、任意の2つ以上のサービスを統合したテストをE2Eと呼ぶかはあいまいです。前者と後者を区別するような組織では、E2Eスコープとサービステストの間のスコープとして、たとえば統合スコープといったスコープを設ける必要があるかもしれません。

こうしてテストスコープによる層分けをしたうえで、テスト方式やテスト技術を層ごとに整理します。Web APIテストを実施する場合にも、どの層、つまりどのようなスコープで実施するかの意思決定が重要です。

Web APIテストの目的からテストスコープを決める

それでは、Web APIテストはどのテストスコープに入るのでしょうか。筆者はWeb APIテストは複数のスコープと相性が良いと考えています。Web APIに関わる複数のサービスを統合してテストするとすれば、それはE2Eスコープや統合スコープと言えます。Web APIが属するサービス以外をすべてスタブ化するのであれば、それはサービススコープと言えます。

どちらのスコープでWeb APIテストを実施すべきかは、Web APIテストの目的次第です。たとえば障害の原因がそもそもチーム単体のサービスの品質にある、といった状況や、ユニットスコープとE2Eスコープだけでは極端過ぎる、という状況では、サービススコープでWeb APIテストを実施することに価値があります。一般には、サービスに属するバックエンドやデータベース等のミドルウェアをすべて統合したうえで、その外側への依存をすべてスタブに置き換えます。このようにスコープを小さくして中間的なスコープのテストにできれば、より効率的・長期的に品質をカバーすることが可能でしょう。

ただし、一般に内部品質寄りのテストがうまく機能するための前提として、コンポーネント間が疎結合に設計されている必要があります。「ドメインロジックがコントローラーに集まっておりユニットテストにリファクタリングが必要」といった状況は多くの方が経験しているかと思いますが、これは内部品質に関わるスコープのテスト全般に言え、サービススコープのテストとサービス間の疎結合性にも当てはまります。サービスのシグネチャや振る舞いがうまく設計されていなければ、単一のサービスを切り出してテストする価値は薄れます。マイクロサービスの大規模な切り直しやモジュラモノリスに移行する予定、サービスのインターフェース技術を変更する予定（たとえばRESTからgRPCへなど）があるような場合なども注意が必要です。

一方で、複数のチームが開発する各サービスを統合したときにうまく動作せず障害が起きる、といったエンジニアリング課題があるのであれば、E2EスコープでAPIテストを実施することには価値があるでしょう。E2EスコープのUIテストと比較すると、UIが含まれない分、ややスコープが小さくなり、テストピラミッドの原理を考慮すると、やや長期的・効率的に品質をカバーできる性質を持つでしょう。ただし、テストピラミッド全体で見たときには最上位の階層のテストにあたるため、やはり主要な機能のみに絞ったり、機能ごとに主要なパターンだけ通したりなど、ケース数を抑えるための努力をすべきと考えます。

個人的には、やはりテストピラミッドの原理

をふまえ、なるべく多くのテストを内部品質に対するものにしたいと考えます。とくにE2Eや統合のスコープで行う理由がない状況であれば、サービススコープでWeb APIテストを行う意思決定をするでしょう。また、そもそも1つのテスト方式を1つのスコープでしか実施してはいけない理由もありません。必要であれば、サービススコープとE2Eスコープのそれぞれで、それぞれのスコープに適したWeb APIテストを設計してもよいでしょう。

意義とシステム特性に応じたWeb APIテストアーキテクチャ



Web APIテストを技術的にどう実施すればよいでしょうか。正解はありませんが、筆者はテストスコープ次第でポイントが変わってくると考えています。小さなスコープでテストするのであれば、その強みを活かせるように、内部品質にアプローチした決定性のある細かいテストをたくさん書くことができるアーキテクチャを採りたいでしょう。大きなスコープでテストするのであれば、より本番環境に近い構成でテストすることで、品質カバレッジを広げられます。

テストフレームワークによるWeb APIテスト

サービススコープでWeb APIテストを行う場合、そのスコープの小ささを最も活かせるアーキテクチャとして、ユニットテスト同様のテストフレームワークで実行する方法があります。フレームワークがテスト関連の機能を提供していれば、実際にリクエストが起きたときの挙動をユニットテストのように簡潔なテストコードで記述できます。また、サービス単体の外部品質の検証にはなりませんが、Web APIテストの目的がインターフェースより振る舞いにあるのであれば、いわゆるユースケースレイヤやアプリケーションレイヤに対してテストを書くという方法もあります。

このようにxUnitやvitestといった各種プログラミング言語のテストフレームワークを使う場合、それは実質的にはユニットテストであると考える方もいるかもしれませんが、筆者はWeb APIテストと呼んでよいと考えています。重要なのは利用している技術ではなく、そのテストでカバーしようとしている品質であると考えるからです。

この方法のメリットは、非常にメンテナンスが楽で実行も速いことです。また、データベースのようなミドルウェアさえテスト環境上で用意してしまえば、あとはそのデータベースにテストのプロセス上からアクセスするため、テストデータの投入や削除がコードで実現でき、アサーションも簡単に書けます。下流サービスをスタブするのも、アプリケーションレイヤでDI（依存性の注入）するクライアントインスタンスを差し替えるといった選択肢があります。データベース、下流サービスのスタブ、バックエンド、テストスクリプト……といった形で、何本も個別のプロセスを立ち上げるようなCIの構築が必要ありません。

この方式のデメリットは、テストスコープを大きく絞ることが前提のため、機能横断的な品質特性をテストできない点です。実行環境や環境変数や実行時のコマンド、OSやハード、コンテナ環境など、テストコードの実行と本番のWebサーバ実行ではさまざまな差異があります。たとえばWebサーバ起動時のコマンドの間違いで、サーバ自体が起動しないバグを混入させたとすると、このテストフレームワークによるWeb APIテストでは摘出することは難しいでしょう。サービス単体でも、PubSub・キューを含んでいたり、Webサーバを複数含んでいたりすると、GitHub Actions上などのCIのコンテナ環境上でこうした構成を簡易的に模倣することが難しい場合もあります。

実リクエストによるWeb APIテスト

反対に、E2Eスコープでの Web APIテストなど、より広いスコープでのテストに適したアー



キテクチャとしては、実際にWebサーバを立てて、その外部から実際のリクエストを送るという方式があります。設計としては、CIのコンテナ上ですべてのコンポーネントを用意するほか、AWSやGoogle Cloudなどのプラットフォームに本番同等の構成でデプロイしてテストを実行する方法もあります。とくに、多くのWebサーバやミドルウェアをpull requestごとにテストコンテナ上にステートレスに作成することは、費用面や実行時間の観点から費用対効果が見合わない場合も多いでしょう。リクエスト送信と検証については、Postmanやrunnなどのツールを使ったり、スクリプトを書いたりします。

この方式のメリットは、サービス間からサービスの中のインフラやランタイムまで、より広範囲を巻き込んでテストするため、最終的なエンドユーザーの体験に対する信頼性を得られることです。また、サービススコープのWeb APIテストであっても、サービス単体の外部品質の検証に強く重きを置くのであれば、実際にWebサーバを立ててテストを行うことには価値があるかもしれません。先述のとおり、テストフレームワークを用いてテストしても、実際に外部からのリクエストに対して正しく振る舞うという安心は得られないからです。一方で、こうした観点は最小限のE2Eスコープのテストや外形監視でまかなうといった形で、それぞれのアーキテクチャを組み合わせていいとこ取りをするという方法もあります。

実際のリクエストによるWeb APIテストのデメリットは、やはりテスト自体のメンテナンスコストです。たとえば、テストデータを投入するのにもテスト対象のWeb APIを呼び出さないといけないかもしれません。E2Eスコープのためより一層外部仕様にフォーカスしたいという要求があるのもそうですし、技術的にも、Webアプリケーション外でのデータベースへの接続とデータ投入は手間がかかります。Webフレームワークやテストフレームワークとは別にAPIテストのためのフレームワークを選定し、それに習熟したり、CIを実装したりすることも必要になります。

まとめ



本稿では、Web APIテストの重要性と、その強みを引き出すためのテストアーキテクチャの設計について議論しました。

Web APIテストは、サービス単体という中間的なスコープを担い、テストピラミッド全体のバランスを整える役割を果たします。また、UI操作経由でテストしづらい部分をカバーする目的により、E2EスコープでWeb APIテストを実施する選択肢もあります。

サービススコープのWeb APIテストでは、特定のサービスやAPIの振る舞いに焦点を当てることができます。E2Eテストよりもスコープを絞って内部品質に向き合うことで、長期的・効率的な、品質にアプローチできる特性を得られます。こうした強みを引き出すためには、テストフレームワークを用いたアプローチなど、同じサービススコープでも、なるべく技術的な要素と手間を削ぎ落とした効率的なテストアーキテクチャが魅力的でしょう。

一方で、E2EスコープのWeb APIテストにおいては、本番環境に近い構成でテスト環境を用意し、実際のリクエストを流すことで、ユーザーに近い視点からの品質検証が可能です。こうした強みを引き出すためには、本番環境に近い構成を用意して実際のリクエストを流すことで、コストに見合うリターンを得られるでしょう。

組織の設計やシステムアーキテクチャ、抱えているエンジニアリング課題を総合的に考慮してWeb APIテストの目的とスコープを決め、それをふまえてテストアーキテクチャを決定することが重要です。SD



4-2 Web APIテスト時のチェック項目

ユーザー体験とセキュリティを両立させるポイント

Web APIのテストを考えるうえで重要なのが、何をチェックするかです。本稿では、Web APIのチェック項目を説明します。Web APIの品質や信頼性、セキュリティを担保するために何を確認すべきか、指針にしてください。

Author 川崎 庸市（かわさき よういち） Postman株式会社 X(Twitter) @yokawasa

APIはインターフェースであり、Web APIはそのWeb版のインターフェースです。APIには外部、内部、パートナーとの連携といったさまざまな目的がありますが、いかなる形態であってもAPIを通じたデータ交換が普及した現在、その品質がビジネスに直接影響を与える場面が増えています。そして、Web APIテストは、そのインターフェースの品質、信頼性、セキュリティを確保するために不可欠なプロセスです。本稿では、Web APIの品質を守るためにWeb APIテストで欠かせないチェック項目を紹介します。

なお、ここでのWeb APIは、REST APIを前提とします。

APIテストにおける確認ポイント

APIテストで確認すること

APIテストは、**インターフェースの品質、信頼性、セキュリティ**を確保するために行います。4-1「Web APIテストの意義」項で議論されているテストピラミッドでいうところのE2Eテストとユニットテストの中間層のテストにあたります。APIテストを大きく機能要件と非機能要件で分けると**表1**のようになります。

APIテストで確認しないこと

各レイヤでのテストには、それぞれカバーできる範囲があります。テストは多層的に行うのが望ましいでしょう。ユニットテストやE2Eテストと併せて、各レイヤのバランスを取りながら、全体としてシステムの品質が担保されるようにすることが大切です。

- モジュールレベルの信頼性（ユニットテスト）
- ユーザー操作／UIレベルの振る舞い、ビジネス要件が満たせているかどうか（E2Eテスト）

機能面のチェック項目

機能面のチェック項目といってもいろいろあ

▼表1 APIテストにおける要件

要件	詳細
機能要件	・APIコントラクトテスト：APIがコントラクトどおりの仕様で実装されているか ・機能テスト：APIが期待どおりの振る舞いであるかどうか
非機能要件	・セキュリティテスト：APIを通じたやりとりがセキュアか ・パフォーマンステスト：APIのパフォーマンスに問題がないか

ります。APIエンドポイントの各リクエストやレスポンスが仕様どおりの形式・構造になっているかのチェック（後述するAPIコントラクトテスト）であったり、APIがビジネスロジックやエラー処理を含めて期待どおりの振る舞い、動作を正しく行っているかのチェックがあります。ここではまとめて機能面のチェック項目として紹介します。

・コントラクト
・CRUD操作の一貫性、冪等性（べきとうせい）
・APIバージョンの互換性
・エラーハンドリング

コントラクト

「コントラクト」とはその名のとおり契約を意味し、Web APIにおけるコントラクトはAPI提供者と利用者間での合意した契約を指します。コントラクトにはリクエストやレスポンスの構造やデータ型、ステータスコードなどが含まれます。

コントラクトテストは、APIがコントラクトどおりに振る舞うことを確認するテストです。たとえば次のようなことを確認します。

・**リクエストとレスポンスのスキーマ整合性**：リクエストのパラメータやレスポンスのデータ構造や形式（JSON、XMLなど）がコントラクト（OpenAPIなど）と一致しているか
・**型と値**：リクエストの入力値が期待されるデータ型（文字列、整数、ブール、配列など）であり、許容範囲内にあるか
・**必須パラメータ、フィールド**：リクエストの必須パラメータやフィールドが欠落している場合、適切なエラーメッセージが返却されるか
・**エンドポイントとHTTPメソッドの動作**：コントラクトで定義されたエンドポイントで使用するHTTPメソッド（GET、POST、PUT、DELETEなど）が正しく機能するか
・**HTTPステータスコード**：各エンドポイントが正しいHTTPステータスコードを返しているか

REST APIにおいては、コントラクト定義の標準フォーマットとして、OpenAPI（Swagger）が広く採用されています。API仕様をこういった標準フォーマットで記述することで、関連ライブラリやツールと連携し、APIスキーマや入力値のバリデーションの実装を効率化できます。チェック漏れを防ぐためにも、標準フォーマットと関連ツール群をうまく活用するとよいでしょう。

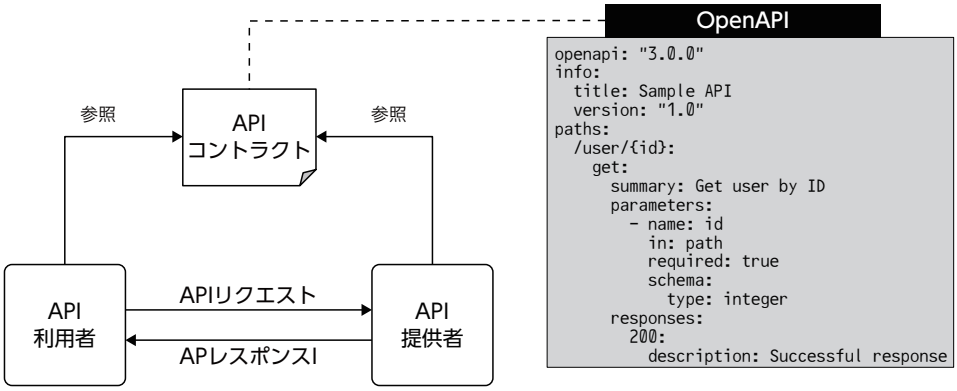
なお、コントラクトテストは、API提供者側と利用者側の両方で行われるべきテストであると言われています。API提供者は、提供するAPIがコントラクトどおりのレスポンスや動作を保証しているか確認します。フロントエンドやほかのクライアントに意図しない影響を与えないようにするためです。また、API利用者は利用しているAPIが期待どおりのデータと動作を提供するかを確認します。たとえば、APIのレスポンス形式が予期せず変更されてしまうと、利用者側のサービスでエラーが発生する可能性があるため、そういったリスク低減のためにテストを行います。したがって、コントラクトテストは、API提供者はコード変更のたびにリグレッションテストとして、またAPI利用者は定期的に実行して検証していくのがよいでしょう（**図1**）。

CRUD操作とデータ一貫性、冪等性

CRUD操作において必ず確認しなければならないポイントは、データ一貫性や冪等性です。たとえば次のようなことを確認します。

・冪等性が期待されない操作（例：POSTでのデータ作成）が重複して行われた場合、リソースが重複しない、ユニーク性が保たれているか

▼図1　API利用者・提供者によるコントラクトテストの利用イメージ



・冪等性が期待されるPUTやDELETEリクエストが複数回行われても、対象リソースの状態が変わらないか

一般的に、GET、PUT、DELETE、HEAD、OPTIONSは冪等性のあるメソッドとされています。これはもちろんサーバ側の実装によります。

冪等性とは、同じ操作を何回繰り返しても同じ結果が得られる性質のことを指します。APIにおいては、複数回APIリクエストを送信しても、1回のリクエストと同じ結果が得られ、リソースの状態が変わらないことを指します。よく、APIによる処理において一部のリクエストが失敗した場合や、タイムアウトでリクエストが再送される場合、同じリクエストが再度送信されることがあります。冪等性が守られていれば、リトライがあったとしてもデータの不整合が防げます。

APIバージョンの互換性

複数のAPIバージョンがある場合、各バージョンが互いに影響を及ぼさず、期待どおりに動作するかの確認が必要です。たとえば、次のようなことを確認します。

・新旧バージョンが互いに影響を及ぼさず、同時運用可能であるか（v2がリリースされてもv1利用ユーザーは引き続きv1を利用できる）
・変更内容が後方互換性をサポートする場合、既存API利用者はバージョンアップにより悪影響を受けないか（v1でサポートされたパラメータは引き続きv2でもサポート）

エラーハンドリング

エラーハンドリングの質はAPIの使いやすさと信頼性に直結します。HTTPステータスコードは、利用者にとって、エラーの種類を知るための重要な指標ですし、エラーメッセージは問題解決のための重要な手がかりになります。テストでは、エラーコード、メッセージが、利用者にとって正確で問題解決に役立つ情報が含まれているか確認しましょう。ただし、これは「セキュリティ面のチェック項目」項でも触れますが、エラーメッセージにAPIサーバ内部を知る手がかりとなる情報を含めるべきではありません。これらは攻撃者に利用される可能性があるためです。

セキュリティ面のチェック項目



APIは、外部と直接やりとりを行うため、セキュリティリスクも高くなります。安全性を保

つには、多岐にわたるテストを行う必要があります。本項では、重点的に検証すべきチェック項目を紹介していきます。

・認証・認可設定
・無制限のリソース消費
・データの漏洩（ろうえい）／過剰なデータ露出防止
・入力データのバリデーション（インジェクション／XSS防止）
・HTTPヘッダのセキュリティ設定

認証・認可設定

認証・認可は、Web APIのセキュリティにおいて最も重要な領域の1つです。認証とは、ユーザーが「誰であるか」を確認するプロセスであり、一方、認可とはユーザーが「何を許可されているか」を決定するプロセスです。この認証・認可が適切に実装されていないと、攻撃者による不正アクセスやデータ漏洩のリスクが高まります。Web APIテストにおいては、認証・認可の両観点で適切に実装されているか必ず確認するようにしましょう。たとえば、次のようなことを確認します。

認証の観点

・認証なしのアクセス試行：認証が必要なエンドポイントに対して、トークンなしのリクエストが拒否されるか
・トークンの有効期限の確認：トークンが付与されていたとしても、そのトークンが適切に期限切れとなり、期限切れ後のアクセスが拒否されるか

認可の観点

・オブジェクトレベルの認可：ほかのユーザーが所有するリソースにはアクセスできない（403 Forbiddenエラーが返る、など）ようになっているか
・機能レベルの認可：ユーザーの役割（例：管理者、一般ユーザー）に応じて、アクセス可能なエンドポイントや許可されている操作（例：他人のデータの削除）が制限されているか

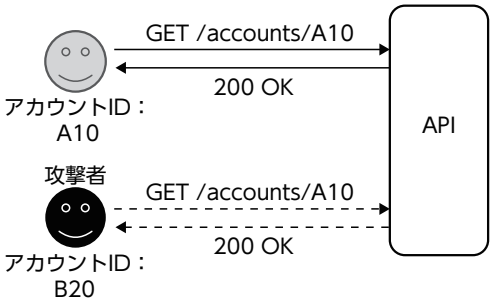
なお、「OWASP API Security Top 10 2023」[注1]では、認証・認可の不備に関する脅威が複数挙げられています（**表2**）。ぜひ、認証認可対策の際に一読することをおすすめします。

注1）https://owasp.org/API-Security/editions/2023/en/0x11-t10/

▼表2　OWASP API Security Top 10 2023における認証・認可の不備に関する脅威

カテゴリ	OWASP API Security Top 10 2023での指摘事項	リスク
認証の不備	API2:2023 - Broken Authentication（認証の欠如または不備）	認証設定に不備があると、攻撃者が認証なしでAPIにアクセスできたり、不正に認証を取得したりする。たとえば、パスワードリスト攻撃、トークンの再利用など。
オブジェクトレベルの認可の不備	API1:2023 - Broken Object Level Authorization（オブジェクトレベルの認可の不備） API3:2023 - Broken Object Property Level Authorization（オブジェクトプロパティレベルの認可の不備）	APIがリソースごとに適切な権限チェックを行わない場合、攻撃者が他人のデータにアクセスできてしまう可能性がある。たとえば、攻撃者が/accounts/B20を/accounts/A10に変更してほかのユーザーの情報にアクセスできてしまう。
機能レベルの認可の不備	API5:2023 - Broken Function Level Authorization（機能レベルの認可の不備）	たとえオブジェクトレベルでアクセス制限がされているとしても、APIが機能ごとに適切な権限チェックを行わない場合、攻撃者が認可のない管理者機能やほかのユーザー向けの機能を使用するリクエストが送信できてしまう。情報の漏洩やデータの改ざんにつながる可能性がある。

▼図2　オブジェクトレベルの認可の不備（BOLA）



▼図3　機能レベルの認可の不備（BFLA）

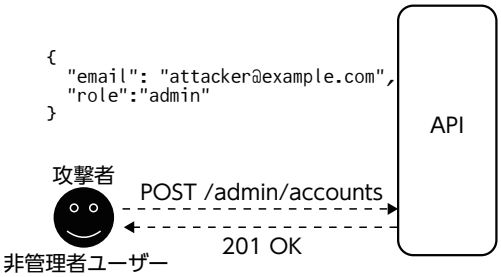


図2、3はオブジェクトレベルと機能レベルの認可の不備がある状況をわかりやすく表したイメージになります。

APIの消費リソースとレート制限

「OWASP API Security Top 10 2023」で挙げられているAPIのセキュリティリスクの1つとして「API4:2023 - Unrestricted Resource Consumption（無制限のリソース消費）」があります。APIリクエストに伴うリソース消費が制限されていない場合、攻撃者によるリソースの過剰消費でサービス妨害やコスト増加を引き起こす可能性があるとされています。

APIリクエストに伴うリソース消費の問題を防ぐために、**APIリクエストごとの消費リソース**と**APIリクエストのレート制限**の2つの観点で対策されているか確認するようにしましょう。たとえば次のようなことを確認します。

- **APIリクエストごとの消費リソースの制限**：各APIリクエストごとの過剰なリソース消費を抑えるための設定がされているか。タイムアウト設定、ペイロードサイズ、アップロードファイルサイズ、1回のAPIリクエストで実行可能な命令の数、返却可能なレコード数、など
- **APIリクエストのレート制限**：各エンドポイントに適切なリクエスト数の制限（例：10リクエスト/秒）が設定されており、制限を超えたらリクエストが適切に拒否されるか（HTTP 429 "Too Many Requests" など）

なお、補足情報として、レート制限には、API全体で固定時間枠ごとの全体リクエスト数を制限する方式、クライアントのIPアドレスやユーザーIDごとにリクエスト数を制限する方式などさまざまな方式があります。通常、API Gatewayで一元的にレート制限を構成するのが一般的です。

データの漏洩／過剰なデータ露出防止

読者のみなさんは、日ごろからプライバシー保護の観点でユーザーの個人情報の漏洩防止に気を使っていることと思います。しかし、意図せず公開された情報を通じて攻撃者に個人情報の特定や、システムの内部構造を理解させ、攻撃の足がかりを提供する可能性があります。APIを通じたデータの漏洩や過剰なデータ露出がないか必ず確認するようにしましょう。たとえば、次のようなことを確認します。

- クエリ文字列やURLパスの安全性確認：機密情報（例：セッションID、APIキーなど）がURLやクエリ文字列に含まれていないか
- APIレスポンスの安全性確認：必要以上の情報（例：内部ID、システムのメタデータ、機密情報など）がAPIレスポンスに含まれていないか



・エラーメッセージの安全性：エラーメッセージに内部システムの生のエラー、スタックトレースなどを含まないか

なお、検証方法としては、大きく「直接リクエスト／レスポンスを検証」「ログを検証」の2種類のアプローチがあります。前者はAPIに対して実際のリクエストを送信し、返されるレスポンスをチェックして、不要なデータや機密情報が含まれていないかを確認します。後者はサーバ側のリクエスト／レスポンスや内部ログを調査し、過剰なデータが送信されていないか、あるいは機密情報が記録されていないかを確認します。それぞれメリット、デメリットがありますが、結論としては、両方を実施するのが最も効果的です。

入力データのバリデーション（インジェクション／XSS防止）

Web APIの入力バリデーションにおけるインジェクション対策は、外部からの不正入力がシステムやデータベース、アプリケーションロジック、外部サービスに悪影響を与えるリスクを防ぐための重要な施策です。機能面でのチェックとして、データ型、長さ、範囲、形式などのバリデーションがありますが、これに加えてセキュリティリスクを考慮したバリデーションが必要です。

代表的な脆弱性としては、SQLインジェクションやクロスサイトスクリプティング（XSS）があります。これらの対策としては、インジェクション防止のライブラリ／ツールを使用したサニタイズや、WAF（Web Application Firewall）での防御などがあります。それぞれのセキュリティリスクを考慮し、適切なツールやサービスを選定し、APIライフサイクルの設計段階など初期のフェーズで導入していくことで、効率的かつ確実なセキュリティ対策が実現できます。ほかのセキュリティ対策同様、導入するだけではなく必ず期待どおりの設定がされているかテストするようにしましょう。

HTTPヘッダのセキュリティ設定

クライアントの利用方法を制限し、セキュリティリスクを軽減するためのすでに業界で確立されたセキュリティヘッダがあります。これら推奨セキュリティヘッダを設定し、APIテストの際にこれらが正しく機能しているかを確認することは、セキュリティレベルを向上させるうえで非常に重要です。具体的な確認ポイントは次のとおりです。

・Cross-Origin Resource Sharing（CORS）設定：クロスドメインの呼び出しがサポートされている場合、必要最小限のオリジンからのみAPIにアクセスできるように限定的な設定になっているか（例：Access-Control-Allow-Origin: https://api.example.com）
・レスポンスヘッダにCache-Control: no-storeが付与されているか
・レスポンスヘッダに適切なContent-Typeのタイプ設定が付与されているか（例：JSONの場合はapplication/json）
・レスポンスヘッダに適切なStrict-Transport-Securityの設定が付与されているか（例：HTTPS有効期間は1年間で、サブドメインもHSTSの対象とする場合はStrict-Transport-Security: max-age=31536000; includeSubDomains）
・レスポンスヘッダにX-Content-Type-Options: nosniffが付与されているか
・レスポンスヘッダにX-Frame-Options: DENYが付与されているか

表3に各レスポンスヘッダの推奨設定を示します（筆者による抄訳）[注2]。

注2）https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html#security-headers

▼表3　APIレスポンスヘッダの推奨設定

ヘッダ	説明
CORS関連ヘッダ（Access-Control-Allow-Origin、Access-Control-Allow-Headers、Access-Control-Allow-Methodsなど）	CORSは、ブラウザが異なるオリジンからのリクエストを制御するためのしくみ。許可するオリジンを制限しないと、APIが不特定多数のドメインからアクセス可能になり、セキュリティリスクが増大する。よって、できる限り必要最小限のオリジン、メソッド、ヘッダのみを許可することが推奨されている。
Cache-Control: no-store	ブラウザによるキャッシュを指示するために使用されるヘッダ。このヘッダにno-storeが設定されていると、このヘッダを含むレスポンスは保存されない。機密情報がキャッシュされるのを防ぐ。
Content-Type	コンテンツタイプを指定するヘッダ。もし指定しない場合、ブラウザは応答のコンテンツタイプを推測して指定したMIMEタイプとは異なる解釈を行うことがある。これによりMIMEスニッフィング攻撃で返される可能性がある。
Strict-Transport-Security（HSTS）	Web APIがHTTPSプロトコルでのみアクセスされるように強制するヘッダ。このヘッダにより、API呼び出しがHTTPS経由で行われるようになり、盗聴や改ざんの防止、クライアントが意図せずHTTPを使ってAPIにアクセスするミスを防ぐ。
X-Content-Type-Options: nosniff	ファイルの内容に基づいてMIMEタイプを判別するのではなく、Content-Typeヘッダで宣言されているMIMEタイプを常に使用するようにブラウザに指示するヘッダ。これにより、ファイルが予期しない方法で実行されるリスクを軽減する。なお、このヘッダの値はnosniffのみ。
X-Frame-Options: DENY	レスポンスを、ほかのサイトにフレーム化できるかどうかを指定するためのヘッダ。DENYを指定することで、レスポンスをほかのサイトにフレームとして埋め込むことを禁止できる。クリックジャッキング攻撃から保護できる。

パフォーマンスのチェック項目



APIは機能が正しく動作するだけでは十分ではありません。パフォーマンスも重要なユーザー体験の1つであると言われています。API提供者は、パフォーマンスの問題がアプリケーションの使いやすさを大きく損なう可能性があることを意識する必要があります。遅いAPIを使いたい人は誰もいません。そこで重要になるのが、「パフォーマンステスト」です。パフォーマンステストは、トラフィックの多い条件下でのAPIの安定性と信頼性を確認するプロセスです。

- **高負荷時の性能**：想定される高負荷のパターンにおいて、レイテンシーやエラーレートが許容範囲内か
- **長期安定性**：長期間の運用で異常なリソース消費（メモリリークやCPU使用率の異常値）がないか
- **スケーラビリティ**：サーバの数などAPIのリソース増減に応じてパフォーマンスが期待どおりに変動（水平スケールなど）するか
- **リソース効率**：CPU、メモリ、ディスクI/O、ネットワーク帯域などAPIサーバのリソースに過剰、無駄な消費がないか
- **APIの限界値**：APIが限界値（最大同時接続数、など）と限界値を超えたときの挙動について確認がとれているか

計測メトリクス

パフォーマンステストにおいて、APIの性能を定量的に評価するために計測する主要メトリクスを**表4**に紹介します。これらのメトリクスを通じてシステムの状態を判断します。

負荷のシミュレーションパターン

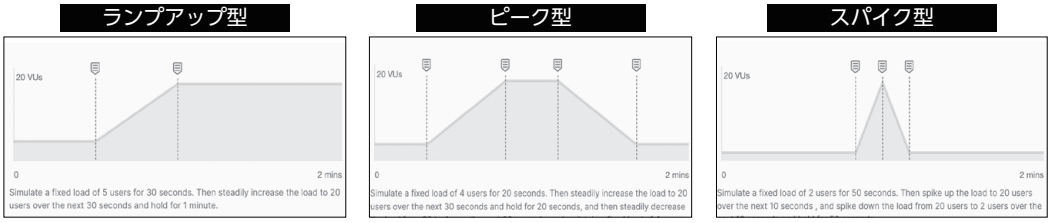
高負荷状態をシミュレーションするといってもさまざまな負荷のパターンがあります。たと



▼表4　計測すべき推奨メトリクス

メトリクス	計測内容	観点
レイテンシ	各APIエンドポイントの平均応答時間、95パーセンタイル(P95)、99パーセンタイル(P99)のレイテンシが許容範囲内か	ユーザー体験に直接影響を与える指標。平均レイテンシのみならず、P95やP99といった極端な負荷状況での性能を把握してボトルネック解消のための指針として利用する。
スループット	APIが1秒間に処理可能なリクエスト数(RPS：Requests Per Second)	APIのスケーラビリティを測る指標。必要なRPSが達成できない場合、インフラ拡張やコード最適化などの対策を検討する。
エラーレート	5xx系および4xx系のエラーレスポンス率	システムの不安定さを示す指標で、5xxエラーはサーバサイドの問題を示す。高負荷下でエラーレート(とくに5xx系)が急増していないか確認する。
システムリソース使用状況	APIサーバのCPU使用率、メモリ消費量、ネットワーク帯域の使用状況	API処理の効率性や、ボトルネック特定のために利用する。

▼図4　Postmanパフォーマンステスト機能で設定可能な負荷パターン(固定以外)



えば、負荷を徐々に増加させてシステムの限界を特定するランプアップパターンや、劇的な負荷を作り出しその変化に対処できるかをテストするためのスパイクパターンなどがあります。使用する負荷テストツールによっては、複数のパターンで負荷をシミュレーションできるものもありますので、ツール選定をうまく行うとよいでしょう。

参考までに、Postmanの場合、パフォーマンステスト機能では、テスト全体を通じて負荷が固定のパターン以外に**図4**の種類の負荷パターンが設定可能になっています。

パフォーマンステストで気をつける点

そのほか、パフォーマンステストの注意点を紹介します。

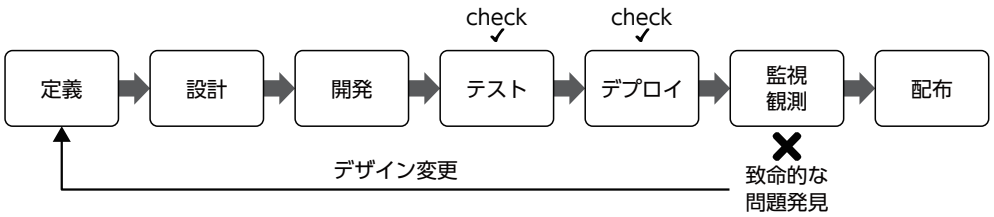
まず、仮想ユーザーごとにユニークなデータを使う点です。同時接続ユーザー数を増加させても、キャッシュが効いてしまうことで実際の負荷が反映されない場合があります。できるだけ仮想ユーザーごとにユニークなデータを使用するようにしましょう。次に、リアルなユーザーシナリオを再現する点です。実際の利用パターンを無視した単純な単一APIの負荷テストだけではなく、複数のAPIリクエストパターンを組み合わせた、リアルな使用シナリオでテストを設計しましょう。さらに、本番環境にできる限り近い専用のテスト環境で実施しましょう。サーバリソース、ネットワーク構成、データの量や多様性など、テスト環境を本番に近づけることで、パフォーマンステストの信頼性を高めることができます。

Web APIテストで心がけたいこと



最後に、Web APIテストで心がけたいこと

▼図5　APIライフサイクルにおける後半フェーズで問題が発見されたイメージ



を3点紹介します。

シフトレフトを心がける

テストを開発の初期段階に移行（シフトレフト）することで、早期にバグや問題を発見し、修正コストを削減できます。中でもセキュリティ、パフォーマンス面のテストおいては、まだまだAPIライフサイクルの後ろのフェーズで実施されることが多いですが、このような非機能要件での問題は往々にして致命的です。問題発見が後ろのフェーズになればなるほど改修コストが大きくなる傾向があります（**図5**）。できるだけAPIライフサイクルの早い段階で実施するようにしましょう。

APIドキュメントを整備する

Web APIのテストを成功させるには、APIドキュメントの整備が欠かせません。APIドキュメントは開発者やテスターにとって「仕様書」であり、「テスト設計」の起点になります。正確なドキュメントがあれば、どのリクエストがどのレスポンスを返すべきかが明確になり、それに基づいた適切なテストケースを作成できます。また、API仕様をOpenAPI仕様のような標準フォーマットで記述すれば、OpenAPIやPostmanなどの自動化ツールを活用することで、大部分のAPIドキュメントやテストスクリプトの自動生成が可能になります。これにより、迅速なテスト環境の構築が実現し、API仕様とテストコードのズレを回避しやすくなります。さらに、適切に整備されたAPIドキュメントを関係者全員が共有することで、仕様の認識齟齬（そご）を防ぎ、効率的なコラボレーションを促進できます。

優先順位をつけてテストを自動化する

テストは一般的に、自動化することが推奨されています。これにより、人間の手によるあいまいさやミスを減らすと同時に、繰り返しの作業を効率化し、開発スピードと品質の両立を図ることができるようになります。ただし、自動化は限られたリソースを考慮し、優先順位をつけて進めることが重要です。自動化すべき部分としては、頻繁に変更される箇所、繰り返しテストが必要で手動では非効率な部分、影響が大きいバグリスクの高い箇所（例：機密データ扱い処理やデータ整合性）、そして再利用性の高い共通機能が挙げられます。これらを優先的に自動化することで、効率的にテスト工数を削減し、見落としリスクを低減できます。

テストケースを継続的にアップデートする

APIは仕様変更や依存環境の変化などにより、時間の経過とともに変化します。APIの変化に合わせてテスト項目を定期的に更新し、不要なテストを削除するしくみを構築しましょう。不要なテストが増えると、テストの実行時間が長引くだけでなく、テストの更新作業に負担がかかり、新しい価値を生み出す作業に集中できなくなります。したがって、テスト項目を継続的にアップデートし、リファクタリングすることが不可欠です。また、必要以上にテストを増やさないことも重要です。SD



4-3 実践的なWeb APIテストの考え方

APIの品質を担保するヒント

本稿では「機能テスト」「パフォーマンステスト」「セキュリティテスト」について説明します。これらは、品質特性を担保するためのアプローチの1つでもあります。それぞれのテストがどんなもので、どのように実践するのかを具体的に見ていきましょう。

Author こたうち さんさん X(Twitter) @kotauchisunsun

機能テスト



REST APIの基本のテストパターン

「これからREST APIのテストを書こう」と思ったとき、どのようにテストを書いていけばよいでしょうか。もちろんプロダクトによって書くべきテストは異なります。これらは、テスト分析や、テスト設計など別の技量を必要とするので、どのようなプロダクトにも使えるような手法を提示することは難しいです。初心者の方に筆者がおすすめするのは、**表1**のようなHTTPステータスをベースにテストケースを考えることです。

たとえば、ユーザーの情報を更新するエントリポイントとして`PUT /users/:ユーザーID`があったとします。HTTPステータスが200のときには、正常にユーザーの情報更新が完了したことを示します。400の場合は、ユーザー名は英数字しか受け付けない仕様のときに、誤ってハイフンなどの記号を入力して送信したケースになるでしょう。また、一般的なシステムであればユーザーの情報を更新できるのは、そのユーザー自身だけです。その際には、認証情報として、Cookieやトークンをつけた状態でAPIを呼び出します。そういった認証情報がAPIの呼び出し時に付与されていない場合が401にあたります。先ほどと近い話になりますが、Aさんのユーザー情報をBさんという他人が書き換えられると問題があるでしょう。これが403エラーにあたります。404エラーは存在しないユーザーIDの情報を更新しようとしたときに発生します。

この手法を用いることで、一般的なCRUDのAPIのテストケースを網羅できます。もう少し幅広い視点を持ちたいのであれば、HTTPステータスをいろいろと調べてみるのもよいでしょう。本質的には、プロダクトごとにドメインが異なるのでまじめなテスト分析が必要になり、それをベースにテストを書くことが大切です。しかし、その補助線としてこういった視点を持っておくと、テストケースの整理にもなるでしょう。

▼表1 代表的なHTTPステータスコードと意味

HTTPステータス	意味
200(201)	正常に終了
400	入力エラー
401	認証エラー
403	権限エラー
404	リソースが存在しない

カバレッジは何%を目指すべきか

自動テストを書く際に、テストの網羅率（カ

バレッジ）というものも気になってくるでしょう。どれくらいのカバレッジを目指せばいいものか、と不安になることもあると思います。筆者の答えは「テストカバレッジは70%を目標にする」です。GoogleTestingBlog[注1]によると、Google社内ではテストカバレッジのガイドラインを次のように定義しています。

> 60% as “acceptable”, 75% as “commendable” and 90% as “exemplary.”

日本語に訳すと「60%が受容可、75%が推奨、90%が模範的」です。また、2014年のGoogle内のデータではカバレッジの中央値が78%になっていました[注2]。

したがって、これからテストを整備していこう！という目標のもと進めていくのであれば、70%程度を目標にしていくのがよいでしょう。もちろんプロダクトの状況によっても異なりますが、新規開発の場合、PRごとにカバレッジを計測できるようなしくみを作り、それぞれのPRで80〜90%程度を目指して開発をし続ければ、プロダクト全体で70%を達成することは、そこまで難しくはないはずです。既存のシステムの場合も似た方針で、新規機能開発する部分は同様にPRごとに80%から90%を目指します。一方で既存システムのテストが動いていない部分は、テストを書くように改善を行い、全体を70%に目指すのがよいと思います。

一方で、「カバレッジの数字に意味はない」という議論もあります。「サービスの運営を止めないために、重要な機能に対しテストを書くべきであって、カバレッジに執着しないほうがよい」という反論も正しいです。「テストを書く能力があるが、ビジネスの優先度として網羅的にテストを書かない」というのであれば、筆者は納得します。しかし、「テストを書く能力がなく、テストを書かない」という選択肢はないと考えています。筆者自身、技術記事として自動テストの内容を執筆していますが、全体を70%程度のカバレッジに保ち、実運用を続けるための技術力は、思っている以上に高いです。これを達成するには、単体テスト・結合テスト・受け入れテストのすべてが一通り書けるようになることが必要です。その難しさの一端に触れ、一通りのテストの実装スキルを学ぶためにも網羅率70%を目指す、ということに一度チャレンジしていただきたいです。

これはサーバサイドエンジニアのポジショントークだと思って聞いてほしいのですが、サービス運営において一番使われている機能はREST APIです。昨今は、Webページであったり、スマホアプリであったり、いろいろなシステムが連携して動きます。そして、それらのアプリの多くはREST APIがないと動きません。別の観点を考えてみると、Androidのアプリへの修正はAndroidユーザーのみに影響し、iOSのアプリへの修正はiOSユーザーのみに影響します。しかし、REST APIの修正の影響はAndroidアプリとiOSアプリ両方に影響する可能性があります。確かにREST APIはUXなどわかりやすく直接的にユーザーへのメリットを提供することはありません。一方で、データの機密性や一貫性、可用性を取り扱うのはREST API側であることが多いのです。したがって、REST APIはユーザーの利用数が多く、厳格なデータの制御を必要とする機能となりますので、自動テストを行い、品質を安定させることが重要だと言えます。

パフォーマンステスト



そのパフォーマンステスト本当に必要ですか？

はっきりと言うと、パフォーマンステストをすることは難しいです。筆者は趣味でベンチマー

注1）https://testing.googleblog.com/2020/08/code-coverage-best-practices.html?m=1
注2）https://testing.googleblog.com/2014/07/measuring-coverage-at-google.html?m=1



クを行うことがありますが、それは「どのプログラミング言語がREST APIの最速最強の言語か？」のような幼稚な知的好奇心を満たすためです。しかし、業務で行うソフトウェア開発では、正確に実施する必要があります。

ソフトウェア開発は主に次の3つに分類できます[注3]。

- 新規開発：ベースとなるシステムが存在せず、新規の開発を行うもの。ただし、ベースとなるシステムが存在する場合でも、新規開発部分が本プロジェクトの開発部分の約90%以上の場合は、新規開発として扱う。
- 改良開発：「改修・保守」と「拡張」の2つを合わせた開発を指す。
  - 改修・保守：リリース後のシステムの運用フェーズで、ベースとなるシステムが存在し、機能追加など改修を伴う開発を行う（新規開発部分は約10%未満である）。
  - 拡張：ベースとなるシステムが存在し、機能追加など改修を伴う開発を行う。（新規開発部分は約10～90%である）
- 再開発：既存システムが存在し、機能仕様を殆ど変更する事無く、作り直す場合。（いわゆるリプレース、モダナイゼーションを含む）

筆者は「ソフトウェアの『再開発』以外では、あまりパフォーマンステストのメリットがないのではないか」と思っています。それぞれの場合で考えてみましょう。

新規開発の場合

「新規開発」においてパフォーマンステストを行う必要があるでしょうか？　筆者は難しいと思っています。

新規開発は「ベースとなるシステムが存在しない」状態で「新しいビジネスが生まれるかもしれない」というもくろみから行うものでしょう。そのため、パフォーマンスの実現可能性よりビジネスの実現可能性を調べるほうが優先度は高くなります。

また、このように新規開発をしても、後々プロダクトの位置付けが変わったり、機能が変わったりしていきます。そうすると、初期で計測したボトルネックも変わってしまいます。そのような状況下でリリース前にパフォーマンステストを行ったとしても、計測したデータの有用性が低くなってしまいます。これはもちろん会社の規模にも依存する部分があり、大規模な会社で最初から大量の顧客がいる場合には行うべきです。しかし小さなスタートアップ企業の場合は、パフォーマンスに問題があれば、サーバのスケールアップなどお金で解決するほうがよいでしょう。

改良開発の場合

「改良開発」ではどうでしょうか。パフォーマンスに懸念が存在するのであれば行うべきではありますが、実態としては難しいのではないでしょうか。改良開発の場合、現行のシステムがあり、それに対して改修を入れることが前提となります。そのため、まず初めに現行のシステムの性能を知る必要があります。では、現行のシステムのパフォーマンスを計測するにはどのようにすればよいでしょうか？

一番ナイーブな解決方法だと、現行の本番システムに対してパフォーマンステストを行う、という方法がありますが、サービスを止める危険性があったり、データを壊すおそれがあったりするため、本番環境に対してパフォーマンステストを行う、ということは絶対にできません。

では、次に考えられるテスト戦略として「現行のシステムと同じ規模の負荷テスト環境を用

注3）独立行政法人 情報処理推進機構 社会基盤センター『ソフトウェア開発 データ白書 2018-2019』P7、P336より整理。https://www.ipa.go.jp/archive/publish/wp-sd/ps6vr70000011kwd-att/000069381.pdf

意する」という方法があります。しかし、本番環境と同等のシステムを用意するためのサーバ代もかかり、それらのサーバを構成するための手間もかかります。そのうえ、パフォーマンスが問題になるということは、すでにそこそこの規模のあるシステムでしょう。また、システム自体が外部のサービスに依存しており、パフォーマンステストを行いたいシステムだけ独立させること自体が難しいこともあります。

改良開発でパフォーマンステストを行いたい要求はあるかもしれませんが、それに対して、下準備にかなり工数がかかってしまうでしょう。

再開発の場合

一方で「再開発」の場合は比較的行いやすい部類に入ります。既存システムと分離され、新規でサーバを設置することも多いでしょう。そのため、現行のシステムに影響を及ぼすことはめったにありません。設計段階で現行のシステムの要求パフォーマンスも判明しているので、パフォーマンスの基準も考えやすいです。

また、改良開発と違って、ある程度システム同士の境界やプログラム上の設計を調整する余地があります。そのため、設計段階で外部のサービスへの依存を最小化する、モックを挟むといった工夫をしたり、負荷テストが行いやすいような構成を取ったりすることもできます。

何より、新規開発に比べて再開発はビジネスが安定しています（そうでなければ再開発する意義があまりありません）。会社としてビジネスとして盤石な部分であるため、変更が少なく、データを計測し、それをもとにパフォーマンスを推論することにメリットもあります。

◆ ◆ ◆

これらの事由から、パフォーマンステストをするなら、再開発のようなケースがコスパよくできるのではないでしょうか。

正確な実験のために

業務で行ったパフォーマンステストにあまり良い思い出はありません。サーバの台数を増やしても処理できるリクエストの量が上がらなかったり、サーバのセットアップに時間がかかり、1日に3、4回しか計測できなかったり、とにかく労力がかかるのに、出てきた計測データは理解が困難で頭を抱えるといったことが多かったです。

そのような悩みの原因は、正確に実験ができていなかった、ということが多かったです。それはサーバやクライアントのセットアップの問題であったり、パフォーマンステストに用いたマシンのスペックの問題であったりしました。この項では、そういったパフォーマンステスト、とくに負荷テストを正確に実験を行うためのTipsを紹介します。

サーバとクライアントを分ける

パフォーマンステストを行う目的は、あるスペックのサーバを本番環境へリリースしたときに、どれほどの時間で、どれほどの同時接続数でリクエストの処理が可能なのか見当をつけることです。したがって、パフォーマンステストの環境は、実際の本番環境と近い環境であるほうが望ましいです。

そのため、まず初めにサーバと負荷をかけるクライアントを別のマシンに分ける必要があります。当たり前ですが、負荷をかけるクライアントとサーバが同居し、両方の処理負荷がCPUやメモリにかかった状態で計測しても、本番のサービスの状況を再現しているとは言い難いです。そのため、クライアントとサーバを別のマシンに分けることが重要です。

サーバを過負荷状態にすることを目指す

筆者がパフォーマンステストを行う際に度々困ったのが「クライアントがサーバに負荷をかけきれない」ことです。

たとえば、同時接続数100で負荷をかけ、成功率100%でスループットが100rpsという結果が出たとします。その次の実験では、同時接



続数200で負荷をかけ、成功率100%でした。しかし、スループットが100rpsのままでした。パフォーマンステストでは、このような直感とは合わない結果が生まれることが度々あります。

この場合、同時接続数200でリクエストするために必要なクライアント側のマシンのスペックが足りないというケースがあります。そのため、パフォーマンステストにおいては、クライアント側にも十分な性能が必要となります。

それが「サーバを過負荷状態にすることを目指す」の真意です。クライアントが負荷をかけきれない場合があるので、クライアント側がサーバを過負荷で停止できる程度に強い状況を作り、正確に負荷がかけられる環境を構築することで、実験を安定させるのが大切です。

クライアント側のスペックはどうすればよいか？

パフォーマンステストを行ううえで、2つのマシンを選定する必要があります。それは、サーバとクライアントです。サーバ側に関しては、サービスを運営していくにあたり、予算の都合もあるので、選択肢が絞られてくるでしょう。では、負荷をかけるクライアント側はどうすればよいでしょうか？　筆者は、最初は「クライアント側のスペックはサーバと同程度にすればよい」と考えています。

これは先ほどの内容と関連しており、クライアント側は、サーバ側を過負荷で停止できる程度のスペックが必要です。サーバ側とクライアント側でパフォーマンステストをする際の負荷について考えると、サーバがリクエストを処理する負荷に比べて、クライアント側がリクエストする負荷のほうが小さいと考えられます。そのため、クライアント側のスペックをサーバ側と同程度にしておけば、テスト時にサーバを過負荷にする程度の負荷をかけられると推察できます。したがって、初期の実験においては、サーバとクライアントを同スペックにしておくとよいでしょう。

サービスの増強戦略と実験計画

小規模な実験でマシンのスペックとシステムの限界値がわかったら、あとはどのような実験を行うか、という問題があります。

クライアント側は、同時接続数やリクエスト数のいろいろなパターンを出し、それに伴うスループットやレスポンス速度を計測することになるでしょう。サーバ側においては、2種類の戦略があります。

- 垂直スケーリング：マシンスペック自体を強化することで、CPUのコア数やメモリを増強する戦略
- 水平スケーリング：リクエストを受けるサーバの台数を2台、3台……と増やす戦略

昨今普及したクラウド環境においては、CPUやメモリの使用量によってサーバの台数を増減できるオートスケールという機能があります。そのためクラウド環境においては、水平スケーリングにより線形に性能向上が見込めると、負荷の状況によりサーバの台数を増減させることができるので、効率的なインフラの運用が可能です。

パフォーマンステストの際には、水平スケーリング・垂直スケーリング、両方のテストを行い、自分たちが作っているソフトウェアの特性を知るのがよいでしょう。たとえば画像をリサイズするAPIの場合、画像をメモリに展開する必要があるため、メモリやCPUを増強する垂直スケーリングが有用でしょう。一方でユーザー情報の取得・更新といった一般的なCRUDであれば、メモリやCPUはそれほど利用しないため、台数を増やす水平スケーリングが有用でしょう。

しかし、これは二者択一というわけではありません。画像をリサイズするAPIでも、リクエスト数が増えれば、水平スケーリングを余儀なくされる場合もあります。また、クライアントの同時接続数に応じ、CPU使用率やメモリ

使用量は高まる傾向にあります。そのため、一般的なCRUDでも垂直スケーリングする場合もあります。ほかにも、サーバの台数が増えれば、それだけリリースにかかる時間が増えるということでもあります。したがって、サーバの運用面まで考えると、垂直スケーリングして、サーバ台数を少なく保っておくほうがリリース時間を短く保つことができるというメリットもあります。

セキュリティテスト

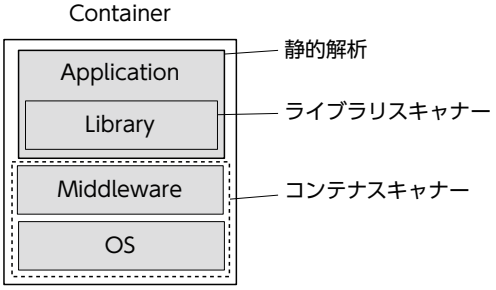


本項ではDockerコンテナを用いたREST APIを題材にセキュリティテストについて説明していきます。Dockerイメージを自作する場合、ベースとなるDockerイメージを選定し、そのうえで、ミドルウェアのインストールを行い、自作のアプリケーションを動かすことになります。このように、サービスを提供するうえでシステムが多層的であるため、それぞれに対して脆弱性（ぜいじゃくせい）のスキャンを行う必要があります。ここでは、それぞれの階層に対して、「コンテナスキャナー」「ライブラリスキャナー」「静的解析」という形式で紹介します（**図1**）。

コンテナスキャナー

Dockerコンテナの内部構造に注目すると、Dockerコンテナの最下層としてOS（ベースイメージ）があり、その上にMiddlewareがあります。Middlewareに関しては適切な説明が難しいですが、「自分で書いたアプリケーション以外に必要なソフトウェア」ととらえるとよいでしょう。REST APIであれば、具体的にはApacheやNginxなどのWebサーバがこれにあたります。これらの脆弱性をチェックするのが「コンテナスキャナー」です。これは執筆の形式上付けた名前であり、一般的な名称ではありません。

▼図1　REST APIのDockerイメージの階層構造と脆弱性スキャナーの検証対象



コンテナスキャナーでは、先ほどの説明どおり、OSやMiddlewareに関する脆弱性の検知を行います。個人的に一番よく使うものはAmazon Elastic Container Registry（以下、Amazon ECR）の脆弱性スキャン機能です。Amazon ECRはAmazonのコンテナレジストリで、主にDockerイメージを管理するために利用するサービスです。このサービスでは、コンテナイメージをプッシュするたびに脆弱性スキャンを実行できます。これは内部的にはClair[注4]というコンテナの脆弱性スキャン用のソフトウェアを利用しています。

ほかに話題になった脆弱性スキャナーとして、福田鉄平さんという日本人が開発したTrivy[注5]があります。これはAqua Security Software Ltd.に売却され、Harborというコンテナレジストリでのデフォルトの脆弱性スキャンとして採用されています。

ライブラリスキャナー

次に出てくるのがアプリケーションの層です。昨今のソフトウェア開発では外部ライブラリを使うことが多いでしょう。そういったアプリケーションで利用する外部ライブラリの脆弱性をスキャンするのが「ライブラリスキャナー」です。こちらも適切な呼称がなかったため、コンテナスキャナーと同様に形式的に名付けました。

たとえば、Node.jsでアプリケーションを作った場合、そのソースコードやライブラリはビル

注4）https://quay.github.io/clair/
注5）https://trivy.dev/

ドを行わず、そのままコンテナにコピーされるでしょう。前述のコンテナスキャナーは、コンテナ上にコピーされたファイルについてスキャンを行います。そのため、Node.jsのようにソースコードを直接置くタイプのプログラミング言語ならプログラム内で利用しているライブラリの脆弱性スキャンができる可能性があります。

しかし、Go言語のようにソースコードをビルドし、実行ファイルをコンテナ上にコピーするものもあります。このような場合、コンテナスキャナーでは脆弱性を検知できないでしょう。そのため、ソースコードの段階で、どのようなライブラリを利用し、それらに対し、脆弱性があるかを判定するライブラリスキャナーが必要となります。

ライブラリスキャナーはいくつかありますが、筆者が一番利用しているのは「dependabot」です。GitHubで利用できる脆弱性スキャン機能なので知っている方も多いでしょう。リポジトリ内のrequirements.txtやpackage.jsonなどのパッケージマネージャーの情報をもとに脆弱性スキャンを行います。一度設定してしまえば、自動で定期的に実行され、PRを作成してくれるので、回帰テストが十分にある環境では非常に低コストで済むソリューションです。

ほかに例を挙げると、Node.jsのパッケージマネージャーであるnpmのnpm auditコマンドもライブラリスキャンであると言えるでしょう。また、先ほど紹介したTrivyもライブラリスキャンの機能があり、composerやpip、mavenなどパッケージマネージャーの情報をもとに脆弱性スキャンが可能です。

静的解析

コンテナ部分のセキュリティスキャンの最後は「静的解析」です。先ほどまでとは少し毛色の異なる内容です。「コンテナスキャナー」「ライブラリスキャナー」は「他人のコードに対する脆弱性検知」でしたが、この「静的解析」は「自分のコードに対する脆弱性検知」です。

どういうことかというと、世の中で公開されているライブラリは脆弱性情報がデータベース化され、自分が利用しているライブラリと脆弱性のあるライブラリの情報が合致すれば脆弱性として検知されます。したがって、原理上、これらのアプリケーションは「自分が今書いたコード」に対して脆弱性があるかは検知できません。ここで紹介する「静的解析」はそうした「自分が書いたコードに脆弱性がないか」を検証したい場合に役に立ちます。

筆者がよく使う静的解析ツールはSonarCloud[注6]です。次のURLにアクセスすると、静的解析例として、Apacheのソースコードに対してSonarCloudによるスキャンをした結果が確認できます。

https://sonarcloud.io/project/issues?impactSeverities=HIGH&impactSoftwareQualities=SECURITY&issueStatuses=OPEN%2CCONFIRMED&id=kotauchisunsun_httpd&open=AZL80LxT0bcYsK7qBd0z

gets関数はC言語の中でバッファオーバーランという脆弱性を起こすことで有名な関数です。SonarCloudでは脆弱性のある関数を検知すると、**図2**のような警告を表示します。

そのほかのソリューションとしてCodacy[注7]というSaaSサービスも存在します。先ほどと同様に、Apacheのソースコードに対してCodacyでスキャンした結果を、次のURLから確認できます。

https://app.codacy.com/gh/kotauchisunsun/httpd/issues/current?levels=Error&categories=Security

図3では、Popenに関する問題が提示されています。これは、Popenがコマンドを呼び出せる関数であるため、Codacyは第1引数である

注6）https://www.sonarsource.com/
注7）https://app.codacy.com

argsに適切なエスケープ処理を求めています。

このように、SonarCloudやCodacyでセキュリティに問題のある、もしくは問題のおそれのあるコードが検出できることがわかります。今回、SonarCloudの場合はtest/test_find.cというソースコードに、Codacyの場合はtest/modules/md/dns01.pyというソースコードに対して脆弱性の指摘をしていました。どちらもtest/配下にあるテストコードであると推察できるので、Apache本体のWebサーバには問題がないと思われます。

◆ ◆ ◆

このように、それぞれの脆弱性スキャナーでも検出している対象や得手不得手があるため、それらの階層や検証する対象を考慮してテストを行う必要があります（**表2**）。

ソフトウェアの品質とは



ここまで「機能テスト」「パフォーマンステスト」「セキュリティテスト」とテストに関して説明してきました。これらのテストはソフトウェアの品質を担保するために行います。では、この「ソフトウェアの品質」とは何なのでしょうか。

JIS X 25010[注8]規格では、次のように「品質モデル」というものが定義されています。

> システムの品質は，システムが様々な利害関係者の明示的ニーズ及び暗黙のニーズを満足している度合い

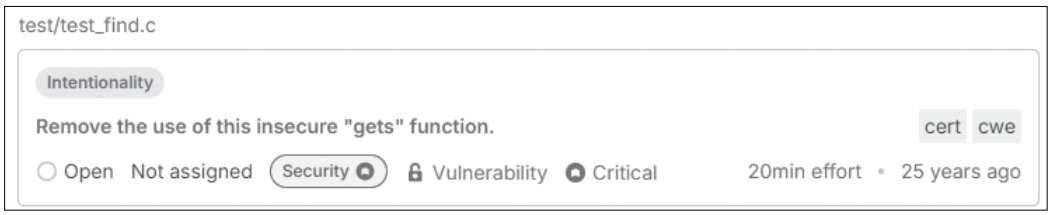
また、この品質モデル自体も「利用時の品質モデル」「製品品質モデル」「データ品質モデル」と3つに分類されています。本稿で主に扱っているのは「製品品質モデル」で、

> ソフトウェアの静的特徴及びコンピュータシステムの動的特徴に関係する，八つの特性（これらは，更に副特性に分けられる。）からなる。このモデルは，コンピュータシステムとソフトウェア製品との両者に適用できる。

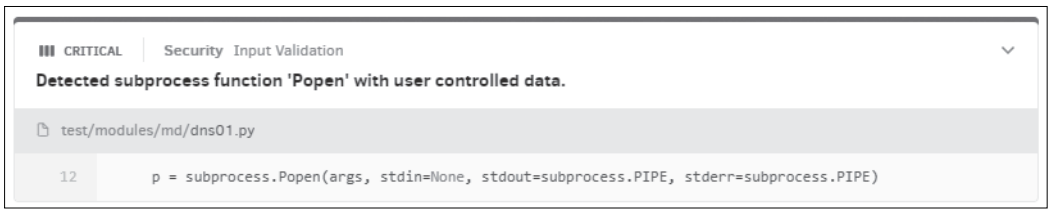
とされています。**表3**は、この特性（品質特性）

注8）https://kikakurui.com/x2/X25010-2013-01.html

▼図2　SonarCloudによる脆弱性検知の例



▼図3　Codacyによる脆弱性検知の例



▼表2　セキュリティテストに関する整理

テストの種類	テストの対象	ツール
コンテナスキャナー	依存ライブラリ（他人のコード）	Clair、Trivy
ライブラリスキャナー	依存ライブラリ（他人のコード）	Dependabot、npm audit、Trivy
静的解析	自社コード（自分のコード）	SonarCloud、Codacy



▼表3　製品品質モデルとテスト種別の対応

品質特性	定義（筆者による修正あり）	テスト種別
機能適合性	明示的・暗黙的ニーズの充足度合い	機能テスト
性能効率性	使用する資源の量に関する性能の度合い	パフォーマンステスト
互換性	システムまたは構成要素の情報を交換することができる度合い	
使用性	目標を達成するために、利用者がシステムを利用することができる度合い	
信頼性	指定の条件下で製品または構成要素が機能を実行する度合い	
セキュリティ	人間またはシステムが、定められた権限下でデータアクセスし、データを保護する度合い	セキュリティテスト
保守性	システムが修正することができる有効性および効率性の度合い	
移植性	現在の利用環境からほかの環境にシステムを移すことができる有効性および効率性の度合い	

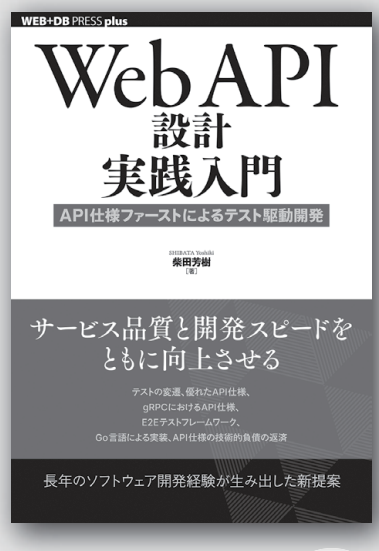
※https://kikakurui.com/x2/X25010-2013-01.html より筆者作成

とテスト種別の対応を行ったものです。

実は、テストですべての品質特性を担保できるというわけではありません。したがって、品質を高めるためには、別のアプローチも必要です。たとえば、保守性であれば、先述したSonarCloudによる静的解析を行い、セキュリティのチェックだけでなくソースコードレベルの技術負債の予防も行うとよいでしょう。

さらに、これらの品質特性は、プロダクトごとに重要度も異なります。自分が作っているプロダクトに対し、どこが重要なのかを考えたうえで、それを担保する手法はソフトウェアテストなのか、それとも別のアプローチなのかを考えることが大切です。したがって、ソフトウェアの品質特性についても一度学んでみることをおすすめします。SD

技術評論社



Web API
設計
実践入門
API仕様ファーストによるテスト駆動開発

本書では、ソフトウェアテストの変遷とWebサービスにおけるAPI仕様の関連を説明したうえで、API仕様とはどうあるべきか、API仕様に何を書くべきかについて説明します。具体例としてはgRPCを取り上げます。第4章で紹介するAPI仕様ファースト開発という開発プロセスは、筆者が日々実践していることですが、多くのソフトウェアエンジニアが実践できていないことです。そのために必要なE2Eテストフレームワーク、さらには、API仕様がきちんと書かれていないために生まれる技術負債の返済方法なども紹介します。

柴田芳樹 著
A5判／208ページ
定価（2,600円+税）
ISBN 978-4-297-14293-3

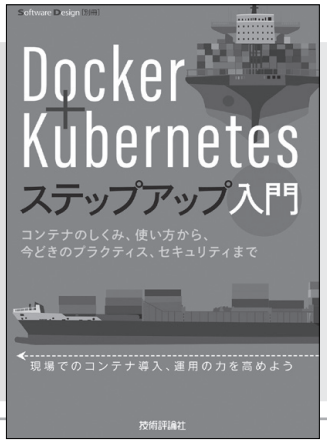
大好評発売中！

こんな方におすすめ
・Webサービス開発者
・API設計を学びたい方
・テスト駆動開発に取り組んでいる方
・より良い開発手法を知りたい方

Masato Matsuwaki : fxy.x@lambda-publishing.com

Software Design［別冊］

「Software Design」のバックナンバーの中から好評だった記事を収録した書籍シリーズです。



Docker＋Kubernetesステップアップ入門
～コンテナのしくみ、使い方から、今どきのプラクティス、セキュリティまで

徳永航平，宮原徹，濱田孝治，清水勲，田中智明，早川大貴，須田一輝，李瀚，前佛雅人，水野源，遠山洋平，森田浩平　著
B5判／192ページ
定価2,420円（本体2,200円＋税10%）
ISBN 978-4-297-14746-4

［入門］ドメイン駆動設計
――基礎と実践・クリーンアーキテクチャ

増田亨，田中ひさてる，奥澤俊樹，中村充志，成瀬允宣，大西政徳　著
B5判／160ページ
定価2,200円（本体2,000円＋税10%）
ISBN 978-4-297-14317-6





データベース速攻入門
～モデリングからSQLの書き方まで

堀内康夫，徳尾秀敬，とみたまさひろ，株式会社リブセンス，村田太，成田優隆，福本誠，大津桜子，藤本正比古，廣山豊，吉村守，杉江伸祐，杉山ジョージ，中村昌登　著
B5判／192ページ
定価2,420円（本体2,200円＋税10%）
ISBN 978-4-297-13362-7

技術評論社
〒162-0846　東京都新宿区市谷左内町21-13
販売促進部　Tel. 03-3513-6150　Fax. 03-3513-6151

Masaki Matsuwaki : fuzk@lambda-publishing.com



あなたのサービスは大丈夫？

ゼロから学ぶ Web APIセキュリティ

第5章 設計から始める攻撃対策

今やWeb APIはWebサービス開発と切っても切れない関係です。それゆえにサイバー攻撃の対象にも容易になり得ます。攻撃事例も数多く報告されており、セキュリティ対策は急務と言えるでしょう。ですが、いざ対策しようとすると、想像以上に配慮する必要のある箇所が多く、途方に暮れた人もいるのではないでしょうか。そんなあなたのために、本章ではWeb APIセキュリティのノウハウを集めました。5-1、5-2ではWeb APIに潜むセキュリティリスクと重要なセキュリティのポイントを紹介し、5-3、5-4では「脆弱性」と「認証・認可」の2点で対策の心得を伝授します。この機会にぜひ、ご自身のWeb APIについて、一度見直してみてください。

5-1 APIの開発・保守をするなら押さえておきたい
Web APIに潜むセキュリティリスク
P.158 Author 石川 朝久

5-2 DevOpsとシフトレフトを起点に考える
Web APIセキュリティの重要ポイント
P.168 Author 徳丸 浩

5-3 正しく検知して攻撃経路をふさぐ
脆弱性
P.179 Author 松本 隆則

5-4 サービスを不正アクセスから守る
認証・認可
P.184 Author 川村 修平

5-1 APIの開発・保守をするなら押さえておきたい

Web APIに潜むセキュリティリスク



Author 石川 朝久(いしかわ ともひさ)
X(Twitter) @scientia_sec

Web APIの利用は増加の一途をたどる一方、その普及の裏にはセキュリティリスクも潜んでいます。本稿では、Web APIの脅威をまとめた「OWASP API Security Top 10」をもとに、開発するうえで押さえておきたい代表的な脆弱性とその対策について紹介します。

Web APIの普及と攻撃の背景

現在、Web APIの利用は急速に増加しています。Web APIのテストツールであるPostmanが発表しているレポートによれば、年間作成されたリクエストは2021年には8.5億件であり[注1]、2023年には12.9億件まで増加しています[注2][注3]。その背景として、Web APIは異なるプラットフォームやアプリケーション間でデータを共有し、連携するための優れたしくみを提供していることが挙げられます。

しかし、この普及の裏側にはセキュリティの問題が潜んでおり、攻撃者はWeb APIを標的にさまざまな手法で攻撃をしかけています。2017年にGartner社が発表したレポートでは、2022年までにAPIの悪用が最も頻度の高い攻撃手法になると予測されていました[注4]。実際、2022年のGoogle社のレポートでは約半数の組織が調査時点から「過去12か月間にセキュリティインシデントを経験した」と回答しており[注5]、Gartner社の予測は的中していると言えるでしょう。そのおもな背景には2つの要因が挙げられます。

Web APIが取り扱うデータの価値

2017年、「世界で最も価値のある資源はもはや石油ではなくデータである」という記事が『The Economist』の紙面に登場しました[注6]。Web APIはアプリケーションやサービス間での重要なデータのやり取りを可能にします。たとえば、ユーザーの個人情報や認証情報、健康情報、取引履歴などが該当します。こうしたデータは攻撃者にとっても魅力的であるため、これらのデータを狙って経済的な利益を得ようとします。

APIエコノミーの形成

Web APIの普及に伴い、さまざまなWebサービスがAPIを介してサービス連携し、より付加価値の高いサービスを提供するAPIエコノミーも普及しました。たとえばライドシェアアプリを開発する場合、従来は地図機能や決済機能などを個別に実装する必要がありましたが、現在はGoogle Maps APIを用いて地図機能を、決済代行サービスAPIを使用して支払い処理を、Twilio APIを活用して顧客と運転手のコミュ

注1) URL https://www.postman.com/state-of-api/2021/api-global-growth/

注2) URL https://www.postman.com/state-of-api/2023/api-global-growth/

注3) 2024年度版では、具体的なリクエスト数は公開されていませんが、SalesforceやWhatsApp、ChatGPTなど各種APIが利用され、総数が増えていることを示しています。より詳しくは、次のURLを参照してください。
URL https://www.postman.com/state-of-api/2024/api-global-growth/

注4) URL https://www.gartner.com/document/code/342236

注5) URL https://cloud.google.com/resources/api-security-research-report?hl=ja

注6) URL https://www.economist.com/leaders/2017/05/06/the-worlds-most-valuable-resource-is-no-longer-oil-but-data

Masato Matsuyuki : my.x@lambda-publishing.com

ニケーションに関する機能をすばやく導入できます。このため、開発者はライドシェアサービスのコア技術である需要予測やインセンティブの提供などに焦点を当てることができます。

また、近年はChatGPTなどで知られるOpen AIをはじめ多数のAI開発企業がWeb APIとして生成AI機能を提供しているため、今後も魅力ある新たなサービスの登場が予想されます。

異なるシステムやサービスが容易に連携できるこの事実は、セキュリティ上の攻撃ポイントが多数存在することを意味します。システム間の連携ポイントが増えるほど、攻撃者にとっては攻撃ポイントが増えることになります。したがって、APIエコノミーの拡大に伴い、セキュリティ対策がますます重要となるのです。

十分なセキュリティ対策が講じられていないAPI

APIが普及する一方で、Web APIのセキュリティが十分に確保されていないことも指摘されています。開発速度や利便性を重視し、セキュリティが後回しにされるケースがあるためです。攻撃者にとって、こうした脆弱(ぜいじゃく)なAPIは魅力的な標的となります。ここでは、過去に発生した具体的な2つの事例を紹介します。

Twitter APIの脆弱性事例

1つめの事例は、Twitter（現X）が提供するAPIの脆弱性です[注7]。Twitterには、「プライバシーとセキュリティ」という設定の中に「見つけやすさと連絡先」という項目が存在します。この項目を有効にすると、メールアドレスや携帯電話番号などの連絡先情報を使用してほかのユーザーが自身のアカウントを見つけられるように設定できます。当該設定を無効にしているユーザーは、連絡先情報からTwitterアカウントを見つけられることはありません。しかし、バグバウンティプログラム[注8]を通じてAPIの実装に不備があり、プライバシー設定によらず、当該機能が利用できることが指摘されました。これは、後に紹介するとおり、認可が適切に行われていなかったことが原因と考えられます。

米国郵便公社のユーザーデータ漏洩事例

2つめの事例は、米国郵便公社のInformed Visibilityというwebサイトが6,000万人のユーザーデータを漏洩(ろうえい)したケース[注9]です。Informed Visibilityでは、利用者が郵便物に関するデータを取得できるようにAPIが提供されていました。しかし、このAPIの設計にはワイルドカードのクエリを受け付けてしまう、住所データをリクエストした際には関連する全情報が返されるなど、過度なデータ露出が発生していました。

Web APIがもたらすリスク

Web APIが引き起こすリスクには、どのようなものがあるでしょうか。ここでは、情報を安全に利用するために必要なセキュリティの3要素の観点から紹介します。

機密性（Confidentiality）：情報漏洩のリスク

Web APIによる脆弱性で最も注目されるのが情報漏洩のリスクです。Web APIにおいて機密性が十分に管理されていない場合、情報漏洩につながる可能性があります。とくにWeb APIはスクリプトなどを用いた処理が容易であり、脆弱性が存在する場合には大量のデータが不正に取得され得ます。先述のTwitterや米国郵便公社の事例もこれに該当します。

完全性（Integrity）：データ改ざんのリスク

のちほど説明する「マスアサインメント」脆

注7）URL https://privacy.x.com/en/blog/2022/an-issue-affecting-some-anonymous-accounts
URL https://gigazine.net/news/20220808-twitter-incident-expose-data-5-4-million-accounts/

注8）自社が開発したソフトウェアやアプリケーションにある脆弱性を発見・報告してもらうことで、報告者に報奨金などを支払う制度です。

注9）URL https://krebsonsecurity.com/2018/11/usps-site-exposed-data-on-60-million-users/

弱性などが存在する場合、Web APIが管理するデータが意図せず改ざんされる可能性があります。APIエコノミーを前提に考えると、ある部分でデータ改ざんが成立すれば、ほかのAPIにも影響を及ぼす可能性があります。

可用性(Availability):リソース消費のリスク

Web APIへの攻撃の一環として、過度なリソース消費を引き起こす攻撃が挙げられます。負荷が多い処理を実行してCPUやメモリのリソースを消費させたり、特定の処理を大量に実行したりすることで、サードパーティAPIを大量に呼び出し、サービス運用コストが上昇する可能性があります。これは、可用性に影響を与えるリスクです。

Web APIの攻撃手法

Web APIへの攻撃手法はさまざま存在します。伝統的なWebアプリでよく見られるクロスサイトスクリプティング攻撃やSQLインジェクション攻撃、DoS攻撃なども依然として存在する中、API固有の攻撃手法も増加しています。Web APIに関連する攻撃手法や脆弱性を整理したものとして、「OWASP API Security Top 10」が挙げられます[注10]。これは、OWASP(Open Worldwide Application Security Project)がまとめた最先端の10大API攻撃手法で、2019年に初版が、2023年6月に第2版が公開されました。

図1に、2019年度と2023年度のランキングとその変更点を示しています。今回の変更で新たに脆弱性が追加された一方で、2019年に存在していた「過剰なデータ露出」と「マスアサインメント」は「オブジェクトプロパティレベルの認可不備」として統合されました。また「インジェクション」と「不十分なロギングと監視」はランク外となりました。筆者の見解としては、「OWASP API Security Top 10」はAPIに特化した内容に進化していると考えていますが、ランク外となったこれらの脆弱性も引き続き重要であると言えます。たとえば2022年にはVaronis社の研究者によって、Zendesk社が提供しているGraphQL APIでのSQLインジェクションが報告されています[注11]。

認可に関する脆弱性

「OWASP API Security Top 10 2023」で

注10) URL https://owasp.org/API-Security/
注11) URL https://www.varonis.com/blog/zendesk-sql-injection-and-access-flaws

▼図1 OWASP API Security Top 10 2023の一覧と2019年からの変化の推移

OWASP API Security Top 10 – 2019			OWASP API Security Top 10 – 2023		
API1	オブジェクトレベルの認可不備	→	API1	オブジェクトレベルの認可不備	
API2	ユーザー認証の不備	→	API2	認証の不備	
API3	過剰なデータ露出	統合 →	API3	オブジェクトプロパティレベルの認可不備	NEW
API4	リソース不足とレート制限	→	API4	制限のないリソース消費	
API5	機能レベルの認可不備	→	API5	機能レベルの認可不備	
API6	マスアサインメント	統合 →	API6	機密性の高いビジネスフローへの無制限のアクセス	NEW
API7	セキュリティ設定のミス	→	API7	サーバサイドリクエストフォージェリ	NEW
API8	インジェクション ランク外	→	API8	セキュリティの設定ミス	
API9	不適切なアセット管理	→	API9	不適切なインベントリ管理	
API10	不十分なロギングと監視 ランク外	→	API10	APIの安全ではない利用	NEW

Masato Matsuyuki : xxx@lambda-publishing.com

▼表1　会員情報管理API（https://www.example.com/api/v2/users）の仕様例

メソッド	指定するパラメータ	概要	一般ユーザー権限	管理者権限
GET	id	パラメータで指定したユーザーIDにひも付くユーザー情報を表示	自分のデータのみ閲覧することが可能	全ユーザーの情報を閲覧することが可能
DELETE	id	パラメータで指定したユーザーIDにひも付くユーザー情報の削除	利用不可	全ユーザーの情報を削除することが可能
PUT	（省略）	（省略）	（省略）	（省略）

は、認可に関する脆弱性が3つ（API1、API3、API5）挙げられています。認可の脆弱性は、設計時におけるアクセス制御の考慮不足がおもな原因です。オブジェクト、オブジェクトプロパティ、機能へのアクセス時に、アクセス権限が検証されていないことが問題となっています。本来であれば、「アクセス制御に関する脆弱性」としてまとめてもよいのですが、それぞれを3つに分類し、丁寧に説明している点は、この種の問題が広く発生する可能性を物語っていると言えるでしょう。



API1：オブジェクトレベルの認可不備



オブジェクトレベルの認可不備は、APIプロバイダがAPI利用者に対して、アクセス権限のないオブジェクト（リソース）へのアクセスを許可してしまった場合に発生する脆弱性です。通常、会員情報や商品情報などの各種オブジェクトにアクセスする際には、識別IDが利用されます。**表1**の会員情報管理APIを例に考えてみましょう。

たとえば、一般ユーザー権限を持つ「技術太郎」というユーザーで、API（/api/v2/users?id=1234）に対してGETリクエストを送信すると、**リスト1**のような会員情報を含むレスポンスが返ってきたとします。APIの仕様に従い、当該リクエストを送信したユーザーは自身（この場合、「技術太郎」）のユーザー情報にアクセスする権限があるので、この結果は問題ありません。

しかし、「技術太郎」のアカウントで他人のユーザーIDを指定してGETリクエスト（/api/v2/users?id=1235）を送信し、**リスト2**の情報にアクセスできてしまったとします。一般ユーザー権限は他人の情報にはアクセスできないはずですが、アクセス権限がない情報にアクセスできていることになるので、「オブジェクトレベルの認可不備」の脆弱性が存在しています。この脆弱性が発生する理由は、オブジェクトへアクセスする際に、アクセスする権限があるか検証するアクセス制御が実装されていないことに起因します。



API5：機能レベルの認可不備



オブジェクトレベルの認可不備に対して機能レベルの認可不備は、ある権限・グループに属するユーザーが、別の権限・グループのユーザーにしか許可されていないAPI機能へアクセスできてしまう機能です。たとえば**表1**のAPIにおいて、一般ユーザー権限を持つ「技術太郎」のアカウントで「技術太郎」自身のユーザー情報を削除するDELETEリクエスト（/api/v2/

▼リスト1　一般ユーザ権限「技術太郎」による会員情報管理APIへのGETリクエスト

```
{
    "id": "1234"
    "first_name": "技術"
    "last_name": "太郎",
    "birthday": "1990-01-01",
    "username": "tech-taro"
    "privilege": "user"
}
```

▼リスト2　「オブジェクトレベルの認可不備」の成功例

```
{
    "id": "1235"
    "first_name": "評論"
    "last_name": "花子",
    "birthday": "1990-10-10",
    "username": "hanako"
    "privilege": "user"
}
```

users?id=1234）を送信して、リクエストが受け付けられたとします。本来、「ユーザー情報の削除」という機能は一般ユーザー権限には許可されていない機能です。しかし実際に削除が成功した場合は、機能レベルのアクセス制御ができていない、つまり「機能レベルの認可不備」の脆弱性があると言えます。



API3：オブジェクトプロパティレベルの認可不備

オブジェクトプロパティレベルの認可不備は「OWASP API Security Top 10」で新しく登場した項目です。オブジェクトにアクセスした際に、当該オブジェクトに含まれるプロパティが過剰にデータを露出したり、当該プロパティの値を不正に改ざんしたりすることができるマスアサインメント脆弱性が該当します。

過剰なデータ露出とは、APIが必要以上の情報をレスポンスに含んで応答することを意味します。たとえば、一般ユーザー権限でユーザー情報を取得できるAPIを利用すると、自分の営業担当に関する情報を取得できる状態にあったとします。このAPIの用途が、自分のユーザープロフィールを表示するだけのものであれば、営業担当の詳細情報は必要ありません。APIが持つべき仕様には依存しますが、**リスト3**のように必要以上の情報がレスポンスに含まれている場合には、過剰なデータ露出に該当します。

▼リスト3　「過剰なデータ露出」と推察される例

```
{
    "id": "1234"
    "first_name": "技術"
    "last_name": "太郎",
    "birthday": "1990-01-01",
    "username": "tech-taro"
    "privilege": "user"
    "sales_representatives": [
        "name": "営業 次郎",
        "id": "3456",
        "email": "xxx@xxxx.com",
        "privilege": "super-admin",
        "admin": true,
        "two_factor_auth": false
    ]
}
```

一方、マスアサインメント脆弱性とは、マスアサインメント機能を悪用した脆弱性です。マスアサインメント機能とは、アプリケーションがユーザー入力を複数の変数・オブジェクトに自動的に割り当てる、開発を簡素化する機能です。この機能を悪用すると、ユーザー入力を追加して変数・オブジェクトの値が操作可能になり、パスワード、許可された権限、管理者アクセスなど、通常はユーザー機能を制限すべきデータ項目を変更できてしまう可能性があります。

具体的な事例を見ていきましょう。たとえば、ユーザー権限にてユーザー情報（ユーザー名、パスワード）を更新できる機能があったとします（**リスト4**）。

攻撃者が当該API群のドキュメントを読み、アカウント作成処理において利用者が管理者権限になるために利用できる追加キー"isAdmin"があることを知ったとします。その際、同じユーザー権限で、**リスト5**のようなリクエストを送信します。管理者権限を取得できた場合、これはマスアサインメント脆弱性と呼ばれます。

当該Webサービスでは、アプリケーション

▼リスト4　ユーザー情報を更新するリクエスト

```
{
    "User": "userA",
    "Password": "Password123"
}
```

▼リスト5　「マスアサインメント脆弱性」の悪用リクエスト

```
{
    "User": "userA",
    "Password": "Password123"
    "isAdmin": true
}
```

▼リスト6　アプリケーションが用意したユーザーオブジェクト例

```
{
    "User": "userA",
    "Password": "Password123"
    "location": "San Francisco, CA",
    "group_membership": [10, 11, 12],
    "isAdmin": true
}
```

が操作するユーザーオブジェクトとして**リスト6**のようなオブジェクトが用意されているとします。入力値検証などを適切に実施していない場合、マスアサインメント機能により指定された"isAdmin"のパラメータが自動的に割り当てられ、データが改ざんされてしまいます。

API固有の脆弱性

2023年版では、APIゆえに発生する脆弱性も多数取り上げられています。

API4：制限のないリソース消費

制限のないリソース消費とは、Web APIによって使用されるAPIを適切に制限していない場合、過度にリソースを消費してサービスプロバイダからの請求が増額されたり、サービスの可用性に影響を与えたりする脆弱性です。攻撃対象となるリソースは、メモリとCPU、サードパーティAPI、データなど多岐にわたります。

たとえば、あるサービスで、パスワードを再設定するために事前登録した携帯電話番号に対し、SMS経由でワンタイムトークンを送信する「パスワード再設定」APIを実装したとします。バックエンドでは、ユーザーが当該APIを呼び出すと、1回数円でショートメッセージを送信するサードパーティAPIサービスが実行され、SMSが送付されるしくみです。攻撃者がパスワード再設定APIを何万回も呼び出すスクリプトを実施した場合、呼び出された回数だけサードパーティAPIも呼び出されるので、数分のうちに多くの損失を被ることになります（**図2**）。

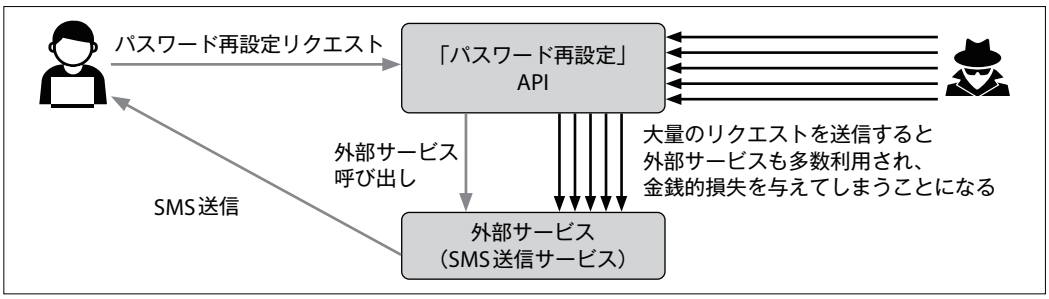
対策としては、メモリやCPUの受け入れ可能な上限値を設定する、APIのレート制限（利用回数制限）を実装する、レスポンスとして含まれるレコード数の制限など、適切な制限を導入することが挙げられます。

API6：機密性の高いビジネスフローへの無制限のアクセス

機密性の高いビジネスフローへの無制限のアクセスは、APIに実装されたビジネスプロセスに対して自動化された無制限のアクセスを行うことで、ビジネスプロセスに影響を与える脆弱性を意味します。たとえば人気が高いゲーム機を販売するWeb APIに対して、攻撃者が商品購入を自動化するスクリプトを作成しゲーム機の購入を独占してしまうことは、この脆弱性にあたります。この攻撃により、正規のユーザーが商品を購入できる機会を阻害するとともに、攻撃者により転売を許してしまう可能性があります。こうした攻撃への対処として、OWASPは2種類の対策を推奨しています。

1つは、ビジネスレベルでの対策として、過度に使用するとビジネスに悪影響を及ぼす可能性のあるビジネスフローを特定することです。APIに実装されるビジネスフローとは、「現実のビジネスフロー（ビジネスロジック、ビジネスルール、ワークフロー）をモデル化し、アプリケーション上で実現したもの」だと定義されます。言い換えれば、こうした悪用は、モデル化・実装する際に考慮漏れなど不備があり、その不備を悪用する攻撃だと言えるでしょう。

▼図2 「制限のないリソース消費」の攻撃メカニズム例



もう1つは、技術レベルでの対策です。たとえば、Web APIによる操作が人間による操作か否か判定する手法を導入することが挙げられます。その実装例として、CAPTCHAの導入や、フローパターン（たとえば、「商品をカートに追加する」アクションから「購入完了」までのアクションを1秒以内で完了していないか）などを分析して自動化攻撃に対処する方法が挙げられます。

API7：サーバサイドリクエストフォージェリ

サーバサイドリクエストフォージェリ（SSRF）は、ユーザーが指定したURLを、APIが検証せずにリモートリソースを取得する際に発生します。この手法により攻撃者はアプリケーションに細工したリクエストを予期しない宛先に送信させることができます。この動作を利用すると、ファイアウォールやVPNで保護されている本来ではアクセスできない対象に対して、攻撃を行える可能性があります。

具体的な事例を見てみましょう。あるSNSにおいて、ユーザーが写真をアップロードできるAPIがあったとします。その際、ユーザーは自分の端末から画像ファイルをアップロードするか、画像のURLを指定するかを選択できます。後者を選択すると、**リスト7**のAPIが呼び出され、指定されたURLの画像へアクセスし、データを取得しようとします。

攻撃者は"pic_url"に悪意のあるURLを指定することで、さまざまな攻撃が可能です。たとえば、**リスト8**のように指定すれば、当該サーバから自分自身（localhost）の8080ポートにアクセスします（**図3**）。当該ポートが開放されているか否かでレスポンスタイムが異なるため、調査対象となるポートを変更していけばポートスキャンを実施できます。

とくにクラウドプラットフォーム環境下においては、このSSRF脆弱性を悪用して、クラウドプラットフォームの認証情報などを窃取する攻撃手法が知られています。2019年に発生したCapital One社の事案では、当該脆弱性を悪用し、インスタンスメタデータ（IMDSv1）を取得してIAMロールのクレデンシャル情報を入手し、最終的に個人情報漏洩へとつながりました[注12]。

▼リスト7　写真アップロードAPIのリクエスト例

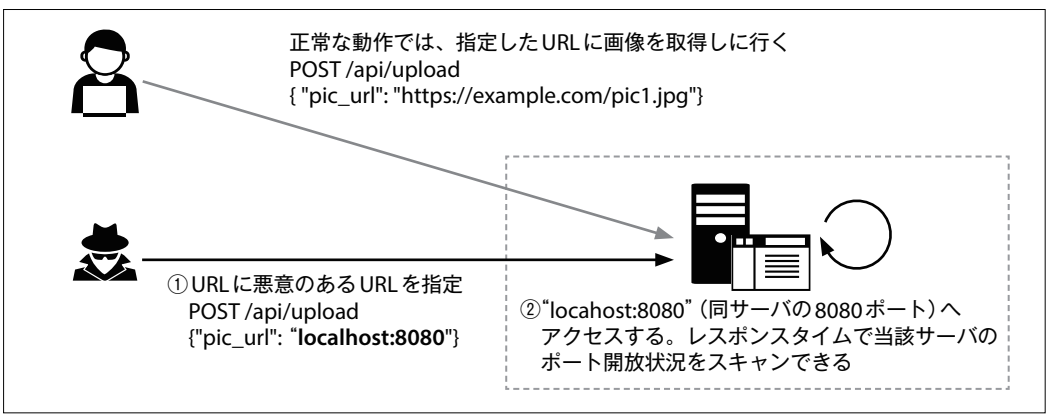
```
POST /api/upload
{
    "pic_url": "https://example.com/pic1.jpg"
}
```

▼リスト8　SSRFの攻撃リクエスト例

```
{
    "pic_url": "localhost:8080"
}
```

注12）URL https://logmi.jp/brandtopics/329553

▼図3　「SSRF」の攻撃メカニズム例



API9：不適切なインベントリ管理

不適切なインベントリ管理は、すでにサービスが終了しているか、ベータ版APIを公開する際に発生します。古いバージョンのAPIは通常、改修が行われておらず脆弱性を抱えている可能性が高まります。同時に、ベータ版APIも脆弱性を抱えている可能性があります。

たとえば、https://www.example.com/v2/が最新バージョンのWeb APIであるとします。https://www.example.com/v1/のサポートが終了しているにもかかわらず、互換性の観点から引き続き公開されている場合、/v1/が攻撃対象になる可能性があります。もちろん、不適切なインベントリ管理だけで直ちに脆弱性が発生するわけではありません。しかし、このようなサポート終了済みのAPIを調査することは非常に有益です。たとえば、過去のドキュメント、GitHubの変更履歴や仕様変更の差分履歴、Issueなどを確認することで、/v1/に残る潜在的な脆弱性を発見できる可能性があります。場合によっては、ドキュメントからは削除されているが、内部的にはまだ存在する可能性があるパラメータなどを発見できるかもしれません。

API10：APIの安全ではない利用

「ユーザーからの入力は信用せず検証する」というセキュリティの原則は広く知られている一方で、サードパーティ（とくに、有名な企業が提供する）APIから受信したデータを信頼し過ぎる傾向があります。攻撃者は、開発者のこの油断を悪用し、サードパーティを経由してさまざまな攻撃を行うことができます。とくに、連携を前提とするAPIにおいて、非常に影響の大きい問題です。

具体的な攻撃アプローチを**図4**に示します。対策として、サードパーティを含むすべてのWeb APIから受け取ったデータについては、利用する前に検証を行うことが重要です。

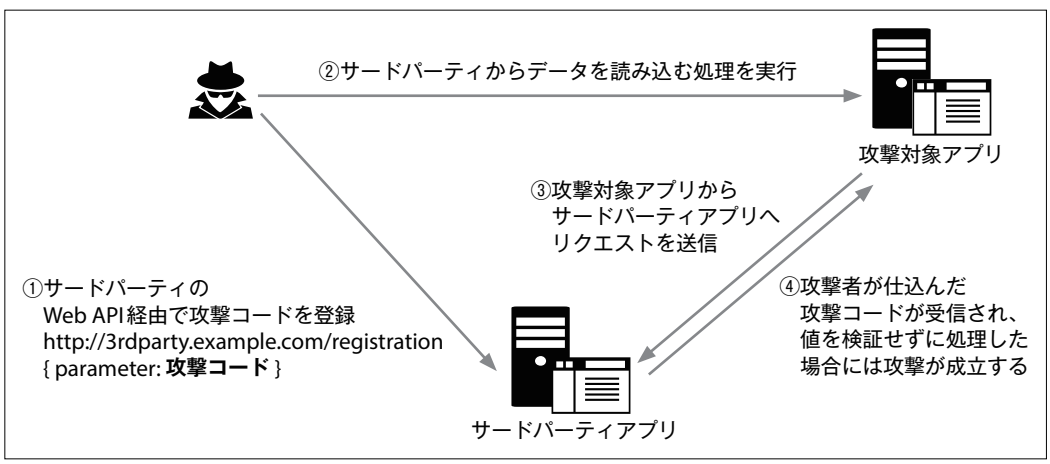
セキュリティバイデザインによるWeb API開発

ここまで、APIで発生する脆弱性について紹介してきました。ここからは安全なWeb APIを開発する手法について解説していきます。

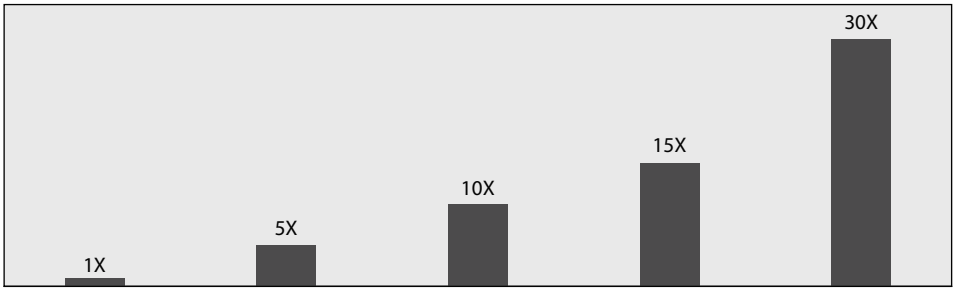
セキュリティバイデザインの定義と重要性

Web APIのセキュリティレベルを守るためには、各工程に応じたセキュリティ対策が重要であり、これはセキュリティバイデザインとして知られています。デジタル庁が2022年に発表したガイドラインでは、「情報システムの企画工程から設計工程、開発工程、運用工程まで含めた全てのシステムライフサイクルにおいて、

▼図4 「APIの安全ではない利用」の攻撃メカニズム例



▼図5　開発フェーズ別の修正にかかる相対的コスト



※出典：NIST, "The Economic Impacts of Inadequate Infrastructure for Software Testing", Table 5-1.（脚注13）より作図

一貫したセキュリティを確保する方策」と定義しています[注13]。また、当該ガイドラインでは、セキュリティバイデザインを導入するメリットとして、一貫したセキュリティ対策の実装が可能となり、致命的なセキュリティ対策の漏れなどによる上流工程への手戻りを防止でき、納期確保やセキュリティコスト低減が可能になることが挙げられます。

NIST（米国立標準技術研究所）の調査研究では、ソフトウェア開発のさまざまな段階で発見された脆弱性の修復にかかる相対的なコストは、バグを発見するまでの時間が長いほど大きくなると分析しています[注14]。具体的には、要件定義フェーズで作り込まれた脆弱性を要件定義フェーズで修正した際のコストを1とした場合、当該脆弱性をリリース後に修正すると、30倍のコストがかかると指摘しています（**図5**）。イギリスのことわざに「1オンスの予防は1ポンドの治療に匹敵する」とあるように、まさに各フェーズで対策することが重要だと言えます。

セキュリティバイデザインの10原則

セキュリティバイデザインを理解するうえで、当該コンセプトが持つ原則を正しく理解していくことが重要です。一例として、英国政府はデジタルサービスを提供することを見据え、**表2**に示す10種類の設計原則を提唱しています[注15]。セキュア設計原則としては、1975年に提唱された「Saltzer & Schroederの8原則」[注16]が有名ですが、こうした設計原則を理解し、実践していくことが安全な開発のために重要と言えるでしょう。

また、情報処理推進機構（IPA）の中核人材育成プログラムにおいて「セキュリティ・バイ・デザイン導入指南書」[注17]を公表しており、

注13）DS-200 政府情報システムにおけるセキュリティ・バイ・デザインガイドライン
URL https://www.digital.go.jp/resources/standard_guidelines

注14）URL https://www.nist.gov/system/files/documents/director/planning/report02-3.pdf

注15）URL https://www.security.gov.uk/policy-and-guidance/secure-by-design/principles/

注16）URL https://web.mit.edu/Saltzer/www/publications/protection/index.html

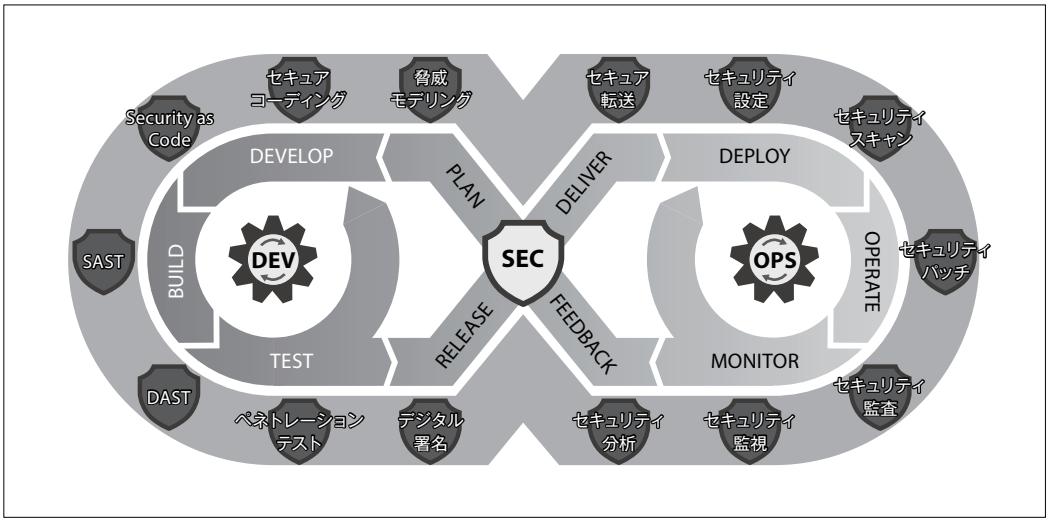
注17）URL https://www.ipa.go.jp/jinzai/ics/core_human_resource/final_project/2022/security-by-design.html

▼表2　セキュリティバイデザインの10原則

原則1	サイバーセキュリティリスクに対する責任を負う
原則2	安全な技術製品を採用・調達する
原則3	リスク主導型セキュリティ設計を行う
原則4	ユーザブルセキュリティを実践する
原則5	検知・対応をふまえたセキュリティ設計を行う
原則6	柔軟なアーキテクチャを設計する
原則7	攻撃対象領域を最小限に抑える
原則8	多層防御を実現する
原則9	実効性のある継続的チェックを実施する
原則10	安全な変更管理を実施する

Masato Matsuyuki : fxy.x@lambda-publishing.com

▼図6　DevOps開発プロセスにおいてとるべきセキュリティ対策



※**出典**："DoD Enterprise DevSecOps Fundamentals", Figure 4. https://dl.dod.cyber.mil/wp-content/uploads/devsecops/pdf/DoD EnterpriseDevSecOpsFundamentals.pdfより作成(訳は筆者による)

CISA（米国サイバーセキュリティ・社会基盤安全保障庁）の報告書[注18]もセキュリティバイデザイン原則とセキュリティバイデフォルト原則を提唱しています。

DevOpsで実現するセキュリティバイデザイン

開発担当と運用担当が連携・協力し、柔軟かつスピーディにシステム開発を行う手法であるDevOpsでは、APIの開発現場で多く取り入れられています。米国国防総省（DoD）は「DoD Cyber Exchange Program」として、DevOps開発プロセスにおいて各工程でとるべきセキュリティ対策を整理しています[注19]。当該プログラムで整理された概念図を**図6**に示します。

まとめ

本稿では、Web APIの普及と攻撃の背景について解説したあと、Web APIがもたらすリスクである「OWASP API Security Top 10」について解説しました。開発者がWeb APIを設計・開発する際、どのような脆弱性とその対策手法があるか知っておくことは、脆弱性を作り込まないためのアンチパターンとして重要です。また、最後にセキュリティバイデザインという考え方を紹介し、企画工程から設計工程、開発工程、運用工程まで含めたすべてのシステムライフサイクルにおいて、一貫したセキュリティを確保する重要性を説明しました。

APIの攻撃手法・対策手法についてさらに知りたい場合は、次の文献を参考にしてください。SD

- 『ハッキングAPI—Web APIを攻撃から守るためのテスト技法』
  Corey Ball 著、石川 朝久 訳、北原 憲、洲崎 俊 技術監修、オライリージャパン、2023年
- OWASP Japan「脆弱性診断士スキルマッププロジェクト」
  https://github.com/OWASP/www-chapter-japan/tree/master/skillmap_project
- APISec University
  https://www.apisecuniversity.com/
- Web Security Academy
  https://portswigger.net/web-security

注18）URL https://www.cisa.gov/resources-tools/resources/secure-by-design
注19）URL https://public.cyber.mil/devsecops/

5-2 DevOpsとシフトレフトを起点に考える Web APIセキュリティの重要ポイント



5-2では、5-1に引き続き、ゲーム販売用APIを題材として、セキュアなWeb API開発を行ううえで重要なポイントを解説します。本稿では、DevOpsとシフトレフトをキーワードに、セキュアなWeb APIを提供する方法を考えます。

Author 徳丸 浩(とくまる ひろし)
EGセキュアソリューションズ株式会社

5-1では、企画工程から設計工程、開発工程、運用工程まで含めたすべてのシステムライフサイクルにおいて、一貫したセキュリティを確保する重要性について説明しました。5-2では、題材として5-1に出てきたゲーム販売を提供するAPIを用いて、セキュアなWeb API開発の重要ポイントについて説明します。

システムライフサイクルの最上流からセキュリティを検証する

セキュアなAPI設計を進めるうえでは、各API固有のセキュリティリスクを正しく把握して、セキュリティ要件を決め、それに沿った設計を行う必要があります。そのためには、APIが持つセキュリティ脅威の分析が重要となります。

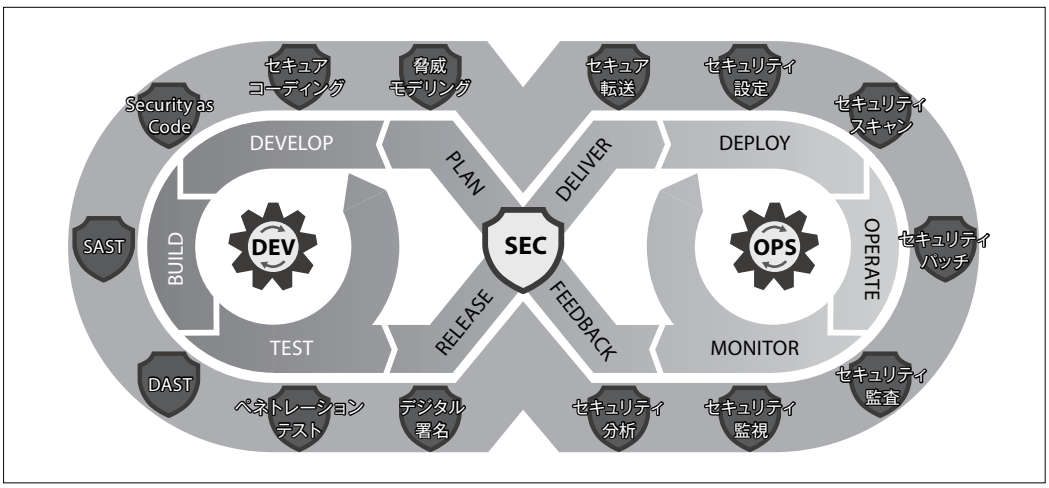
5-1ではDevOpsでセキュリティバイデザインを実現するという話題がありましたが、DevOpsのサイクルを回し始める前から、セキュリティを検討する必要があります(**図1**)。

DevOps開発に取り掛かる前の検討事項としては次があります。

・開発者のトレーニング
・サービス企画時のセキュリティ検討
・アーキテクチャ検討
・脆弱性対応の方針検討

そのうえで、個別の機能開発時のセキュリティについては、DevOpsサイクルの中で検討することになります。

▼図1 DevOpsのサイクルにおけるセキュリティ対策(5-1図6より)



Masato Matsuzaki : mzy.x@lambda-publishing.com

開発者のトレーニング

DevOpsにセキュア開発を取り入れるうえでは、開発者にセキュリティの知識を持ってもらい、開発プロセス全体でセキュリティを担保することが重視されます。このため、開発者全員にセキュア開発のトレーニングを定期的に実施するとともに、開発チーム内のセキュリティのリーダーを「セキュリティチャンピオン」として任命する取り組みが先進的な企業では始まっています[注1]。

サービス企画段階のセキュリティ

Web APIの開発企画の段階では、「このような機能を提供するAPIを開発したい」というアイデアから段階的に具体的な機能仕様に落とし込んでいくわけですが、その過程で次のセキュリティ課題を検討して、セキュリティ要件として具体化することが重要です。

- コンプライアンス上の問題
- サービスレベルの脅威分析

コンプライアンス上の問題

事業に供するAPIを開発するうえでは、各種法令やガイドラインを遵守しなければならないため、これらコンプライアンスに関する検討を企画の早い段階にて実施する必要があります。そのためには、まずサービスの概要を検討して、概略のサービスイメージや重要機能、認証や決済の方式について決めておく必要があります。

一例として、ゲーム販売を提供するAPIがクレジットカード決済機能を持つ場合、次の法令に従う必要があります。

- 割賦販売法
- 個人情報保護法
- 消費者契約法
- 特定商取引に関する法律
- 景品表示法
- ⋮

また遵守すべき、あるいは参考にすると良いガイドラインとしては次のものが挙げられます。

- クレジットカード・セキュリティーガイドライン[注2]
- PCI-DSS
- 各決済事業者が定める規約
- ECサイト構築・運用セキュリティガイドライン（IPA）[注3]
- ⋮

クレジットカード・セキュリティーガイドラインは、クレジット取引セキュリティ対策協議会がカード決済のセキュリティガイドラインをまとめたものですが、割賦販売法から参照されていて、カード決済を用いる事業者すべてが当ガイドラインを遵守する法的な義務があります。同ガイドラインの現在の運用では、決済代行事業者にセキュリティ対策の実施状況を記載した報告書を提出することになっている[注4]ため、開発の早い段階で決済代行事業者とコミュニケーションをとって、対応方針を決める必要があります。

サービスレベルの脅威分析

APIの機能に由来する脅威を検討するためには、企画の段階で機能概要を洗い出して、各機能にまつわる脅威を検討します。一般的にゲーム販売のAPIには次のような機能があるはずです。

- 会員管理（会員登録・変更・退会）
- 認証機能（ログイン、セッション管理、ログアウト）
- 商品検索

注1）URL https://careers.mercari.com/mercan/articles/19137/

注2）URL https://www.j-credit.or.jp/security/document/index.html

注3）URL https://www.ipa.go.jp/security/guide/vuln/guideforecsite.html

注4）URL https://www.meti.go.jp/press/2022/03/20230315001/20230315001.html

・ショッピングカート
・決済機能

前項で説明した各種ガイドラインや業界ごとのガイドラインでは、あらかじめ典型的なサービス機能をもとに、脅威分析がなされている場合があります。

たとえば、クレジットカードを取り扱う際のガイドラインであるPCI-DSSの場合は、クレジットカード情報という重要資産をベースにした脅威分析があらかじめなされており、カード情報を保護するためのセキュリティ施策がまとめられています。

一方、QRコード決済に関する業界のガイドラインである「一般社団法人キャッシュレス推進協議会 コード決済に関する統一技術仕様ガイドライン[注5]」では、QRコード決済機能にまつわる脅威とその対策がまとめられています。以下は、「コード決済に関する統一技術仕様ガイドライン【店舗提示型】」「5.2.1 静的QRコード」からの引用です。偽造した画面による不正決済の脅威と対策の必要性について説明されています。

> *静的QRコードを用いたコード決済では、原則的には利用者のモバイルデバイスに表示された決済完了画面を契約店が視認することによって決済の完了を確認する。当該決済完了画面が、以前に行われた決済画面のスクリーンショットや偽造・変造された画面等の不正な画面であった場合、契約店は決済が行われていないにもかかわらず、決済が行われたと**誤認してしまう可能性がある**。そのため、決済完了画面は容易に偽造等できるものであってはならない。*

ゲーム販売を行うAPIに戻ると、当該APIでは個人情報や決済情報という重要な資産を預かることになります。これら情報資産の保護は当然必要であるわけですが、重要情報に関するセキュリティーベストプラクティスはPCI-DSSなどの形で既知であるため、これらに従えばよいでしょう。

一方、ゲーム販売APIにて独自の機能を提供する場合は、必ずしも既知のベストプラクティスが適用できない場合があります。その例として、5-1でも紹介されていた転売対策について検討してみましょう。

サービスレベルの脅威分析例：転売防止の検討

人気のあるゲームを販売する場合は、そのゲームを大量に買い占めて転売するユーザーの存在が脅威として存在します。このため、5-1でも触れられていたように、ビジネスフローとしてこの転売対策を行っておく必要があります。これは実装工程で検討したのでは手戻りが発生する可能性が高いため、企画からサービス設計というかなり上流の段階で対応することが望ましいといえます。次に転売対策として考えられる対策案を列挙します。

・アカウントあたりの販売数の制限機能を設ける
・自動化された大量購入を検知する機能を設ける
・アカウント登録の際に本人確認（KYC）を強化する

とくに本人確認（KYC）については、開発や運用にコストがかかる場合もあるため、API開発の上流工程で仕様や対応方針を決めておく必要があります。

アーキテクチャ検討

APIのアーキテクチャ検討もDevOpsの開発サイクルに入る前に決定する必要があり、セキュリティにも大きな影響があります。次に、アーキテクチャ検討時の留意点を説明します。

インフラストラクチャの選定と脅威

最近では、Web APIの配置場所としてクラウドを用いることが多いと思いますが、仮想マ

注5）URL https://paymentsjapan.or.jp/publications/guidelines/

Masato Matsuyuki : xyx@lambda-publishing.com

シン（EC2などのIaaS）、コンテナ、サーバレスなどインフラストラクチャの方式によってもセキュリティ脅威と運用が変わります。とくに、脆弱性対応の責任範囲と脆弱性対応方法が変わることに注意が必要です。**表1**に各方式とミドルウェアの脆弱性対応の要否、その方法についてまとめました。

一方、いずれの場合でもアプリケーションおよびクラウド利用者側が配置したコンポーネントについては、利用者側で脆弱性対応が必要になります。

認証方式の選定

APIが用いる認証方式としては、まだまだパスワード認証が広く用いられていますが、Open ID Connectなどによる認証連携も広く用いられています。また、パスワードの運用が難しく不正ログインの温床になっていることからFIDOやパスキー、あるいはSMS認証マジックリンク方式などのパスワードレスの認証も検討するとよいでしょう。これらの方式を検討する際には、認証機能に対する脅威分析を実施する必要があります。

認証機能における脅威

一般的に、認証機能には次のような脅威があります。

- 弱いパスワードに対する辞書攻撃やパスワード推測
- パスワードリスト攻撃
- フィッシング

これらに対する対策を**表2**にまとめました。

これら脅威と対策を勘案しながら、どの認証方式を採用するかを決定します。現在のセキュリティ関係者の認識ではパスワード認証は脆弱なものとみなされ、パスキーなどパスワードレスの認証方式が望ましいとされています。しかしながら、パスキーがまだ普及途上であること、パスキーの実装に関するベストプラクティスがまだ十分そろっていないことなどから、本稿で題材とするAPI企画ではパスワード認証とソーシャルログインの2通りを採用したとしましょう（これはあくまで脅威分析の題材として紹介したものであり、パスワード認証を推奨しているわけではありません）。

パスワード認証に関しては、前述した認証の脅威はすべて存在します。このため、これらへの対策が必須になります。以下、パスワード認証のセキュリティ強化としてはSMSを用いた2段階認証を採用する想定で話を進めます[注6]。

セッション管理方式

パスワードなどによる認証は利用者がサービスを使い始める段階で行われ、認証後のAPIには認証状態を引き継ぐ必要があります。本稿では認証状態の引き継ぎのことをセッション管理と呼ぶことにします。セッション管理には、セッションIDをCookieに保存する伝統的なセッション管理のほかに、JWTというトーク

注6）SMSを用いた2段階認証には、SIMスワップなどの脅威がありますが、誌面の関係で立ち入らないことにします。

▼表1　インフラストラクチャごとの脆弱性対応

	ミドルウェアの脆弱性対応	対応方法
仮想マシン	利用者側責務	aptやdnfコマンドの実行
コンテナ	利用者側責務	対応済みイメージと差し替え
サーバレス	クラウド事業者側責務	利用者側は対応不要

▼表2　認証の脅威と緩和策の表

脅威	影響	対策
弱いパスワードの推測・辞書攻撃	不正ログインによるなりすまし	・パスワード登録時のチェック ・2段階認証
パスワードリスト攻撃	同上	・2段階認証
フィッシング	同上	・フィッシング耐性のある認証方式の採用 ・ユーザーへの注意喚起

▼リスト1　ヘッダー

```
{
  "alg": "HS256", ←署名アルゴリズム
  "typ": "JWT" ←JWTであることを示す
}
```

▼リスト2　ペイロード(トークンの中身)

```
{
  "sub": "1234567890", ←ユーザーを示す識別子
  "name": "Hiroshi Tokumaru", ←ユーザーの名前
  "iat": 1516239022, ←トークン作成日時(UNIX時間)
  "exp": 1516242622 ←トークン有効期限(UNIX時間)
}
```

ンの利用が広まっています。



JWTによるセッション管理の問題



まずJWTの説明をします。JWTはログインIDやメールアドレスなど、認証保持に必要な情報をJSON形式で記述して署名により改ざん防止をした形式です。**リスト1、2**にJWTの例を示します。

これらをBase64URLという形式でエンコードして署名を付けたものがJWT(JSON Web Token)です(**リスト3**)。

JWTを使うメリットはサーバ側で状態を保存することがないという点です。このため、Webサーバを増やすだけで簡単にスケールアップができます。一方、JWTはサーバ側に状態を持たないことから、セキュリティインシデントなどがあっても、即座にログアウトを実現することができません。

このため、セキュリティ要件に応じて、古典的なセッション管理とJWTのどちらの方式を使うかという検討が必要になります。このように、セッションIDによるセッション管理とJWTなどのトークンによる方式には一長一短があります。このため、サービスの要件とセキュリティ要件を併せて考慮して、どの方式を使用するかを決めることになります。



Cookie vs. localStorage論争



セッションIDやトークン類をCookieに保存するのがよいか、localStorageに保存するのがよいかという議論があります。「localStorage使うな」などでネット検索すると非常に多くの記事を読むことができます。典型的な主張は次のとおりです。

Cookie擁護派：

localStorageにトークンを保存するとクロスサイトスクリプティング(XSS)により容易に盗まれるのでセッションハイジャックされてしまう。CookieにはhttpOnly属性があるので、これを設定しておけばXSSにより盗まれることはない

localStorage擁護派：

Cookieにトークンやセッション ID を保存するとクロスサイトリクエストフォージェリ(CSRF)の脅威が生じる。localStorageにトークンを保存すると原理的にCSRF脆弱性は混入しない

これら指摘は部分的には正しいのですが、結論を言えば、両者に大きなリスクの差はありません。Cookieを使うと確かにCSRF脆弱性の対策が必要ですが、CSRFはアプリケーションフレームワークの機能で容易に対策することができます。一方XSSについては、仮にhttpOnly属性のついたCookieを使用していてもFetch APIやXMLHttpRequestによる通信にはCookieが付与されるので、結局なりすましはできてしまいます。すなわち、どちらが安全というほどの違いはなく、セッションIDやトークンをどこに保存するかは、セキュリティ上大きな差はありません。

▼リスト3　署名を含めたJWT全体の例

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6Ikhpcm9zaGkgVE9rdW1h
cnUiLCJpYXQiOjE1MTYyMzkwMjIsImV4cCI6MTUxNjI0MjYyMn0.OgqiSdIeUJOUo8WgKiGg5TcgxH1jRWB3LZzuC-yOsRE
```

Masato Matsuyuki : xxx@lambda-publishing.com

脆弱性対応の方針検討

次に脆弱性対応の方針を検討します。APIが利用するコンポーネントのインストール方法により、脆弱性対応の方法が変わるため、インストール方法と脆弱性診断の方法はセットで考える必要があります。たとえば、Linuxディストリビューションの標準パッケージでインストールしたコンポーネントは、dnf（Red Hat系ディストリビューションの場合）やapt（Debian系ディストリビューションの場合）などでアップデートすることができ、インストール済みパッケージの種類やバージョンもこれらパッケージ管理ツールのコマンドで参照することができます。また、RubyのgemやNode.jsのnpmなどの言語ごとのパッケージ管理システムも同様です。

古くからUnix系OSを使っている人の中には、「オープンソースソフトウェアはソースコードからビルドすることが望ましい」と考えている人も多いと思いますが、この方法だとコンポーネントの依存関係が管理されていない状態になるので、できるだけパッケージ管理システムを使ってコンポーネントを導入して、依存関係が管理された状態を維持することを推奨します。

アプリケーションフレームワークの脆弱性対応機能を活用する

モダンなアプリケーションフレームワークには、SQLインジェクションやXSSなどの脆弱性には自動的に対応してくれるものも多いです。プロジェクトの初期段階にて、次に述べるような著名な脆弱性については、フレームワークの機能を活用して対応する方法を検討しておくとよいでしょう。

クロスサイトスクリプティング（XSS）

一般的にWeb APIではJSON形式をレスポンスとして返す場合が多く、JSON形式へのエンコードはライブラリなどで自動的に行われるため、Web APIにXSS脆弱性が混入するケースは多くはありません。1つ注意があるとすれば、レスポンスのContent-Typeです。まれに、JSONなのにContent-Typeがtext/htmlになっている場合がありXSSの原因となります。ソースコードレビューとテストの両方で、Content-Typeが正しいものであるかを確認しておきましょう。

SQLインジェクション

SQLインジェクションについては、フレームワークなどのO/Rマッパーやクエリビルダーの機能を自然に使っていると混入しにくいものではありますが、「フレームワークを使っているから大丈夫」と過信すると脆弱性が混入するケースはあります。具体的には、O/Rマッパーなどで、生のSQL文やWHERE句を記述できる機能を使って、WHERE句などを文字列処理で作成するとSQLインジェクションの原因になります。

このため、開発標準などで、SQLクエリの生成方法を決めておくとよいでしょう。

認可制御

認可制御についてもアプリケーションフレームワーク側で対応機能を持つものもありますが、XSSやSQLインジェクションとは異なり、開発者が自覚的に認可制御機能を組み込む必要があります。

認可制御の要点は次のとおりです。

- 認可制御の基本は、次の直前に権限を確認すること
  - 秘密情報の表示
  - 権限の必要な機能実行
  - その他、権限を必要とする操作
- 権限を確認する基準は、セッション変数に保存したログインIDやJWTで保持したID

また、ユーザーごとに権限を抽象化した「ロール」を定義したうえで、各ロールが持つ権限を権限マトリクスとしてまとめることを推奨します。

図2は、企業向けSaaSにおいて、全権限を持つ「システム管理者」、クライアント企業内の管理権限を持つ「企業管理者」、一般の「ユーザー」がそれぞれ持つ権限をまとめた権限マトリクスの例です。

認可制御不備については、後述の脆弱性スキャナでは検出できない場合が多いので、開発者が記述するAPIの自動テストの中で権限のテストも行うとよいでしょう。権限マトリクスは、テスト作成のインプットとしても重要です。

クロスサイトリクエストフォージェリ（CSRF）

Web APIでもCSRF脆弱性が混入するケースはありますが、これはCookieを用いてセッション管理をしている場合に限られます。この場合は、アプリケーションフレームワークが提供するCSRFトークンの機能を有効化してCSRF対策を行います。一方、トークンをHTTPリクエストヘッダーに設定してログイン状態を保持する場合は、CSRF脆弱性は混入し得ないので対策を行う必要もありません。

脆弱性診断の方法を決める

DevOpsにセキュリティを導入する場合、脆弱性診断もできるだけ早期に自動化することが推奨されています。このために利用できるツールに次のものがあります。

SAST（Static Application Security Testing）

- ソースコードを解析することで自動診断する
- プログラムが実行可能にならないうちからセキュリティチェックが早期に可能

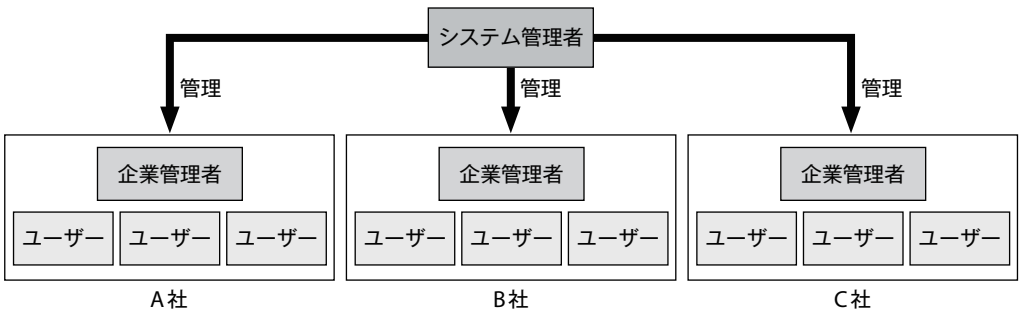
DAST（Dynamic Application Security Testing）

- 攻撃者の立場でインターネット越しに疑似攻撃を行う
- DASTというと一般的にはアプリケーションに対するツール診断を指す
- ZAP（旧称OWASP ZAP）、Burp Suite、App Scan、AeyeScan、Vexなど

SCA（Software Composition Analysis）

- アプリケーションが依存するライブラリやフレームワークを分析して脆弱性を把握
- npmやgemのpackage.jsonなどを確認
- Synkや、簡易的なものとしてnpm audit、

▼図2　企業向けSaaSにおける権限マトリクス



		システム管理者	企業管理者	一般ユーザー
権限	企業の追加	○	×	×
	企業管理者の追加・削除	○	×	×
	企業内ユーザーの追加・削除	○	○	×
	自分自身のパスワード変更	○	○	○
	他人のパスワード変更	○	企業内に限り○	×

Masato Matsuyuki : mxy.x@lambda-publishing.com

GitHub Dependabotなど

コンテナイメージスキャン

・コンテナイメージに対する脆弱性スキャン
・OSやソフトウェアの依存関係から脆弱性情報を抽出
・Trivy、Snykなど

これらを使うと、ソースコードをGitHubにコミットした段階で自動的にSASTやSCAによりアプリケーションや依存コンポーネントに対する診断がされ、ツールによっては脆弱性修正のプルリクエストまで作成してくれます。このため、開発者がプルリクエストの修正内容を確認してマージすれば脆弱性対応ができてしまいます。

図3はSnykというツールのSAST機能により、Express.js記述のAPIのXSS脆弱性が指摘されている（11行目）例です。前述のように、ライブラリを用いてJSONを生成する限りXSS脆弱性は入りにくいのですが、この例では、エラーを返すJSONを文字列処理で生成しているため、XSS脆弱性が混入しています[注7]。

APIのデプロイにコンテナを使っている場合は、コンテナイメージスキャンというツールでOSやミドルウェア、コンポーネントの脆弱性診断を簡単に行うことができ、CI/CDに組み込むことも容易です。

DASTはAPIがステージング環境でテストできるようになったタイミングで、APIを動かしながら自動診断するものです。

ただし、ここで紹介した自動脆弱性診断ツールには制約が多く、診断できない脆弱性も多いので、最終的にはセキュリティ専門家による手動の脆弱性診断を実施する必要があります。

◆ ◆ ◆

以上がDevOpsのサイクルに入る前に行うべきセキュリティ対応です。実際の開発に入る前の検討が多いことがおわかりいただけたかと思います。このように開発のできるだけ上流工程でセキュリティを検証する考え方は「シフトレフト」と呼ばれています[注8]。

DevOpsの開発サイクルにおけるセキュリティ対応

上流工程でセキュリティの方針が決まれば、DevOpsの開発サイクル内では、その方針を実行するだけです。ここでは、DevOpsのPLANにおける「機能レベルの脅威分析」の例として、架空の開発チームにおける2段階認証実装の脅威レビューの模様を紹介します。

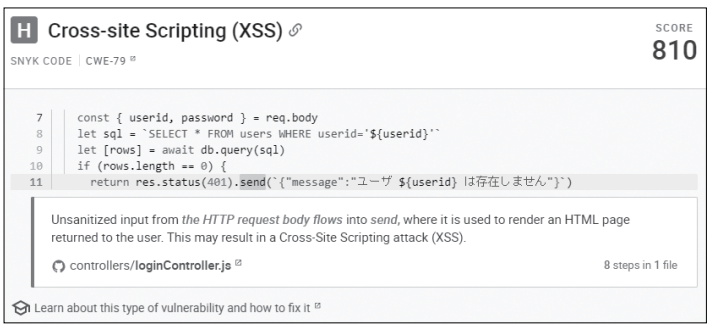
APIの実装に由来する脅威

このチームのプロジェクトではSMSを用いた2段階認証を実装することになり、担当者が実装方式を検討してレビューにかけるために、2段階認証の状態遷移図を作成しました。担当者が作成した状態遷移図を**図4**に示します。

この状態遷移図を使って開発チームでレビューを実施したところ、**図5**の吹き出し部分のようなポイントが指摘されました。実際のレビューでも有識者から受けた指摘を設計チャートなどに付箋などで貼り付けていき、後で脅威分析表などの形にまとめると効果的です。

図5の指摘をまとめると次のようになります。

▼図3　Snykによる脆弱性診断を実施した例。XSS脆弱性が指摘されている



注7）この例にはSQLインジェクション脆弱性もあります。

注8）レフトとは時間軸上の左側、すなわち過去を示し、上流工程を指す言葉として使われています。

・そもそも状態遷移の正常系しか記載されていない
・SMSで送信されてきた6桁数字の確認回数が制限されていない
・確認回数のタイムアウトが定義されていない

このレビュー結果を反映して、担当者は**図6**のような状態遷移図による設計に変更して、レビューの承認を得ました。

現実に2段階認証の確認コードの回数制限が漏れていたために不正ログインされた事例として、ゆうちょ銀行が提供していた振り込みサービスmijicaの例があります。mijicaでは、mijicaカード裏面の5桁の番号を2段階認証用に用いていましたが、その5桁の番号のチェックの回数制限がなされておらず、何回でも試行することができました。このため高々10万回の試行で2段階認証を突破できたことになります。記者会見[注9]にてそのような攻撃が行われたことが明言されています。

ケーススタディ：パスワード変更機能

今度はもう少し複雑な例を示します。**図7**は典型的なパスワード変更の画面であり、旧パスワード（現在のパスワード）を確認した後、新しいパスワードを指定して、現在のパスワードが正しい場合のみパスワードを変更できるようにしています。

この処理を**図8**のようなシーケンスで処理するとします。

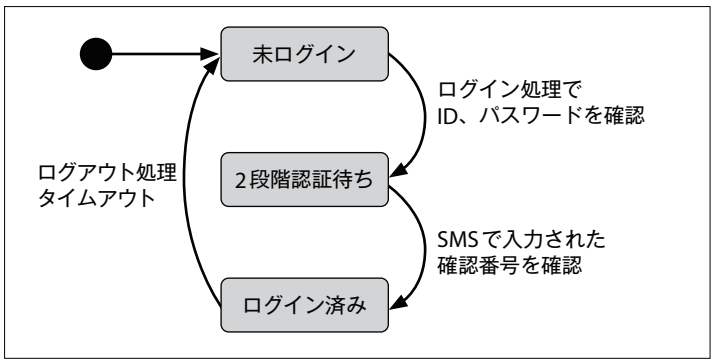
第一のAPIで再認証を行います。これは現在パスワードを入力して、ログインチェックを再度行い、正しければOKを返すものです。JavaScriptによりこのAPIを呼び出し、再認証がとれた場合のみパスワード変更のAPIを呼び出す流れになっています。

しかしこのチェック処理は簡単に回避されてしまいます。たとえば、このAPIを用いたサイトにXSS脆弱性があった場合、ハッカーはXSS攻撃によりパスワード変更のAPIを呼び出すことができ、この場合パスワードのチェックは作用しません。仮に、再認証のチェックに脆弱性がなければ、ハッカーは利用者のパスワードは知らないので、このような攻撃ができないことになります。

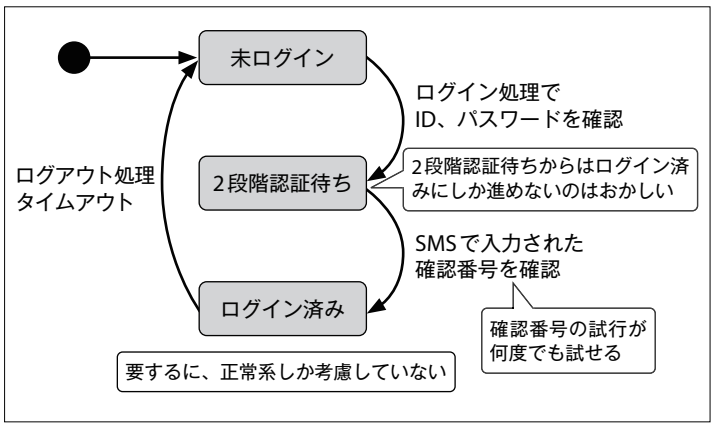
この問題に対応するには次の方法が考えられます。

・パスワード変更APIの中で

▼図4　2段階認証機能の状態遷移



▼図5　状態遷移に対する指摘を追加した例



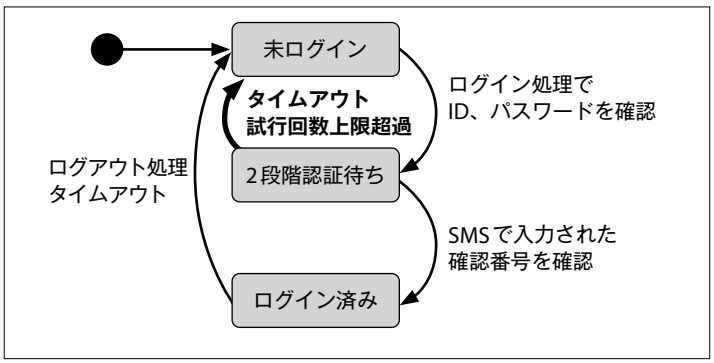
注9） 2020年9月24日ゆうちょ銀行会見 URL https://www.youtube.com/watch?v=AoNj__65k0A

Masato Matsuyuki : xxx@lambda-publishing.com

旧パスワードの確認まで行う
・JWTのログイン日時クレーム（auth_time）を用いて、ログイン後一定時間が経過している場合はエラーにする

このようなAPIの仕様変更を伴う場合があるので、できるだけ早期に検討しておかなければなりません。

▼図6　改善した設計の状態遷移図



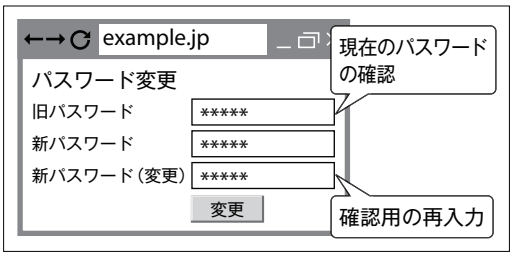
継続的な脆弱性対応の重要性

開発したWeb APIは、ソースコードを修正しない限りは新しい脆弱性が生じることはありません。一方、APIが用いるコンポーネントは、時間とともに新しい「既知の脆弱性」が発見され、対応が必要になります。

そのような有名な脆弱性としてLog4Shell（CVE-2021-44228）があります。Log4Shellの特徴を次に示します。

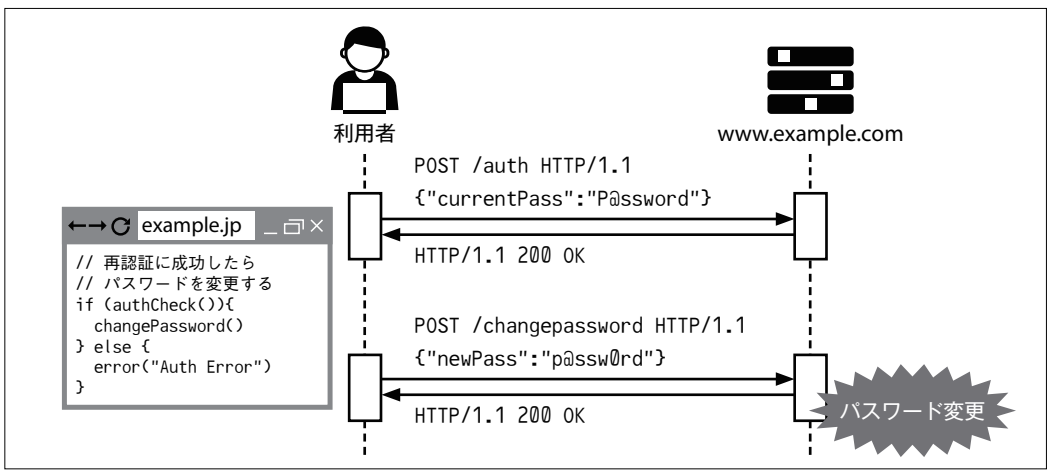
・Log4Shell（CVE-2021-44228）は、Java向けのログ出力ライブラリLog4jの脆弱性で、2021年12月に公開された

▼図7　パスワード変更画面



・Log4jは著名なライブラリであり、Javaアプリケーションやライブラリの多くで使用されている
・Log4Shellは外部からコマンド実行可能（RCE：Remote Code Execution）な危険な脆弱性
・Log4Shellの攻撃ルートは多様であり、外部公開していない、内部のサーバにも攻撃の可能性がある

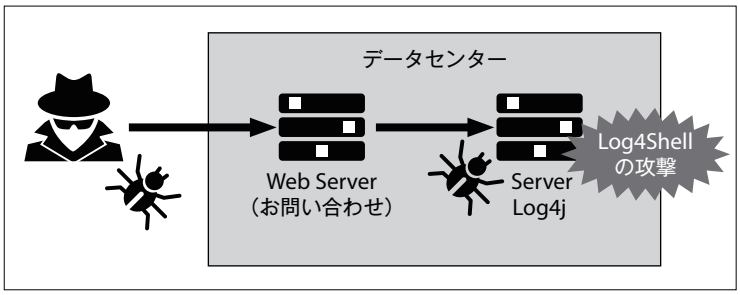
▼図8　パスワード変更のシーケンス



・脆弱性対応の正式版が公開される前に攻撃コードが出回った

Log4Shellが存在すると内部サーバに対しても攻撃できる様子を**図9**に示します。この例では、外部公開されていないログサーバに対して、外部ハッカーがWebサイト経由で攻撃する様子を示しています。

▼図9　Log4Shellにより内部サーバが攻撃される様子



前述のように、新しい脆弱性は日々発見されるため、APIの運用・監視のフェーズで脆弱性情報を把握して対応する必要があります。この目的のためには、SCAによる継続的な脆弱性対応が効果的です。

図10は、DevOpsの監視フェーズでSCA運用によりLog4Shell脆弱性を把握した後に、自動的に脆弱性対応のプルリクエストが生成され、開発者がマージした後に、自動テストが行われるイメージを示しています。

まとめ

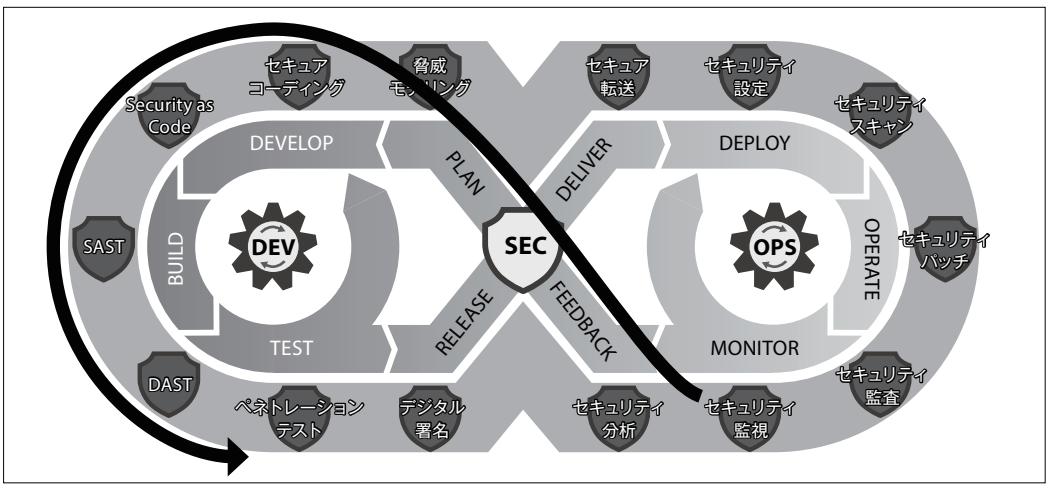
5-2では、DevOpsとシフトレフトの考え方に基づいて、Web APIのサービス企画から脅威分析を行ってセキュリティ要件や開発方針や脆弱性対応の方針を立てる方法を説明しました。アプリケーションフレームワークによる脆弱性対応の自動化やSASTやDAST、SCAなどによる脆弱性診断の自動化が普及しつつありますが、開発者が自発的にセキュリティ対策を実施しなければならないことは変わりありません。

Web APIに関連して、筆者の次のスライドも参考になると思うのでご参照ください。SD

・SPAセキュリティ超入門[注10]
・SPAセキュリティ入門[注11]
・脆弱性対応入門[注12]

注10）URL https://www.docswell.com/s/ockeghem/K2PPNK-phpconf2022
注11）URL https://www.docswell.com/s/ockeghem/ZM6VNK-phpconf2021-spa-security
注12）URL https://www.docswell.com/s/ockeghem/5LL666-Introduction-to-Vulnerability-Response

▼図1　DevOpsの監視フェーズでLog4Shell脆弱性の把握後自動的に脆弱性対応を行うイメージ



5-3 正しく検知して攻撃経路をふさぐ

脆弱性



本稿では、Web APIに対する脆弱性診断の具体的な手法を解説します。Web APIへのアクセス方法を確認し、「やられWebアプリケーション」とクロスサイトスクリプティング脆弱性を題材に、デベロッパーツールや脆弱性診断用ツールを駆使した診断を体験します。

Author 松本 隆則(まつもと たかのり)
脆弱性診断研究会
X(Twitter) @nilfigo

Web APIに対する脆弱性診断

Web APIといえども、一般的なWebアプリケーションに対する診断と基本的な考え方は同じです。Web APIの仕様を把握し、APIのエンドポイントやAPIアクセスに必要なデータを用意する必要があります。

Web APIへアクセス

まず初めに、コマンドプロンプトやターミナルなどで実行できるcurlコマンドを使用してWeb APIにアクセスする方法を示します。

例として、日本の統計が閲覧できる政府統計ポータルサイト「e-Stat 政府統計の総合窓口」が提供しているAPI「e-Stat API機能[注1]」に対して、curlコマンドを使用してアクセスしてみます(**図1**)。

「{"GET_STATS_LIST":{」から始まる文字列がWeb APIからのレスポンスです。「\uXXXX」形式の文字列は、認証なしにWeb APIへアクセスしたので返されたエラーメッセージなのですが、「Unicodeエスケープシーケンス」としてエンコードされています。このままでは読めないので変換が必要です。

Unicodeエスケープシーケンスを変換

Webブラウザに標準で搭載されているデベロッパーツール[注2]を利用するとUnicodeエスケープシーケンスの文字列を元の日本語に変換(デコード)できます。

以降の操作ではWebブラウザはChromeを使用します。

Webブラウザ右上にある3点メニューから「その他のツール」>「デベロッパーツール」とたどってデベロッパーツールを起動します。「コンソール」タブを開いてエンコードされている文字列を貼り付けると、デコードされた結果が表示されます(**図2**)。

Web APIアクセス時の注意事項

「Web APIへアクセス」項で試行した操作は、脆弱性診断で使用する文字列やデータ(ペイロー

注1) URL https://www.e-stat.go.jp/api/

注2) Chrome/Chromiumの場合。Firefoxでは「ウェブ開発ツール」、Safariだと「Webインスペクター」。

▼図1 curlでe-Stat機能にアクセス

```
$ curl -X GET "https://api.e-stat.go.jp/rest/3.0/app/json/getStatsList"
{"GET_STATS_LIST":{"RESULT":{"STATUS":100,"ERROR_MSG":"\u8A8D\u8A3C\u306B\u5931\u6557\u3057\
u307E\u3057\u305F\u3002\u30A2\u30D7\u30EA\u30B1\u30FC\u30B7\u30E7\u30F3ID\u3092\u78BA\u8A8D\
u3057\u3066\u4E0B\u3055\u3044\u3002","DATE":"2023-12-20T01:03:50.592+09:00"},"PARAMETER":{"
DATA_FORMAT":"J"}}}%
```

▼図2　デベロッパーツールでデコード



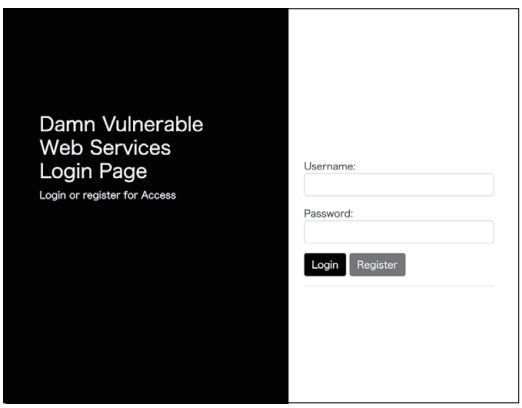
▼図3　DVWSを起動

```
$ git clone https://github.com/snoopysecurity/dvws-node.git
$ cd dvws-node
$ docker-compose up
```

ドと呼びます）を伴わないアクセスですのでセキュリティ上は問題ありません。しかし、脆弱性診断を目的としてアクセスを試みる場合、必ず守らなければならない注意事項があります。

脆弱性診断の方法を覚えると、すぐにでもあちこちのWebアプリケーションやWeb APIに対して診断を実施したくなりがちですが、他者が管理・運営しているWeb APIに対して、許可なく脆弱性診断を実施しないでください。無許可で診断を実施すると、不正アクセス禁止法などの法律に触れて処罰されたり、診断の副作用でシステムに障害が発生して損害賠償請求を受けたりするおそれがあります。

▼図4　DVWSのトップ画面



脆弱性診断を安全に体験

誰にも迷惑を掛けずに脆弱性診断を体験するために、OWASP Juice Shop[注3]やWebGoat[注4]のような、いわゆる「やられWebアプリケーション」を用意しましょう。本稿では「Damn Vulnerable Web Services（DVWS）[注5]」というWebアプリケーションを使用します。

DVWSについて

DVWSは、さまざまな脆弱性が埋め込まれているWebサービスやWeb APIを内包するWebアプリケーションです。

DVWSのソースコードは「GNU GENERAL PUBLIC LICENSE Version 3（GPL-3.0）[注6]」ライセンスで公開されているので、誰でも自由にアプリケーションを使用できます。

DVWSの起動

DVWSの起動方法はいくつかありますが、Dockerを利用するのが簡単です。

https://github.com/snoopysecurity/dvws-nodeのREADMEに記載されているコマンドを実行してDVWSを起動します（**図3**）。

コマンド実行後にWebブラウザで「http://localhost/」にアクセスするとDVWSのトップ画面が表示されます（**図4**）。

脆弱性の検出

DVWSに存在している「クロスサイトスクリプティング（XSS）」脆弱性を検出してみます。

注3）URL https://owasp.org/www-project-juice-shop/
注4）URL https://owasp.org/www-project-webgoat/
注5）URL https://github.com/snoopysecurity/dvws-node
注6）URL https://www.gnu.org/licenses/gpl-3.0.ja.html

Masato Matsuyaki : fxy.x@lambda-publishing.com

Webブラウザで診断

トップ画面の「Username」および「Password」欄に「sample」と入力して［Login］ボタンを押下すると、「Login Failed! User sample not found!」というメッセージが表示されます。

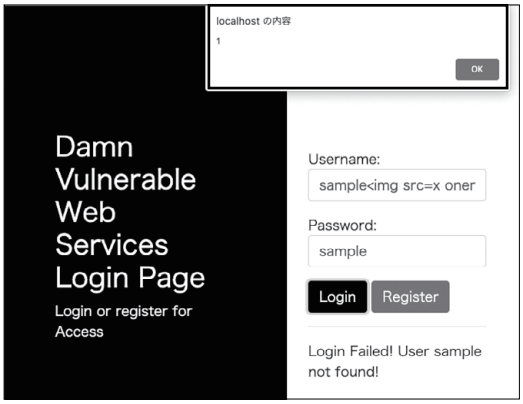
次に、Passwordには同じ文字列を入力し、Username に「sample<img src=x onerror=alert(1)>」を入力して［Login］ボタンを押下するとダイアログが表示されます（**図5**）。

ここで、ダイアログの［OK］ボタンの押下後に、画面上で右クリックして「ページの表示」メニューからHTMLソースコードを確認しても、入力した文字列や実行されたJavaScriptのコードを見つけることはできません（**図6**）。入力した文字列がJavaScriptにより動的に画面に反映されているためです。

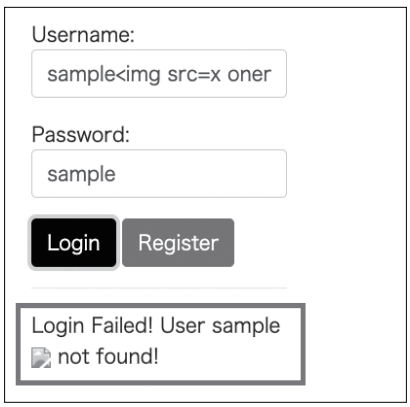
このような場合はWebブラウザのデベロッパーツールで確認します。

図5で［OK］を押下すると表示されるメッセージ「Login Failed! User sample not found!」（**図7**）で、壊れた画像を示すアイコンの上で右クリックして［検証］メニューを選択します。そうすると、XSS検証のために入力した文字列に含まれるHTMLタグや属性内のJavaScriptコードがWebブラウザに解釈されて表示されたことを確認できます（**図8**）。

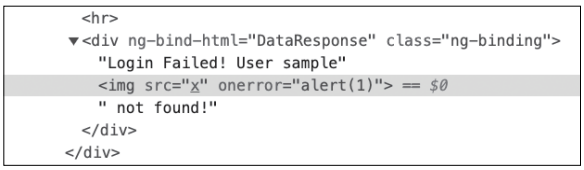
▼図5　XSSを検証



▼図7　図5の［OK］押下後のメッセージ



▼図8　デベロッパーツールで確認



```
      <hr>
    ▼<div ng-bind-html="DataResponse" class="ng-binding">
       "Login Failed! User sample"
       <img src="x" onerror="alert(1)"> == $0
       " not found!"
      </div>
    </div>
```

▼図6　HTMLソースコードを確認




```
1
2  <script src="./js/jquery.min.js"></script>
3  <script type="text/javascript" src="./js/angular.min.js"></script>
4  <link href="./css/bootstrap.min.css" rel="stylesheet" id="bootstrap-css">
5  <script src="./js/bootstrap.min.js"></script>
6  <link href="index.css" rel="stylesheet" id="inline-css">
7  <!------ Include the above in your HEAD tag ---------->
8
9  <div class="sidenav">
10         <div class="login-main-text">
11            <h2>Damn Vulnerable Web Services<br> Login Page</h2>
12            <p>Login or register for Access</p>
13         </div>
14      </div>
15      <div class="main">
16         <div class="col-md-6 col-sm-12">
17            <div class="login-form">
18                  <div ng-app="app" ng-controller="MyController">
```

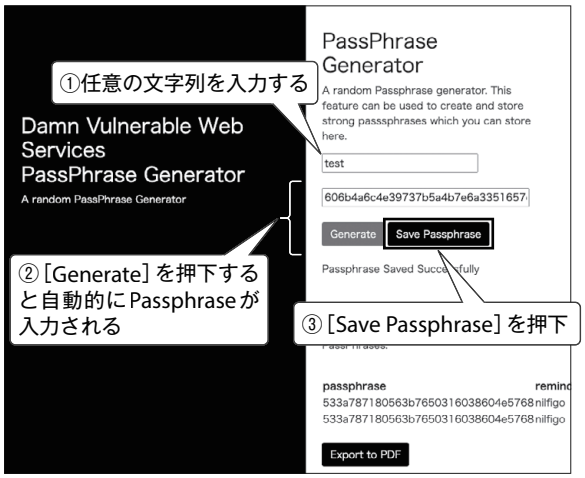
専用ツールによる脆弱性診断

Webブラウザのみでも脆弱性診断は可能ですが、快適に作業を進めるためにZAP[注7]やBurp Suite[注8]などの脆弱性診断用ツールを使用します。ツールを使用すると、手動診断を効率化するだけではなく、自動診断による迅速な脆弱性の検出を期待できます。

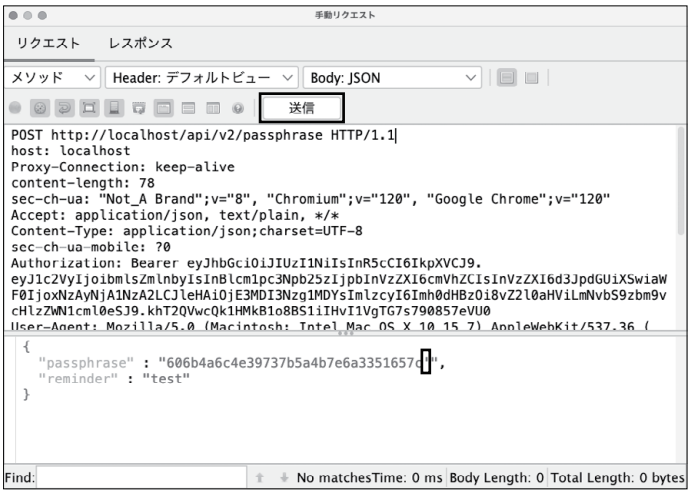
注7) URL https://www.zaproxy.org/
以前は「OWASP ZAP」という名前でしたが、2023年8月にZAPのプロジェクトがOWASPから離脱したため、OWASPが付かない「ZAP(Zed Attack Proxy)」が正式名称です。ちなみに、「OWASP」は「The Open Web Application Security Project」の略でしたが、現在は「The Open World wide Application Security Project」の略とされています。

注8) URL https://portswigger.net/burp

▼図9 Passphraseを保存



▼図10 シングルクォーテーションを付加して送信



診断ツールによる手動診断

本項ではDVWSに存在する「SQLインジェクション」脆弱性をZAPにより手動で検出してみます。

ZAPのセットアップは公式サイトの「Getting Started[注9]」などを参考にして進めてください。なお、ZAP使用時は「プロテクトモード」に切り替えて診断を実施することを推奨します。プロテクトモードで診断することで、他者のシステムに対して誤って脆弱性診断を実施してしまうリスクを軽減できます。

DVWSのトップ画面で任意の「Username」と「Password」を入力して[Register]ボタンを押下してアカウントを登録します。登録したアカウントでログインして[Passphrase Generator]というリンクを押下します。

フォームの最初のテキストボックスに任意の文字列を入力して[Generate]ボタンを押下すると2番目のテキストボックスに自動的に「Passphrase」が入力されます。この状態で[Save Passphrase]ボタンを押下すると、「Passphrase Saved Successfully」と表示されてシステムにパスフレーズが保存されます(**図9**)。

保存されたらZAPの「履歴」タブで[Save Passphrase]ボタンを押下した際のリクエストを探します。ボタン押下時のリクエストのURLはhttp://localhost/api/v2/passphraseでメソッドはPOSTです。リクエストボディは次のようなJSON形式です。

注9) URL https://www.zaproxy.org/getting-started/

```
{
  "passphrase" : "606b4a6c4e39737b5a4b7e6a3351657c",
  "reminder" : "test"
}
```

見つけた履歴行を右クリックして［再送信…］メニューを押下すると「手動リクエスト」ダイアログが表示されます。

ここで、ダイアログの下側のリクエストボディ内に記載されている"passphrase"の値の末尾に「'（シングルクォーテーション）」を付加して［送信］ボタンを押下します（**図10**）。

SQL関連のエラーメッセージを含むJSONが返されたことを確認できます（**図11**）。

このように、脆弱性診断ツールを使用すると、リクエストの改ざんやレスポンスの確認を、Webブラウザのデベロッパーツールよりも直感的な操作で実施できます。

診断ツールによる自動診断

ZAPやBurp Suiteの Professional 版（Burp Suite Professional）には自動診断ツールが付属しています（誌面の都合で診断手順は省略します）。

試したところ、ZAPの自動診断ツール「動的スキャン（Active Scan）」では残念ながらSQLインジェクションを検出できませんでした。一方、Burp Suite Professionalの「Scanner」ではSQLインジェクションの検出に成功しています（**図12**）。

この結果のように、自動診断では、存在している脆弱性を検出できない可能性があることについて留意が必要です。しかし、自動診断は手動診断よりも大量の診断作業をすばやく実行できるため、それぞれの診断手法のメリットとデメリットを理解したうえで診断作業を進めることが重要です。

まとめ

Web APIに対する脆弱性診断について、考え方や具体的な実践方法を解説しました。

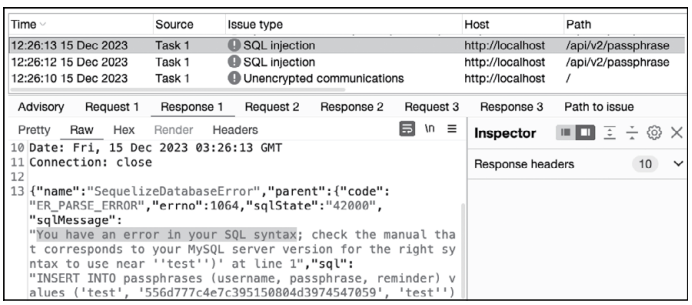
繰り返しになりますが、Web APIに対する脆弱性診断は、一般的なWebアプリケーションに対する脆弱性診断と同様の考え方で進められます。まずは脆弱性診断の基本を学び、そのうえで、Web API特有のアクセス方法や脆弱性などについて知識を深めることをお勧めします。

本稿がシステムの安全性向上に少しでも寄与できれば幸いです。SD

▼図11　SQLインジェクションを確認



▼図12　Burp Suite Professionalの結果



5-4 サービスを不正アクセスから守る

認証・認可



本稿では、Web APIセキュリティにおける認証・認可の関係や標準仕様のアプリケーションへの組み込み方について解説します。認証・認可の基本事項を確認した後、OpenID Connect 1.0、OAuth 2.0とその組み込み方を説明します。

Author 川村 修平(かわむら しゅうへい)
Kong株式会社

認証・認可とは

そもそも認証・認可とは何なのか?という基本事項から押さえていきます。

まず、認証(Authentication)とはIDやパスワードに代表される情報を用いて、ユーザーやシステムの本人性を検証することです。このとき、本人性の検証は、次の3要素のうち1つ以上を用いて行われます。

- 知識要素:パスワードなど本人のみが知っている情報のこと
- 所有要素:スマートフォンやICカードなど本人のみが所有している要素のこと
- 生体要素:指紋や顔など本人の生体情報のこと

従来、Webの世界では知識要素のみを用いた認証が行われてきましたが、近年ではより安全・確実に認証するために2つ以上の要素を用いて認証を行う多要素認証(MFA:Multi Factor Authentication)という方法も広く使われています。

次に、認可(Authorization)とはあるリソースに対して処理を行うことを許可することです。身近な例ではよく電車の切符に例えられます。電車の切符は支払った金額の範囲で電車に乗車することを許可されたものですが、誰が使うか?は決められていないため切符の購入者以外も使用できます。

認証と認可は似て非なる概念なため、まずは基本をきちんと理解しておくことが重要です。

セキュリティと認証・認可

認証と認可は、セキュリティの分野と深い関係性があります。実際に、「OWASP Top10 API Security Risks - 2023」でも半分近くが認証と認可に関するトピックです。関連箇所を簡易的に見てみましょう。

「API1:2023 - Broken Object Level Authorization」は、ユーザーがアクセス権限を持つべきオブジェクトにしかアクセスできないことを検証する機構が壊れていることを表しています。オブジェクトのIDを受け取り、そのオブジェクトに対して何らかのアクションを実行するすべてのAPIエンドポイントは、オブジェクトレベルの権限チェックを実装するべきですが、この機構が壊れていることにより、不正な情報開示、修正、破壊へつながります。

「API2:2023 Broken Authentication」は、認証メカニズムが壊れていることを表しています。たとえば、認証エンドポイントに対して、クレデンシャルスタッフィング攻撃[注1]が有効であったり、署名されていないJWTトークンを受け付けたり、JWTの有効期限を検証しなかったり、

注1) あるサイトから盗んだ有効なユーザー名とパスワードのリストを総当たりで使用し、ログイン試行するサイバー攻撃手法のこと。

Masato Matsuyuki : xxx@lambda-publishing.com

などです。この機構が壊れていることにより、本来のユーザーの代わりに機密性の高いアクションが実行できてしまうなどの問題へつながります。

「API3:2023 Broken Object Property Level Authorization」は、ユーザーがアクセスしようとしている特定オブジェクトのプロパティに対する認可制御が壊れていることを表しています。本来、ユーザーがアクセスするべきではないオブジェクトのプロパティの参照や更新が行われることで、データの漏洩（ろうえい）や損失、破損を引き起こす可能性があります。

「API5:2023 Broken Function Level Authorization」は、ユーザーに対する機能レベルの認可制御が壊れていることを表しています。一般ユーザーが管理ユーザー用のエンドポイントへアクセスが可能であったり、HTTPメソッドの変更（例：GETからDELETE）によって機密性の高い処理が実行できたりしてしまうと、データの漏洩、損失、破損につながる可能性があります。

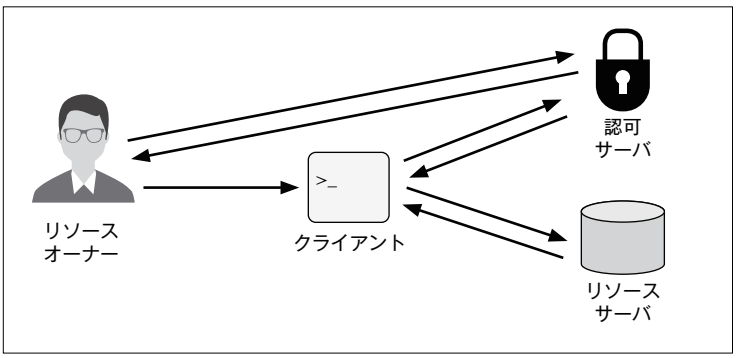
いずれのケースも車輪の再発明をせずに標準仕様を正しく使い、適切な認証・認可メカニズムを実装することが重要だと感じています。

Web APIにおける認証と認可の標準仕様

Web APIの文脈で認証と認可の仕様として今も広く使われているのは、OpenID Connect 1.0、OAuth 2.0[注2]という標準仕様です。以降で簡単に紹介します。

まず、OAuth 2.0は認可を扱うフレームワークであり、RFC 6749[注3]で定義されています。

▼図1　OAuth 2.0のアクターの関係



アクターとしては、次の4つが挙げられます。

- リソースオーナー：保護対象のリソースへのアクセスを許可できるエンティティのこと
- リソースサーバ：保護されたリソースをホストし、アクセストークン（※後述）を用いたリソース要求を受け入れ、応答できるサーバのこと（Web API）
- クライアント：リソースオーナーの代わりに保護対象のリソースに対してリソース要求を行うアプリケーションのこと
- 認可サーバ：リソースオーナーを正常に認証して許可を得た後に、クライアントに対してアクセストークンを発行するサーバのこと

それぞれの関係は**図1**のようになっています。

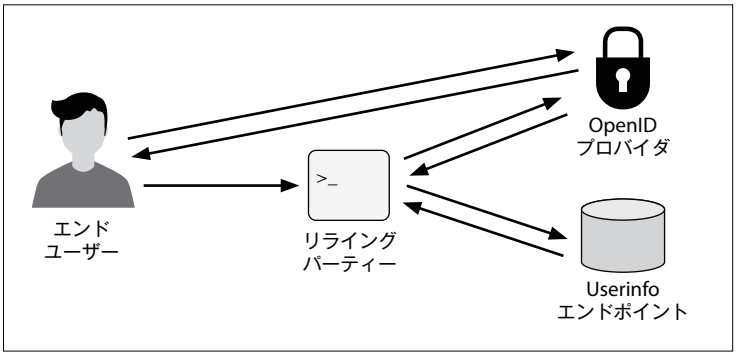
認可サーバから発行されるアクセストークンの実装方法は、RFC 6749で定められていませんが、大別するとトークンが認可制御に必要な情報に対する一意な識別子となっている場合と、トークン自身に認可制御に必要な情報が含まれている場合に分類されます。いずれの場合もトークンの発行主体、発行対象、有効期限やどのリソースに対してどのような操作が許可されているのか？といったことを確認することでクライアントからリソースサーバに対する認可制御を実現します。

次に、OpenID Connect 1.0についてです。OpenID Connect 1.0は、OAuth 2.0をベースに安全にID連携する仕様を定めたもので、アク

注2）現在、OAuth 2.1という今あるOAuth 2.0関連仕様を整理し、1つの仕様としてまとめなおしているものが存在しますが、現在はドラフト版であるため引き続き広く使われているOAuth 2.0を題材に取り上げています。

注3）URL https://datatracker.ietf.org/doc/html/rfc6749

▼図2　OpenID Connect 1.0のアクターの関係



ターとしては、次の4つが挙げられます。

・エンドユーザー：認証の対象となるユーザーのこと
・UserInfoエンドポイント：保護対象のリソースの1つで、アクセストークンを提示するクライアントに対して、エンドユーザーの情報を提供するAPIエンドポイントのこと
・OpenIDプロバイダ：エンドユーザーを認証可能なOAuth 2.0認可サーバのことで、エンドユーザーの認証イベントに関する情報をリライングパーティーに提供する
・リライングパーティー：OpenIDプロバイダが認証した結果（IDトークン）を受け取り、検証を行うOAuth 2.0クライアントのこと

それぞれの関係は**図2**のようになっています。

OpenIDプロバイダからリライングパーティーに対して発行されるIDトークンの実装は、アクセストークンと異なりエンコードの方法や含める情報が標準化されています。たとえば、トークンの発行主体、発行対象、有効期限、エンドユーザーの識別子などです。OpenID Connect 1.0では、これらの認証イベントに関する情報（いつ、誰が、誰を、誰のために、など）をリライングパーティーで検証し信頼することでID連携を実現します。

ところで、OAuth 2.0やOpenID Connect 1.0にはセキュアに使うためのプロファイルや拡張仕様が数多く存在します。たとえば、認可コードの横取り攻撃への対策を目的としたRFC 7636 - Proof Key for Code Exchange by OAuth Public Clients[注4]（PKCE）や金融サービスを始めとした高度なセキュリティ要件が求められる分野向けのOAuthセキュリティプロファイルであるFinancial-grade API Security Profile（FAPI）、OpenIDプロバイダによるエンドユーザーのログアウトをリライングパーティーから要求できるようにするための拡張仕様であるOpenID Connect RP-Initiated Logoutなどです。このような認証と認可の標準仕様を安全に実装し、かつ拡張仕様なども組み込むためにどのような手段があるのか？を以降で説明します。

認証・認可の組み込み方

前項のようなOpenID Connect 1.0、OAuth 2.0ベースの認証・認可のしくみをWeb APIへ組み込む方法は大きく分けると2つあります。1つめは、OpenIDプロバイダ／認可サーバを自分たちで実装する方法です。要件に合わせて柔軟にカスタマイズが可能であったり、最新の拡張仕様も実装すればいち早く取り込むことができたりするというメリットがありますが、安全に実装するために標準仕様に対する深い理解が必要であったりとなかなかハードルが高いです。2つめは、OpenIDプロバイダ／認可サーバを実装したプロダクトやサービスを用いる方法です。拡張仕様やセキュリティプロファイルへの追従はプロダクトの開発状況に依存しますが、標準仕様を熟知している開発者の実装したプロダクトを簡単に使うことができるというメリットが得られます。今回は、2つめの方法例とし

注4）URL https://datatracker.ietf.org/doc/html/rfc7636

Masato Matsuzaki : mzx@lambda-publishing.com

てKeycloak[注5]というプロダクトを紹介します。

Keycloakは、モダンなアプリケーションやサービスのためのオープンソースなIdentity and Access Managementソリューションであり、CNCF[注6]ではIncubatingプロジェクトとして登録されています。開発者が実装する必要のあるセキュリティ機能の提供を目的としており、DockerやKubernetes、OpenJDKとさまざまなプラットフォームでの動作をサポートしています。おもな提供機能は次のとおりです。

- OpenID Connect 1.0、OAuth 2.0、SAMLといった認証・認可の標準プロトコルのサポート
- GoogleやGitHubなどを用いたSocial Login
- Google AuthenticatorやFreeOTPを用いた2要素認証
- カスタムフローによるユーザー独自の認証フローなどの作成

これに加え、Keycloakは拡張仕様やセキュリティプロファイルへの追従にも積極的であり、PKCEやRP-Initiated Logoutなどの拡張仕様への対応はもちろんのこと、高度なセキュリティプロファイルであるFAPIにも対応しています（※FAPI 2.0は現在最新版である26系ではドラフト版のサポート）。

今回は、オープンソースなIAMソリューションであるKeycloakを取り上げましたが、OktaやAuth0を始めとしたIDaaS（Identity as a Service）と呼ばれるサービスの利用も認証・認可基盤の開発、運用負荷軽減につながる1つの選択肢となります。実際に選定する際は、開発、運用チームのスキルセット、標準仕様やその拡張仕様、セキュリティプロファイルへの対応状況などを鑑みて検討することが重要でしょう。SD

注5）URL https://www.keycloak.org/

注6）Cloud Native Computing Foundationというコンテナ技術の推進と、その進化を取り巻くテクノロジー業界の足並みをそろえるために設立された財団のこと。

Software Design［別冊］ 技術評論社



大竹 章裕、瀬戸口 聡、ほか 著
B5判／160ページ
定価2,178円（税込み）
ISBN 978-4-297-13354-2

大好評発売中！

今さら聞けない
暗号技術
&
認証・認可
Web系エンジニア必須のセキュリティ基礎力をUP

今やWebシステムは社会や経済を支える基盤となっており、Webシステムの開発・運用に携わるITエンジニアはセキュリティの技術の理解が欠かせません。暗号技術、認証・認可にかかわる基礎教養と具体的な規約・プロダクトをこの1冊で学べます。公開鍵暗号、共通鍵暗号、ディジタル証明書、電子署名、認証・認可などの基礎技術の用語や理論の説明から、それらを応用したSSL/TLS、SSH、OAuth、OpenID Connectなど各種の規約やプロダクトの使い方までを解説します。

こんな方におすすめ
・Webシステムに携わるエンジニア
・セキュリティに携わるエンジニア

Masato Matsuwaki : fxy.x@lambda-publishing.com

表紙・目次デザイン　トップスタジオデザイン室（轟木 亜紀子）
記事デザイン　トップスタジオデザイン室
マップス（石田 昌治）

■お問い合わせについて

本書に関するご質問は記載内容についてのみとさせていただきます。本書の内容以外のご質問には一切応じられませんので、あらかじめご了承ください。
なお、お電話でのご質問は受け付けておりませんので、書面またはFAX、弊社Webサイトのお問い合わせフォームをご利用ください。

〒162-0846　東京都新宿区市谷左内町21-13
株式会社技術評論社 第５編集部
『Software Design別冊
Web API開発実践ガイド』係

FAX　03-3513-6173
URL　https://gihyo.jp/book/2025/978-4-297-15062-4

ご質問の際に記載いただいた個人情報は回答以外の目的に使用することはありません。使用後は速やかに個人情報を廃棄します。

ソフトウェアデザインべっさつ
SoftwareDesign 別冊

Web API 開発実践ガイド

ウェブ　エーピーアイ　かいはつじっせん

――REST/gRPC/GraphQL からテスト、セキュリティまで

レスト　ジーアールピーシー　グラフキューエル

2025 年 8 月 23 日　初版　第 1 刷発行

著者　杉本 和也、津郷 晶也、太田 佳敬、武田 大輝、宮崎 将太、福岡 秀一郎、武上 将樹、渋川 よしき、真野 隼記、小谷 優空、和田 祐汰、浦 優太、金井 祐樹、内山 高広、上原 拓也、sumiren、川崎 庸市、こたうち さんさん、石川 朝久、徳丸 浩、松本 隆則、川村 修平
発行者　片岡 巌
発行所　株式会社技術評論社
東京都新宿区市谷左内町 21-13
電話　03-3513-6150　販売促進部
03-3513-6170　第５編集部
印刷／製本　港北メディアサービス株式会社

定価はカバーに表示してあります。

本書の一部または全部を著作権法の定める範囲を越え、無断で複写、複製、転載、あるいはファイルに落とすことを禁じます。

©2025　技術評論社

造本には細心の注意を払っておりますが、万一、乱丁（ページの乱れ）や落丁（ページの抜け）がございましたら、小社販売促進部まで送りください。送料負担にてお取り替えいたします。

ISBN 978-4-297-15062-4 C3055
Printed in Japan

Masato Matsuwaki : fxy.x@lambda-publishing.com

Gihyo Digital Publishing
https://gihyo.jp/dp

本コンテンツは，Gihyo Digital Publishing において，
電子データとして再発行されたものです。

電子初版発行日　2025 年 08 月 18 日
最 新 版 発 行 日　2025 年 08 月 18 日
電子版ISBN 978-4-297-15063-1